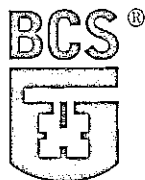


VECTOR

Dyalog APL at LO-Skolen:

- | | |
|-----------------------------------|-----|
| • Meeting reports and pictures | 25 |
| • Walter Fil on APL to C# | 38 |
| • Morten on Oops for APLers (1) | 56 |
| • Phil Last on Lucidity | 72 |
| • McDonnell finds Belgian Numbers | 96 |
| • Pair Programming with the Users | 128 |



*The Journal of the
British APL Association*

A specialist Group of the British Computer Society

ISSN 0955-1433
www.vector.org.uk
Vol.22 No.1 November 2005

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2005–2006

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: gill@apl385.com.

Contents

		Page
Editorial: The Wind from Denmark	Stephen Taylor	2
RESPONSES		
One thir dex cubed?	Norman Thomson	4
NEWS		
News from Sustaining Members		5
Product Guide	Gill Smith	8
Meeting of NYC JUG, June 2005		
Meeting Report	Devon McCormick	20
Signature Tagging, Fun with Pascal's Triangle		22
Dyalog APL Users Meeting, Elsinore, October 2005		
Meeting Report	Adrian Smith	25
My Experience at the Dyalog Conference	Horacio Alejandro Tovar	33
DISCOVER		
The Causeway APL to C# Translator	Walter Fil	38
Using Delegates in an Interpreted Language	Richard Smith	49
LEARN		
Object Oriented Programming for APLers (1)	Morten Kromberg	56
Lucidity Through Infix Notation	Phil Last	72
Using DyalogAPL based .NET extensions in ASP.NET applications	Pertti Kalliojärvi	77
At Play with J: Belgian Numbers	Gene McDonnell	96
J-ottings 45: A bit of OUTReach	Norman Thomson	102
PROFIT		
ARQUE Assessment Reports on the Quality of University Examinations	Alan Sykes	109
Statistical Methods Discovered by the Power of APL	Peter-Michael Hager	115
Pair Programming With The Users	Stephen Taylor	128
Index to Advertisers		143

Editorial: The Wind from Denmark

Insight Systems ApS marked their acquisition this year of Dyalog APL by hosting the Dyalog APL User Meeting in Elsinore. The autumn wind from Scandinavia blows cold and clear; this issue of Vector is dominated by reports and papers from Hamlet's home town.

Perhaps the biggest news for Dyalog users is the arrival of native support for classes and instances, the foundation stones of object-oriented programming. Readers new to the APLs, possibly unaware there are useful alternatives to OO programming, may find it striking to see so fundamental an OO construct added to APL at such a late date. (They should take this as an indication of just how much can be done without it.)

Object-oriented programming (OOP) and functional programming (FP) are not rivals but complements to each other. Nothing beats OOP for managing complex collections of states, most especially GUIs; nothing is as succinct as FP for describing data transforms. Most complex software can be viewed as an OOP layer encapsulating an FP layer. Morten Kromberg of Insight Systems begins in this issue a series on "OO for APLers".

One of the drivers towards object orientation has been the access it provides to processes outside APL. In the Windows world, .NET is a key interface, and Pertti Kalliojärvi writes about using APL behind ASP.NET. The .NET interface not only provides a way to expose processes written in APL for use elsewhere; it provides Richard Smith the components for his own APL interpreter, Rowan. In this issue, Richard discusses how the use of delegates simplifies construction of the interpreter, and Walter Fil reviews the work of Richard's father Adrian in exploiting .NET to write an APL to C# translator.

Fresh from a napkin at the conference comes Phil Last's advice on infix alternatives to control structures. Control structures make for greater clarity than the universally deprecated branch arrows; but can still seem laborious compared to the deftness of an infix

```
(age<65) accept ELSE reject application
```

Deft writing is a theme of my own report on "Pair Programming with the Users", in which I describe how traditional APL development techniques and careful writing have enabled us at Britain's largest life and pension office to outperform even Extreme Programmers. My talk, recapitulated here, covers some themes I shall expand on in coming issues.

The APLs have always been tools of choice for analysing large sets data. Alan Sykes discusses his use of APL for assessing the quality of university examinations, and the colourful Peter-Michael Hager shares a simple and useful method for spotting exceptional results in large arrays. (We can't afford to reproduce in print the subtle use of pastels in his displays, but you can see them online in their full glory.)

Congratulations to J Software on their new jsoftware.com website, and in particular on the J Wiki, a valuable and growing repository of documentation, code, and essays.

Quick Reference Diary

Date	Venue	Event
Mar 30-31 2006	Helsinki	FinnAPL
Oct 16 2006	Denmark	Dyalog 2006 Conference

Dates for Future Issues of VECTOR

	Vol.22 No.2	Vol.22 No.3	Vol.22 No.4
Copy date	in press	in press	3rd March
Ad booking	in press	in press	10th March
Ad Copy	in press	in press	17th March
Distribution	Jan 2006	Jan 2006	May 2006

(Ken Iverson edition)

I am sure you have heard of the show Desperate Housewives, well I feel like I am in my own TV show, Desperate Recruiter. I am writing you because you were listed as an influential member of the APL community, I have no where else to turn.

I have a position for a Sr. APL Programmer and I am willing to give anyone who refers me someone that I place \$3000. The position is in Jersey City, NJ and is for a financial company so financial experience is helpful. Knowledge of Dyalog APL and PostScript a plus as well.

This is a permanent position.

Thank you in advance for your help, it is most appreciated and hopefully it can profitable for both of us.

Tina Pabon
Bendyl Solutions LLC
Executive Recruiter
Tel: +1 973-580-8087 Email: BendylLLC@aol.com

RESPONSES

One thir dex cubed?

from Norman Thomson (ndt2@tutor.open.ac.uk)

Stephen's story about the APL T-shirt reminds of the following yarn which still makes me chuckle :

Two male mathematicians are in a bar. The first one says to the second that the average person knows very little about basic mathematics. The second one disagrees, and claims that most people can cope with a reasonable amount of maths. The first mathematician goes off to the washroom, and in his absence the second calls over the waitress. He tells her that in a few minutes, after his friend has returned, he will call her over and ask her a question. All she has to do is answer "one third $\times$ cubed". She repeats "one thir-dex cue?" He repeats "one third $\times$ cubed". Her: "one thir dex cubed?"

"Yes, that's right", he says. So she agrees, and goes off mumbling to herself, "one thir dex cubed...".

The first guy returns and the second proposes a bet to prove his point, that most people do know something about basic maths. He says he will ask the blonde waitress an integral, and the first laughingly agrees. The second man calls over the waitress and asks:

"What is the integral of x squared?"

The waitress says "one third $\times$ cubed" and while walking away, turns back and says over her shoulder "plus a constant!"

NEWS

News from Sustaining Members

Dyalog Ltd

First, a big Thank You to everyone who came to our conference in October, and in particular to the speakers who made it such an interesting event. Feedback on the venue has been very positive, and as a result we think we will hold the next conference at LO-skolen, during the week which begins on Monday October 16th. As APL2000 did this year at the Naples conference, we will add at least one, possibly 2 "All APL Days" to the conference, where users and vendors of other APL systems will be invited to join us and share experiences, tools and techniques of interest to the array language community. We aim to run a series of workshops on a wide variety of topics, from the latest features of Dyalog APL to techniques for interfacing APL with Office products and other external tools. We aim to make this the APL networking event of the year, and hope to break the "100 barrier" for participants in 2006 (we were one short this year). Set the week aside in your calendar today and watch this space for further announcements!

Free Educational Licenses

Dyalog is pleased to announce the availability of free licenses for Educational use. If you would like to use APL to teach any subject, or use APL in a project at any level of education, contact gc@dyalog.com. Recipients of educational licenses must sign an agreement whereby they promise not to use the software for commercial projects, not to copy the software – and are also required to report to Dyalog once a year on their use of the product. Dyalog APL is also available to pensioners and for other not-for-profit use under similar arrangements. Printed documentation must be purchased separately for free licenses.

APL Summer Camp for Teenagers

We are looking into running a week long APL summer camp for teenagers. The very rough idea is to have APL lessons/workshops in the mornings and outdoor activities in the afternoons. We think the best time for the camp is in the second half of July, and we are looking for a place near Toronto, Canada. If you know a teenager who might be interested in spending a week with a dozen or so other "APL kids", please contact mkrom@dyalog.com as soon as possible! Parents are welcome to come along too.

SQAPL for Unix

Microsoft's Open Database Connectivity interface for SQL databases – known as ODBC – is becoming a standard for database access under Unix(!). Insight Systems is currently

working on porting version 5.0 of SQAPL, which will be shipped with Dyalog APL version 11.0 for Windows, to Solaris. The SQL interface will subsequently be made available under the other Unix platforms where APL is available. Under Unix, SQAPL will not be bundled with APL, but sold separately. If you are interested in database access from Dyalog APL under Unix, please get in touch.

Version 11.0 Availability

Finally, the bad news: We were hoping to ship version 11.0 before the end of 2005, but it has taken a little longer to wrap everything up. We are currently expecting to be able to make the product generally available around the end of January.

MicroAPL Ltd

The Windows version of APLX64, MicroAPL's new 64-bit version of APLX announced in Vector 21.4, is now ready for beta testing. We are looking for further beta testers, so please contact us if you would like to participate in this programme. To run the software, you will need a PC with an AMD 64-bit processor (Athlon, Turion, or Opteron), or an Intel Xeon with EM64T or Pentium 4 with EM64T. You will also need to install Windows XP Professional x64 Edition (currently available in a free trial version from Microsoft).

Meanwhile, we are introducing a new low-cost student edition of APLX, and we have also issued a maintenance release (3.0.9) of APLX for Windows, Linux and MacOS. This fixes a number of minor problems. It is a recommended upgrade for existing users of APLX Version 3, who can download it free of charge from our website.

Kx Systems

A new kdb+ White Paper is available on kx.com. With tick data volumes continuing their out-of-sight escalation, an end-to-end high-speed database is more essential to competitive advantage than ever. To get the details on how kdb+ offers a unified view of streaming, in-memory and historical data, please download the new kdb+ white paper.

Kx and Intel co-sponsor Realtime Data event in London

In October Kx and Intel co-sponsored a seminar on "Trends in Real-time Data" organized by Lepus, a consultancy specializing in financial services research. Lepus is currently preparing a white paper on the topic. If you'd like us to notify you when it is available, please contact us.

New kdb+ applications from *First Derivatives*.

FD now offers three new applications for kdb+. QDBA is a database administration and development tool that provides a development interface to the power of kdb+. Kdb+fix offers a framework supporting client applications for different sell-side FIX engines. Program Trading Platform provides a set of tools to support algorithmic trading. For more information contact Michael O'Neill – moneill@firstderivatives.com.

Causeway Graphical Systems Ltd

The Dyalog APL conference left me with two strongly conflicting emotions: pride in the apparent pervasiveness of much of our APL utility software, and concern about the degree of dependence of so many successful APL-based products on it (and by implication, ourselves). Let's take a very brief survey of the programme, looking specifically at the business-oriented presentations.

Hologram (from John Miller) is end-to-end *CPro* for the Gui, with *NewLeaf* reports and *RainPro* charting. John and Guy would be positively shocked at the thought of programming the Dyalog Gui directly.

The PX-Edit Statistical Table Editor (from Statistics Finland) uses *RainPro* for graphs. As far as I know, that is the end of it.

X2.Net – for TakeCare. A long-term *NewLeaf* customer, this one.

APL-based .NET Extensions (from Pertti Kalliojärvi) – it seems they chose Dyalog because they needed SVG output from a graphics engine, and the only decent one they could find was *RainPro*! Enough said.

ARQUE (from Alan Sykes) is written in *CPro*, with *RainPro* graphics and *NewLeaf* reporting. I don't think Alan would know a 'ff' □WC 'Form' if it emerged from Swansea bay one dark night and bit him!

Assembly and Morning Song (from Michael Baas) was the kind of little Gui-based utility that experienced *CPro* users can whip up in a day. Mike has been coding in Causeway from the beginning, and is responsible for most of the 'cool' stuff (like clicking on data-points and running callbacks) that Alan Sykes was exploiting so effectively in *ARQUE*.

Which is a remarkable hit-rate. *RainPro* and *NewLeaf* are no great worry for me. They are both shipped as open code and really don't do anything very clever. *NewLeaf* in particular just takes away the mind-numbing tedium of report-formatting. They work fine in Dyalog 11, and are likely to keep on working into the indefinite future. One day soon, I can start encouraging my users to work with *SharpPlot* (and whatever the .Net build of *NewLeaf* is called) which are smaller, faster, and use the .Net tools directly to make much better raster output.

CPro is another animal entirely – it started back in 1993 (before Namespaces) and prospered as a way to superimpose a rational object-oriented model on the original Dyalog Gui. It is a very productive development tool, and I still like it a lot. But I am seriously concerned that Dyalog 11 may (quite reasonably) leave people wanting more. They may want direct access to the .Net Gui, they will probably expect Causeway classes to behave like the native V11 classes, and so on. This may just be my natural paranoia showing through, but I would ask all our users to start thinking sooner rather than later about what comes after CausewayPro. It won't be a 2-minute job to rebuild it.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADVOCORP Oy	APL+Win, APL+Link, APL+Linux, APL+Unix, APL+Plus Sharefile	poa	Complete APL2000 and Statgraphics(StatPoint) product range and links to various third party products.
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000 APL interpreters and toolspoa		Distributor of APL2000 products for the UK Consulting and maintenance for APL applications
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS.Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Download from IBM APL2 ftp site or contact APL Products.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-B74
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL2000	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zankner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only

Books and accessories			
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
	APL*PLUS	poa	Manugistics
	APL 68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and JSoftware products USA	poa	
	Dyalog and JSoftware products	poa	
ubJL GmbH (APL Software Team)	Dyalog APL and Dyalog Systems Ltd. Products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Acadvent	ARQUE	poa	Software for the quality assurance of university examinations.
APL Software Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick_holt@email.com.
Assured Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only)

Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability. Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
	SharpPlot for Microsoft .Net	500	<i>RainPro</i> rebuilt as a fully-managed .Net DLL for use by APL and non-APL applications alike. May be an attractive alternative for Dyalog 11 and Santa Fe users, as it has much improved image support as well as running rather faster, and not cluttering your workspace with APL sourcecode.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/KCoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Dynamic Logistics Systems GmbH	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.

	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "lab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>			
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravellLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education, APL tutor news and hotline phone support.

APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADVOCORP Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API level interfacing to any system under Windows, TCP/IP network and database connectivity, APL based financial client/server applications, Cognos Planning and ReportNet consultancy.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+W/in, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
Assured Systems	Consultancy	poa	APL+Win and/or J and/or VB custom systems development. Conversion of APL+DOS to APL+W/in a speciality.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Ray Cannon	Consultancy	500/day +VAT	APL, C, C++, Assembler, Windows, Graphics on PC and IBM Mainframe. Experience in Insurance, Chemical, and Airline Industries
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evastic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivatives plc	Consultancy	poa	Financial trading software in Q, K and kdb+
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work. full members of DSDM consortium and Agile Alliance.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C#, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP

APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.

KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambert Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
Alastair Kinloch	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming
Ellis Morgan	Consultancy	poa	Business Forecasting & APL Systems.
Nussbaum gift	Consultancy	poa	IT Consultant with a strong focus on APL.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT1 customisation and issues concerning the internet and/or web design.
Graeme Robertson	Consultancy & Training	250-500	Design and write custom software. Maintain and upgrade APL systems. Deliver customized APL training courses
Skelton Consulting	Consultancy	poa	K/Q Consultancy
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms 17-23 years experience.

	MACfns	MACfns has been released. A suite of over 100 high-accuracy, high-speed assembler code utility functions for APL+Win and APL+DOS. MACfns has been thoughtfully designed and implemented for APL programmers. It includes functions for segmented string manipulation, data restructuring, translation and datatype adjustment, and many numerical computations. Typical speedups range from 4-10 times, with reduced workspace usage, datatype conservation, identity detection, and greater numerical precision amongst the other benefits. Documentation of MACfns is extremely detailed and thorough.
ubJL GmbH (APL Software Team)	poa	Analyses, design and programming.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification. APL unicode fonts designed and built for all flavours of APL.
APL Borealls Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL Interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email acam@blueyonder.co.uk.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm) \$69.95 (edu)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint. Vox Proxy for educational use.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join Material fee 5,000yen a year
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings	\$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
Acadvent Ltd.	Dr A M Sykes	171 Gower Road, Swansea SA2 9JH, UK. Tel: 01792-201776 Email: Alan.Sykes@which.net

ACM SigAPL		ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADVOCORP Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: adrian@apl385.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: curtis_jones@prodigy.net
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828 Fax: (416) 482-6582 Email: info@aplborealis.com
APL Consultants Japan	Kyosuke Salsuga	1507 shimomizo, sagami-hara, kanagawa, Japan 2290015 Tel: +81-042-778-2127 Fax: +81-042-778-2205
APL Germany e.V.	Dr. Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany Email: reiner@nussbaum-gift.de
APL Italiana S.p.A.	Stefano Lanzavecchia	via Monferrato 1, 20144 Milano, Italy. Email: stf@apl.it
APL Software Services	Dick Holt	1570 Heather Field Lane, Earlysville VA 22936 USA. Tel: +1 434-973-4255 email: dick_holt@email.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
Assured Systems	Lols & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hllrj@melbpc.org.au
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
Beautiful Systems, Inc.	Jim Goff	PO Box 2235 Jenkintown, PA 19046, USA Tel: +1 (215) 635-0375; Fax: +1 (215) 635-9212 Email: info@beautifulsystems.com
Ray Cannon	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
The Carlisle Group	Paul Mansour	544 Jefferson Avenue, Scranton PA, 15110 USA. Tel: +1 570-963-2036
Causeway Graphical Systems Ltd	Adrian Smith	Brook House, Gilling East, YORK, UK. Tel: 01439-788413 Email: adrian@causeway.co.uk
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33 4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Ditrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dyalog Ltd.	Morten Kromberg	Grove House, Lutyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-338461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
Dynamic Logistics Systems GmbH	Michael Baas	Wilhelm-Schöffer-Str. 29, 63571 Gelnhausen Germany. Tel: +49 6051 13067 Fax: +49 6051 16142. Email: info@dls-planning.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6548 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivatives plc	Michael O'Neill	First Derivatives House, Kilmorey Business Park, Kilmorey Street, Newry, Co. Down, N. Ireland BT34 2DH. Tel. 028 3025 2242 Fax 028 3025 2060 Email enquiries@firstderivatives.com
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com

General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gsoft.plus.com
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469, Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028 Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	11 Bolshaya Monetnaya Str., suite 6, St. Petersburg 197101 Russia. Tel: +7 812 325-8797 Fax: +7 812 233-9685 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Japan APL Assoc	Juichiro Takeuchi	Keio Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel: +81(045)563-1411 Fax: +81(045)568-1617 E-mail: takeuchi@ae.keio.ac.jp
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470- 7345 Fax: +1 (952) 470-9202 Email: info@software.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56, 00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: kimmo.kekalainen@kjk-tieto.com
Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com (in Europe simon@kx.com)
Lambert Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lamberttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-833807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Alastair Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)7802 430 202 Email: alastair.kinloch@btinternet.com
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: apl@ellismorgan.co.uk
Dr. Nussbaum gift mbH	Dr Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany Tel: +49 621 7152190, Email: reiner@nussbaum-gift.de
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Qualedi Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083 Email: sales@qualedi.com; info@qualedi.com
Renaissance Data Sys	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxproxy.com
Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@nildram.co.uk
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Skelton Consulting GmbH	Charles Skelton	Weinklinge 9, 70329 Stuttgart, Germany. Email: c.skelton@skelton.de

Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
Soliton Associates		Soliton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@soliton.com
SovAPL	Alexander Skomorokhov	P.O. Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Russian Chapter of SIGAPL		
Strand Software Inc		P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Toronto SIG	Richard Procter	P.O. Box 39, Mansfield, Ontario, L0N 1M0, Canada, email: info@torontoapl.ca
ubJL GmbH		
(APL Software Team)	Kai Jäger	Thusneldastr. 22, 90482 Nürnberg, Germany. Tel + 49 911 482512; Fax: +49 911 482518 Email jaeger@ubjl.de
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2 <ftp.software.ibm.com/ps/products/apl2>

WORLD WIDE WEB SITES

Acadvent Ltd.	www.acadvent.com/
ACM SigAPL	www.acm.org/sigapl/
ADVOCORP Oy	www.rett.fi/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.apl385.com
APL Forum	www.aplforum.com
APL Germany e.V.	www.apl-germany.de
APL Italiana S.p.A.	www.apl.it/
APL Journal, Germany	www.rhombos.de/shop/a/show/article/?216
APL Borealis Inc.	www.aplborealis.com
APL Consultants Japan	http://aplcons.com
APL to ASCII	see Waterloo Archive
Assured Systems	www.assuredsystems.com.au/
AUSCAN	http://home.ca.inter.net/rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
Dynamic Logistics	www.dls-planung.de
Systems GmbH	www.dls-planung.com
FinnAPL	www.pyr.fi/apl/

First Derivatives plc	www.firstderivatives.com
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo-allegro.com/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Nussbaum gift	www.nussbaum-gift.de
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.com/
Right Seat Software, Inc.	www.voxproxy.com
SigAPL	www.acm.org/sigapl/
Skelton Consulting GmbH	www.skelton.de
Snake Island Research Inc.	www.snakeisland.com
Soliton	www.soliton.com/
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.ca/
ubJL GmbH (APL Software Team)	www.aplteam.de
Waterloo Archive	www.math.uwaterloo.ca/apl_archives/Welcome.html
Jim Weigang	www.chilton.com/~jimw/

Meeting of NYC JUG, June 2005

notes by Devon McCormick

Various Contributions

Dan raised an idea not related to our project but interesting in its own right: how can we define J primitives in terms of other J primitives? This might have some pedagogical merit if we can separate the most basic primitives (first learned) from the more advanced ones. We could then define the more advanced ones in terms a novice may already understand. However, Dan seems more interested in this than is the rest of the group. We probably are more interested in developing and publicizing functions that accomplish small, commonly needed tasks.

Jim's work on an indexer is relevant to this latter notion. As we've noted repeatedly, J's richness sometimes makes it difficult to find what we need for a task at hand. John reiterated how deeply difficult it is to automatically create a good index – he bases this on his own extensive experience working with bibliographic data and academic papers.

Jim's preliminary work on an APL/J Rosetta Stone may provide a leg-up for the small community of APL programmers learning J. This points up how useful it is to be able to relate the vocabulary of one language to another. While some aspect of J, such as array indexing, may differ deeply from how this is done in APL, it's a good start to be able to look up the common term.

This again leads us to a "Rosetta Stone" notion of synonyms: simply listing related or equivalent terms could be useful, especially if the synonyms tie into a widely-used language like Basic.

An alternative to either the automatic indexing scheme or the Rosetta Stone is the notion of logical classification. However, this patently requires the most manual labour so is an approach we'd like to put off. Also, one person's logical classification will not necessarily agree with another's. For an unusual and widely discussed notion of classification, see the last section here titled "Classification".

John talked a little more about his method for combining the best of both GUI and command-line interfaces using his "inert" adverb. This takes an argument of a command and either runs it or saves the command for future use, e.g. in a script. He called this the J analogue of the "command pattern".

John provided an example of using the “inert” adverb – see *copy.ijs* on the Vector website.

The “Save Work” Project

My “action item” for the this meeting was to come up with a “Vicki” use-case. “Vicki” is our hypothetical, primary target user.

Vicki is a middle-aged, but young-at-heart, married woman who works in arts administration. She organizes a performing arts festival about once a year, usually in a different location around the U.S. She also teaches a class in theater arts, so she needs to keep regular notes and lesson plans for this.

She may also have a play or two of her own that she’s working on from time-to-time, as the inspiration seizes her.

Her two children or her husband may occasionally use her computer: the children mainly for games but possibly for some homework; the husband may also use it to write something for his job or for personal correspondence.

Case 1: Vicki installs the software – what helpful screens does she see? Has anyone worked on an install script?

Case 2: Vicki wants to save all the recent work she’s done, either to a thumb drive, a floppy, or a CD.

Case 3: Vicki wants to archive a group of works that may be in different directories but pertain to the same project, e.g. last year’s class notes or multi-media files relating to a play she’s been working on.

Case 4: Vicki wants to get all the junk off her hard drive that her husband and children have left scattered there over the years but wants to save a copy in case there’s anything that someone may need later.

Case 5: Vicki needs to locate something she saved to a CD a while ago.

Looking at these cases, we debated whether we would need to examine the entire hard disk each time to find files to back up. Dan came up with a list of likely locations where we’d probably find all files modified by a naive (Windows) user. Our preliminary list looks like this:

- Windows executables history
- Run box history
- Application registry history
- Recent Documents

- Application data directories
- Desktop shortcut directories
- Desktop
- My Documents

Does the registry track all most-recently-used files? We have to figure this out and if we can depend on this to be complete.

We'd like to accommodate other platforms besides Windows, so this sort of thing may or may not have analogues in the Mac and Linux worlds. We also came up with the idea of having interval-based classification of backups into something like daily, weekly, and infrequently. The idea is to have pre-set menu options for the most common use cases.

The converse problem to backing up is the potentially much more difficult problem of locating backed-up information. This may be ameliorated by imposing a naming convention on backups or by maintaining a database of items backed up.

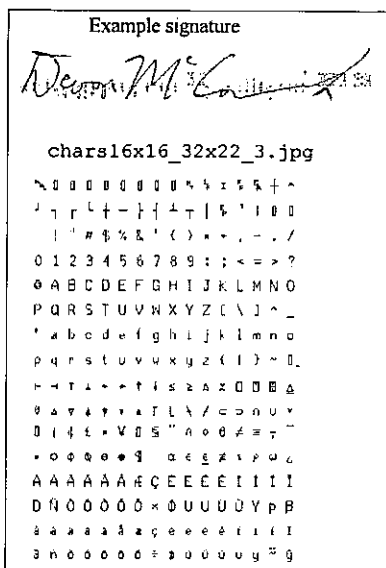
An additional consideration is how to handle multiple target devices to store backups. I believe John suggested the possibility of somehow aliasing different devices.

Miscellaneous

We also discussed a few off-topic notions. One was the idea of tagging bitmaps of hand-written signatures with unique, identifying information. This was generally dismissed as not too useful. One somewhat interesting issue this raised was how to write text programmatically onto an existing image. See *sigtag.ijs* on the vector web site.

Example of Using "SigTag"

```
qts ''
2005 6 14 15 31 5.964
load '~user/code/sigtag.ijs'
flsz=. createNewSigBMP
'\amisc\ainfo\NYSSA_Dues_Renewal_05_06DHMFilledIn.doc 20050614
15:34';'\amisc\my thin blue
signature.bmp';'\amisc\ainfo\sigsig0506141534.bmp'
```

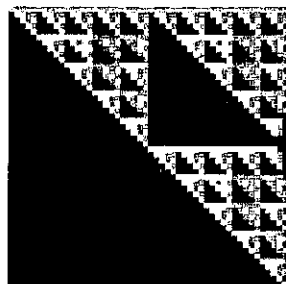



Another thing I brought up, not as off-topic because it relates to learning about math and computers in general, was inspired by the cover of Howard Peelle's new J book where he lists functions with old, familiar names like "pythagoras" and "pascal".

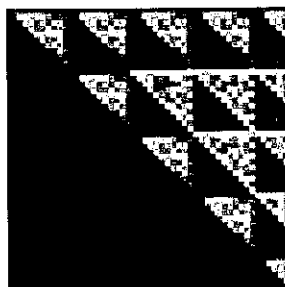
Fun With Pascal's Triangle

```
Pascal =: i. !/ i.

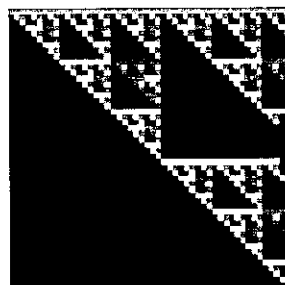
Pascal 51
1 1 1 10 1 2 3 4
0 0 1 3 6
0 0 0 1 4
0 0 0 0 1
load 'viewmat'
$pt50=. Pascal 50
50 50
viewmat 5|pt50
load 'Logo'
'save*' names 3
saveBMPFL
saveBMPFL
'C:\amisc\j\nycjug\Pascal50mod5.bmp'
```



```
viewmat 11|pt50
saveBMPFL
'C:\amisc\j\nycjug\Pascal50mod11.bmp'
```

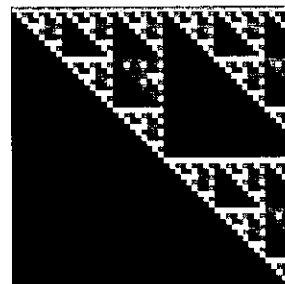


```
viewmat 3|pt50
saveBMPFL
'C:\amisc\j\nycjug\Pascal50mod3.bmp'
NB. Why does it look "broken" on the
right?
>./,pt50
6 .3205303e13
/::~,pt50
0 0 0 0 0 0...
\::~,pt50
6.3205303e13 6.3205303e13 5.8343357e13...
3|10{.\\:,pt50
```



```
0 0 0 0 0 0 0 0 0 0 NB. Aha! Looks
suspicious.....
```

```
$pt50=. Pascal 50x
50 50
\\:,pt50
63205303218876 63205303218876...
3|10{.\\:,pt50
0 0 0 0 1 1 1 1 0 0
viewmat 3|pt50
saveBMPFL
'C:\amisc\j\nycjug\Pascal50xmod3.bmp'
```



His Pascal's triangle function led me to revisit some interesting topics that this touches on, especially the ideas of colouring different modulus bases of the triangle and degradation of precision for large values.

Dyalog APL Users Meeting, LO-skolen, Elsinore, October 2005

reported by Adrian Smith (adrian@apl385.com)

Location

Architecture and modern art, sun, sea, woodlands, knife-throwing, 9-pin bowling ... need I go on? This was Denmark at its best, and almost everyone there really enjoyed themselves. LO-skolen was built by the Danish Trades Union movement in the 60s, as a showpiece of all that is most characteristically Danish. The whole place is plain, simple, beautifully crafted out of good materials and just feels 'right'.



This picture was taken from the garden, which runs down to the little railway, and then to the sea.

The view from the breakfast table was lovely, with Sweden in the distance across the blue sea. It almost made you want to get up in the mornings! I think Morten and Gitte are set on returning to the same venue next year, and it is hard to see anyone objecting.

This was also the setting for the conference banquet, which has to be rated as one of the best ever – on a par with APL88 for those who remember Sydney. The entertainer was a very professional magician, who pitched the show just right for a bunch of nerds who were really very interested in how the tricks were done. I was sandwiched between Richard Smith (who escaped from Cambridge for the

duration) and Brooke Allen who both spent the rest of the evening testing out card tricks on anyone they could find. I had the honour of being picked as the stooge for the 'cut the rope' trick – and in spite of knowing quite well how this is done, I can report that I was completely taken in, even from 2 feet away!

Conference Highlights

In no particular order (personal choice, of course) ...

- John Scholes on language extensions. John's usual low-key style was the perfect foil to some really exciting new stuff. The 'power' operator is a great addition to the language, and the ability to raise functions to power(-1) may look like a toy, but has got to have some really useful applications.
- Stephen Taylor on Agile development. I do enjoy speakers who can hold an audience with minimal notes and no PowerPoint.
- John Miller on Hologram. Slicing and dicing made all the more powerful by being a very thin cover on APL. I know we have seen dozens of these before, but this one actually looked like a professional product with a properly engineered Gui.
- Discovering an unexpected personal talent for knife-throwing.

Day-1 Wednesday October 19th



Welcome and overview of Version 11

Gitte and Morten

Gitte welcomed us (all 99 of us) and took us through the new logo, the new colour (orange, for those reading this in black & white), and the new ownership structure. Gitte is a shrewd businesswoman and I get the strong impression she knows where this ship is headed and will make very sure it stays profitable. I think the Insight team may have taken control at a very good time for the company, and that from now on the only way for Dyalog Ltd is up. A personal opinion, it's true, but reinforced by the feeling of a strong surge of support from everyone present.

Morten took us very quickly through the up-coming features in the new release. Core language extensions are the *power operator*, *squad indexing*, *least common multiple* and *greatest common divisor* (not too sure about the last two, but I guess

they are harmless). Morten also touched on the 64-bit version, and on the idea of *classes* which are an attempt to bring APL a little closer to mainstream OO thinking without throwing away all the flexibility that we anarchist programmers have learned to rely on. Oh, one other thing that caught my attention - the ability to have *trigger functions* which fire whenever a variable is changed. *CausewayPro* could make use of these to great effect, and also I could see huge benefits for debugging - now you can see exactly which line of code just trashed your precious data!

Hologram – Intelligent Business

John Miller

I have worked with John on and off over the years, but this was the first time I have seen his new system demonstrated. He does all the usual things (extracting data from huge tables, summarizing it instantly in APL, drawing nice graphs and reports) but with a very scalable architecture and the option of a zero-footprint web front-end. Interestingly, he is happy to interface out to R (basing the interface on Sasha's presentation to FinnAPL 2 years ago) rather than attempting to code up his own statistics library. I liked the look of this system a lot, and I can appreciate the amount of work that has gone into the backend processing, with proper load-balanced servers handling the serious grunt-work.

Version 11 language extensions

John Scholes

```
f ← (32 ÷ 5) × (°F - 32)
f ← 273 40 0 100
c ← 459.4 40 32 212
c ← f × -1
c ← 459.4 40 32 212
f ← 273 40 0 100
```

Yes, the new *power operator* can be a lot of fun. I can see it being used all over the place with arguments of 0 or 1 (e.g. in the obvious 'enclose if simple' utility) but it can do lots of other tricks too.

This example takes something that looks very like a J tacit definition, applies it to some temperatures in Centigrade to get the Fahrenheit equivalent, then applies the inverse function to

reverse the transformation. I can see Phil Last (of napkin fame) having a ball with this one!

Squad indexing is much less fun, in that it simply provides a functional equivalent to the (admittedly anomalous) square brackets and semi-colons which I banned years ago. You can use it to the left of assignment, but not for 'reach' indexing which is one of those nice Dyalog things that the APL2 people never caught up with. However it does work nicely with operators (*each* being the obvious one) so I can see it gradually creeping into my code.

Now why would you want to extend *and* and *or* to cover LCM and GCD on integers? I am really rather unsure about the value of this one, as these are pretty trivial to code as dfns when you need them. I would rather have had a C-style bitwise variant which is much messier to code around in APL, and is going to be very necessary as soon as APL hits the real .Net classes, with expressions like `FontStyle.Bold|FontStyle.Italic` where you *can* get away with adding the flags, but not when you want to test if your own flags are present in some preset combination.

Version 11 object-oriented features

John Daintree

This is far too large a subject for me to cover adequately here. There are things about it that I like a lot, things that I really don't like at all, and things that I am completely unsure about! So to get an argument started, here are some of the things I like:

- You can make an *instance* of a namespace really easily. If a function dies half way through (as APL code always does, particularly when liberally dosed with jots) you just hit ShiftEnter, fix it and carry on running. When the instance goes away, your changes really are saved back in the original namespace. Wonderful. I want this, and I want it now!
- There are lots of new magic words (preceded by colons such as `:Access public`) which give the APLer a smart and simple way to add declarative stuff to functions which can then be saved out as real .Net libraries.

... and here are a couple of things that I feel could really spoil my day:

- Having made your instance, you can assign arbitrary crud into it at any time. This goes right against the normal OO rules, and has the huge downside that simple spelling errors may look like they worked but actually they just assigned a value to a name somewhere. I think this is dreadfully misguided, and I hope that there is (at the very least) a big fat switch that disables it.

- Someone needs to have a hard think about nameclass. Now we have Gui things AND namespaces AND classes AND instances all sharing nameclass 9. Oddly, Classes don't show up in jobs even though they are in the same nameclass. I really think that this is a false economy, and that these are very distinct types of things.

The thing that bothers me most is why we need *two* new beasts here. Surely a Class is just a Namespace with a few extra attributes set and almost nothing else changed. OK, an *instance* is a wholly different animal and should be recognised as such, but there seems to be an awful lot of baggage here to do something fundamentally simple. Probably I am missing something, as usual. Read Morten's *Introduction to OO for APLers* (serialised in this and the next Vector) and form your own opinions.

A keyboard viewer for Dyalog APL

Kai Jäger

This was a nice little toolkit which was basically a DIN editor (a much needed tool in the Dyalog world) but could also be used by novices to locate the more obscure APL symbols on the keyboard. Let's hope it shows someone at Dyalog the way to a simple APL toolbar that attaches to your current edit session and allows anyone (even me) to go mousing for things like *notmatch* which have to be there somewhere!

Is that an APL system in your pocket?

Richard Procter

Dec. 31, 2003 Meal: DINNER

Resident: LAVERNE COOTS Room#: 1A

Diet: Regular

Served Items	%Left	%Left
1 CUP TEA, DECAF	80	100
1 PC APPLE PIE	60	90
4 OZ GINGERALE	60	80
1 CUP CARROTS	30	70
2 OZ MUSHROOM GRAVY	0	60
0.5 CUP APPLE BROWN BETTY	0	50
1 IND PLAIN MUFFIN	50	40
8 OZ GARDEN BEEF STEW	50	30
		20
		10
		0

Up Down

☐ Ate All
☐ Ate None
☐ Tray Lost

Cancel Save

The screenshot says it all, really! Richard is working with the Canadian health service on a system to improve nutritional planning for hospital meals. Of course you can plan all you like, but if the patient throws the tray on the floor then your estimates may be way off the mark!

So bring on the Pocket PC which can be used in the hospital to record what the patients are actually getting to eat, not just what went on the tray in the kitchen.

From Dyalog to pure .Net runtime*Adrian Smith*

As it happens, this followed on really nicely from Richard Proctor's talk, as one of the things I showed was how easy it is to ship a compiled APL application on the pocket PC. I rather enjoyed giving this talk – helpful APLers in the audience kept offering me the laser pointer (and failing that a simple stick) but leaping about trying to indicate stuff on the projection screen is much more fun. See Walter Fil's introduction to the Causeway C# translator (in this Vector on page 38) for a much better summary of what it is, and what it does, than I could ever give.

The PX-Edit statistical table editor*Veli-Matti Jantunen*

This is a nice example of a little piece of utility code taking on a life of its own and spreading way beyond its author's expectations. There is a very well known text format for statistical data called PX (a bit like CSV but with a lot more metadata) so any software which wants to do stats should be able to read it. Then of course it is a good idea to be able to fix errors in it! PX-Edit grew from this simple beginning to a very flexible tool for managing data held in this file format, and now pops up all over the world as part of a huge variety of other statistical tools.

After dinner entertainment – Lists*John Scholes*

OK, John needs to write this one up. I *think* that all the code shown was completely useless (from a practical perspective) unless you suddenly need to implement a LISP system in APL. But it sure made you think, and the few remaining active grey cells did their best to keep up.

Homage to Ken Iverson

This being the first anniversary of Ken's death, we once again ran the film of the 1974 discussion on APL design, and then followed this up with a series of reminiscences and tributes from many of those present who knew Ken well. This struck just the right balance, and was attended by almost everyone, even though the time was pushing towards 11:00pm before it was done.

Day-2 Thursday October 20th

Tutorial – introducing OO to APLers

Morten Kromberg

Morten had the stamina to run this entire show twice in one day. I went to the afternoon rerun, and greatly enjoyed the experience. The advantage of having the workspaces on the conference keydisk was that it was really easy to play along with the presenter, and of course to try a few experiments along the way.

Morten's notes will be serialised in this and the next few Vectors (starting on page 56), so it is probably best to leave my report at that.

Using APL-based extensions in ASP.Net

Pertti Kalliojärvi

Dinosoft chose Dyalog APL for this project because they needed SVG charting, so they had to build on a platform that ran *RainPro*, and could be accessed from ASP. Fascinating – now if we had been able to offer them SharpPlot 2 years ago, maybe they would not have used APL at all? Who knows. See page 77.

The Viking Challenge



Yes, it was Axes again, and blowpipes too, but this time we had pistol-shooting and knife-throwing as well. The weather rather let us down, with some very poor light (not ideal for the blowpipes as it was almost impossible to see what you had hit) and a

bit of rain towards the end of the session. As you can see from the picture, our team managed at least two axes on target.

As before, the winning team got a special version of the tee-shirt. It will be interesting to see the full breakdown of the scores when the Insight team finally manage to decode the rather scrappily written sheets from the various events.

ARQUE – Assessing the quality of university examinations*Alan Sykes*

You can read Alan's paper in full in this Vector on page 109, so just a few comments here. This is yet another example of a user-written system which has escaped from its first home and is now setting out to conquer the world. In Alan's case, he is starting with the market he knows best – the universities of Wales – and then gradually honing the software to make it more generally useful.

Day-3 Friday October 21st**Assembly and Morning Song***Michael Baas*

Just think what your local church could save on hymnbooks and service sheets if it invested in a decent projector and a simple computer system (written in APL, of course). Michael has created a very nice interface which uses the dual-monitor capability of the PC to project the words of any song in his library while the congregation sing along to each verse. Simple stuff, but nice to prove that it worked in practice, with a little help from Paul Mansour on guitar.

Array languages at Morgan Stanley*David Crossley*

Morgan Stanley have a huge investment in array thinking, and array languages. They started with APL, then had A+ written just for themselves, then partially abandoned it, finally moving to license Kdb from the man who they originally hired to write A+ ... and through all this they still have Dyalog APL in there somewhere.

Agile development*Stephen Taylor*

Now this was fun, so much so that I completely failed to take notes and thus absolutely failed to learn anything at all from Stephen's excellent presentation. Except, I suppose that the 'waterfall model' of systems development relies on reliable communication between people, and so is about as much use as a chocolate teapot. Which I knew anyway. Just be patient and keep doing what APLers do anyway – sit beside the user and code what they tell you. As Stephen illustrated over and over again, there is only one way to tackle a complex problem – find an expert user with a bunch of test cases and write code until they all work. Never mind if you can't follow the logic – all that the real world needs is the right outputs from the chosen inputs!

I think this used to be called common sense, until the software engineers got their mitts on it and tried to make software development into a quantifiable discipline. This can work rather well for systems with logical, stable rules, but for a really well argued thesis as to why it invariably fails to match expectations in so many real-world software projects, read Stephen's article in this Vector on page 128.

Wrap Up

This was a tough schedule, with a huge amount of good material and excellent speakers. Most sessions were very well attended, even the early starters on the second day. An amazing number of people made breakfast before 8:00 on all three days, which says a lot for their commitment to the APL cause.

And it was in a very good location. Easy to get to, pleasant to walk around, and gorgeous to look at from almost any angle. Even the bowling alley was an engineering joy. If they are back here next year, do find an excuse to come.

My Experience at the Dyalog Conference

by Horacio Alejandro Cisneros Tovar

When I was about 11 years old, I remember that my APL professor, Dr. Javier Fernández Pacheco told me that, if we kept up the good work with APL, we might make it to a user conference, in Denmark maybe. At that time I thought of it as a dream that would never come true, but I enjoyed the idea for a long while. Two years later I gave a talk for the Mexican APL Society titled 'Horacio, 13 years and APL.' Prof. Fernández organized it and showed how APL was the perfect programming language for educational purposes in early levels. During the talk I presented some basic calculation algorithms, compared iteration against recursion and finally presented a small game I had programmed in APL*PLUS v.4, 'The Boatman' (an animated version of the problem about the boatman that had to take a goat, a dog and a pack of grass to the other side of the river, one by one). We made a very good impression and I was accepted as a member of the Mexican APL Society. But that was all. The Society eventually disappeared when it ran out of money, and so too my dream of meeting other APL users from around the world.

Fifteen years later, I was invited by Dyalog Limited to their annual user conference, to present my thesis project on Computer Engineering, 'An On-Line APL Tutorial' for the National Autonomous University of Mexico. It was a funny feeling. From the moment I requested support for that project, approximately one year ago, all the memories of my happy times with APL came back as if they had happened yesterday. The re-discovery of APL through Dyalog's product which is surrounded now by new and modern implementations along with its integration to current technologies and the Internet, made me feel like the child I was when I accidentally switched an IBM 5110 machine from BASIC to APL.



After almost one year of hard work on my thesis project, I was invited by Morten Kromberg to the conference in Elsinore, Denmark, from October 19th to 21st. Dyalog offered to co-sponsor my trip along with the British APL Association. As I said, it was a mixture of feelings: joy, thankfulness, excitement and also sadness, because

my APL teacher had passed away two years earlier and he was not with me to share this reality that was born as a dream of my childhood, that had been eclipsed when APL disappeared in Mexico and that was reborn again when Pete Donnelly sent me the Dyalog APL/W CD and granted a free licence to develop my thesis project. (APL2000 had still one distributor in Mexico City, but its high price was one of the reasons why the Mexican APL Society disappeared).

Even though this was not the first time I given a talk on an APL subject, it was the first time I had ever left Mexico and also my first time on a plane. This particular experience was terrifying in the beginning, but after 14 hours in the air from Mexico City to Amsterdam, I somehow ended up getting used to the floating floor of the plane. I left Mexico at 22:00 on October 18th and reached Copenhagen at 21:00 on Wednesday 19th, so unfortunately I missed every session of the first day. Bjørn Christensen picked me up at Elsinore train station and thanks to him I made it to the final comments of the Ken Iverson memorial and for the evening snacks. From the very first minute of my arrival at Elsinore and LO-skolen I felt very good even though I was surrounded by strangers in an unknown country. I met Peter Donnelly at LO-skolen's reception and it was like meeting a long-distance good friend. I also met Morten Kromberg, Helene Boesen and John Daintree. I had been

in touch with them months earlier through email so it was very good to finally meet them in person. During the evening snacks I talked with them and that was when I realised that, despite coming from a different country and speaking a different language, I was somehow among my own. We knew the most important and universal language, which was also the powerful reason that gathered us all there: APL. So, as I mentioned in my talk, after working for a long time almost on my own with APL in the years after the Mexican APL Society disappeared and during the development of my thesis project, it was a wonderful feeling to find myself among new people that shared the same fundamental interest in this amazing programming language.

For the next two days I didn't miss a single minute. I attended three presentations in the morning, even though I didn't sleep the previous night. I found the FlipDB presentation particularly interesting, because of the clever way that a database model was conceived using directories for the general structure and the power of dynamic functions to make queries and other operations. Later on that day, I participated in the 'Viking Challenge' which I enjoyed a lot more than I expected and I didn't mind having three embarrassing blank targets. I was the best of my team in the blowpipe along with Stefano Lanzavecchia, so at least I kept the pride of knowing that I have a special talent that may come handy in the rainforest. I enjoyed very much the Viking Challenge despite the rain. I had to run back from that area to LO-skolen and was a little bit late for my own presentation.



So, it was my moment of truth and I was sweaty and nervous. About 30 people attended and it was actually more than I had expected. I started by explaining the origins of my thesis project, how I came up with the idea and the reactions of my professors at my university. I briefly showed the structure of the website, talked about the content

and gave a brief summary on each chapter. I also showed, with some difficulties, the examples and exercises evaluation window. Finally, I spoke about what is yet to be developed, my expectations for the work and also a little bit about my personal experience. I dedicated my talk to several people, among them my father Alejandro Cisneros Muñoz, my uncle Sergio Cisneros Muñoz, my APL professor Javier Fernández Pacheco and of course the people from Dyalog Limited

and the BAA. In the end, I overcame my initial nervousness and, despite my terrible accent, I think everything went very well. After my presentation, I received several nice comments, congratulations and very useful suggestions. From that I think it was a success.



As with everything else during the conference, the flavours in the banquet where new to me – in fact, every meal there was. I found the Danish food excellent, although I missed hot peppers! Apart from all the delicious things in the banquet, from the the first course to the desserts, I met and talked with very nice

people, among them Stephen Taylor, from whom I learned interesting concepts in teaching APL. One thing he passed on to me from Dr Iverson's APL Instructors Course I remember most clearly is that the best way to teach is to support people in their study rather than to supply answers – empowering them as students. That was a very interesting point of view, since I was educated in the old-fashioned way and always believed that there are some subjects that you must learn even if you don't like them, and you need a teacher to teach it. I also met Carlo Spinicci, from Italy. He approached me after the banquet to congratulate me and shared his experience on how he found a book about APL in a library in Milan and how he started his company, APL Italiana. Carlo was very nice to me, he invited me for a walk in Elsinore after the conference ended, along with the rest of APL Italiana crew, and we had a very nice time. Thank you very much for everything Carlo!

During those days I felt that I had found my place in this world. However everything has to come to an end, and so too the conference. The closing program was fun and interesting, starting with Brooke Allen telling us how to be lucky, with Stephen Taylor talking about the new *Vector* and finally with Gitte Christensen and Paul Grosvenor, who closed the conference with brief but moving speeches. Actually I felt very grateful and deeply touched when Gitte and Paul again welcomed me to this community.



So now I am writing this report while I fondly remember every minute I spent with the people I met there. Paul told me that there was no need to write a report, but still I wanted to do it as a testimony of my gratitude to Dyalog Limited and the BAA. I also wanted to reiterate

my commitment to finish the tutorial, publish it and have it translated. I would be honoured to work with the BAA on this if they are still interested. In a way I think my on-line APL tutorial fits Stephen's conception of teaching APL because it will be available for anyone interested but no one will be forced to visit it nor to follow the course strictly. I hope that when the tutorial is exposed to search engines, at least one person will find it and maybe learn something on APL from it. If such a thing ever happens then the tutorial would have achieved its main goal. I hope a day will come when I will be able to tell you about such an achievement in some other conference... perhaps next year.

Like I said in my talk, only good things happen to me when I work with APL. This time I visited beautiful places, met interesting people and made new friends. I would like to mention and give special thanks to Gilgamesh Athoraya who took all the pictures above and also speaks very good Spanish. ¡Gracias hermano! I discovered that there are others like me in this world, disregarding borders or nationalities. That is why I would love to work full time with APL not just developing applications, but also in educational projects, and – why not? – developing APL itself. And I am sure I will somehow make it happen.

In the meantime, I hope to see you all again next year!

DISCOVER

A First Look at the Causeway APL to C# Translator

by Walter Fil

Earlier this summer, I attended the first Causeway APL to C# translator seminar, where I learned a bit about the product and how to use it. As the product is in a rather early stage of its life, it would be futile if not inappropriate to document aspects of its usage. Rather, I've opted for a narrative description of its initial capabilities, as these most certainly will be extended in the near future.

Background

The notion of compiling APL has been around almost as long as APL itself. In the mainframe days, APL was thought by many, typically those not programming it, as an excessive consumer of otherwise limited and extremely expensive computer resources. The dream at the time was to compile APL simply to achieve execution speeds comparable to that of Fortran. What that really meant, no one was sure, as it was possible to write both slow Fortran and fast APL, nonetheless, the dream stuck. Over the years, starting in the early 1980s, mainframes became cheaper, and the need for severe reductions in computing costs became less apparent. The emphasis was more on the programmer productivity.

At the same time, the APL language itself was going through changes. APL2 became the evolving norm, and with its nested arrays, the topic of APL compilation had become less frequently visited. It was largely felt that the addition of nested arrays rendered the APL language truly uncompileable.

Nonetheless, some APL compilers, or more correctly translators, appeared, usually in the form of research projects. The APLc translator, by Professor Timothy Budd, successfully translated a subset of the APL language to C code (C++ was not widely available at the time). Although far from a commercial product, the APLc implementation was a clearly positive feasibility study for compiling APL.

Code generated by APLc was standalone as it required no existing APL environment – the compiler itself was written in C, and C code emitted by the product would be compiled to an executable form in the normal manner.

Another successful compiler, this one commercially available in the mid 1980s, was the one offered by STSC as part of the vendor's IBM VSAPL extensions. Here the compiler favoured scalar-oriented and iterative APL code. Through a batch process, it returned functions in the form of compiled modules. Only a subset of APL primitives was supported, calls to unimplemented functions were resolved by a call to the mainframe interpreter.

The STSC compiler depended on the APL environment – not only was the compiler itself written in APL, but the compiled functions were materialised as a sort of magic function in the workspace.

Although the IBM personal computers were available from the early 1980s, it was not until the advent of the 386-class machines which came out around 1988, possible to comfortably migrate a small-to-medium sized mainframe APL application to the PC environment. The speed issue was largely forgotten, as PC applications tended to run faster than their mainframe counterparts, and Moore's law was very much in session – processor speed tended to double every 1½ years. It had become possible to “buy your way out” of performance problems by upgrading systems or adding memory.

Enter Sun and Microsoft

By the late 1990s, with the various flavours of Windows, from Windows 3.1 to Windows NT, Microsoft development platforms had become immensely popular. Notably, Visual Basic skyrocketed to popularity, with its comfortable development environment, easy language, and emphasis on the visual aspect of Windows programming. Similarly, Sun Microsystems' Java language, developed as a general purpose Internet-oriented language, gained popularity in the non-Microsoft world. Internally, the Java language had been designed with the goal to be able to ship the compiled Java code, a form of P-code, to a Java language engine located in each machine. VBA was hybrid language implementation where true executable code would “trampoline” to parts of the VBA language engine as needed. In short, the Microsoft .Net platform evolved from the need to have a managed environment (MSIL, Microsoft's internal .Net P-code) similar in concept to Java.

In reality, both the Java and .Net run-time engines are “interpreters”, or “virtual machines”, as they're known in non-APL circles. Performance is typically less

than that of a true compiled language such as C++. However, the language engine encapsulates many of the details of the underlying hardware, thus may be able to apply optimizations to p-code on the fly. More importantly, as hardware changes, a language engine based approach can insulate the user from compatibility issues, thus guaranteeing that code written a few years earlier runs on entirely different hardware.

.Net Languages and Object Orientation

At this time, numerous .Net languages exist from Microsoft, the best known ones being VB.Net, J+, and C#. J+ is Microsoft's implementation of Java; VB.Net is the successor to VB, as VB (and VBA, Visual Basic for Applications, found in the automation-enabled Microsoft products such as Word and Excel) is being "deprecated" by Microsoft. As such, both VB.Net and C# are true object-oriented languages. VB.Net has been characterized as simply "C# without the semicolons", whether this is true, we will concentrate on C#.

As its name would imply, C# would resemble C or C++. Certainly C# is a simplification of C++, as many of the mechanisms central to program development, such as memory allocation and other memory administration tasks, are silently handled by the C# language or .Net environment. Further, like C++, C# supports full object orientation. This combination of features makes C# a very APL-friendly target language with which to build a compiler.

To recap, Object Orientation is about the creation and management of specialised and application-specific data types (*classes*) and functions to deal with these new data types (*methods*). Object oriented languages normally contain the following features. Some of them are Big Words:

- Data Abstraction

- Inheritance

- Polymorphism

- Extensibility

- Overloading

Inheritance: Suppose you have a number of related classes which describe similar data types which differ in slight but important details. One class can inherit some or most of its functions from its parent, thus avoiding code duplication and work.

Polymorphism: Suppose you have a number of classes which should return some common entity, such as the size of the object or some kind of count. Yet the actual code to do this differs wildly from one object to another. By keeping the name of

the function the same, one achieves polymorphism, i.e. a function that does the same sort of thing which exists in a number of places.

Data Abstraction: Suppose you want to implement complex numbers in your program. In reality, one complex number can be built using two floating point numbers, one for the real part, the other for the imaginary. Similarly, quaternions and octernions can be represented with vectors of floating point numbers. But from the perspective of the programmer actually using these data types, they look like one item.

Extensibility: With these features, it is possible to add extensions to at least the datatypes of the language.

Overloading: As the compiler keeps track of functions and their arguments, it is possible not only to use the name in different contexts, but to inform the compiler to allow the use of more familiar symbols as a substitute for functions. For example, the "+" symbol may be overloaded with the functionality of the "plus()" function.

The APL to C# Translator

Why would anyone want to compile APL?

Speed, speed, and speed

One-way migration to C#

C# as a delivery platform

Speed: The performance issue is no longer what it used to be in the days of timesharing where a few extra functions added up to a huge bill. Nor is APL thought of as unreasonably expensive to run. Many applications are delivered in interpreted or semi-interpreted languages such as Visual Basic or Java, and complaints that something is much too slow are comparatively rare. Thus speed for speed's sake should not be the sole reason to consider this approach. Be that as it may, compilation to C#, which implicitly included the healing effects of a cleanup, may result in some speedup and it would be safe to say that overall, APL functions compiled to C# would be overall faster. Code optimisation is a very fertile area in which to expect considerable improvements over time.

One-way migration: The output of the APL to C# translator is designed to be as readable as possible, with user maintainability of the final output an important consideration. APL comments are propagated to the compiled code, the classes used in code is kept to a minimum, no code movement takes place, control structures are nearly identical in APL and C#, and the end result is code which

looks at worst very familiar. Array and vector operations are handled by an array engine, and its use in application code is not only easy to understand, but documented. Thus it should be possible to turn compiled code over to a C# professional and let her take over.

C# delivery: A similar approach would be to continue development of a set of programs in APL, while delivering the end result in C#. As different parts of a product can be written in different .Net languages and linked together, this type of mixed language paradigm may be very efficient in certain circumstances or organisations. For example, GUI work may be handled by a different individual while the central computations of an application are done by the original APL developer. Clearly, the APL coding style may require discipline, but the code itself remains in APL. It is written in APL, tested and debugged in APL, then converted to C#. This module, combined with many others written in APL, C#, or VB.Net, contributes to the final application.

C# delivery is a convenient and compact alternative to application delivery using the run-time version of APL. Most importantly, the end result is comparatively small, which may make online or internet delivery of software that much quicker or more bearable. Interestingly, executable C# code in its initial form is relatively easily reverse engineered where an operating likeness of the original program can be easily reconstructed. Third-party products exist which obfuscate the executable code, rendering attempts at reverse engineering ineffective.

Implementation

Although APL may be the preferred tool for thought, C# can be viewed as a tool for deployment. APL and C# have some irreconcilable differences which in the end require some care on the part of the APL programmer. The APL to C# translator does what it can to smooth out the differences between the two, but in the end, the APL developer is ultimately responsible for the quality of the compiled code.

Conceptually, the design of the translator is simply to take a line of APL code and "convert" it to C#. Line by line, the translator plods through the functions, and translates them verbatim. Depending on the arguments, for an ordinary addition, the translator inserts either " $A + B$ " or "`AE.Plus(A, B)`" depending on whether arrays are involved. Code consisting of operations on scalars frequently compiles to standard C#. The actual parsing of the APL syntax is handled using a subset of the J parse table, described in full detail in Vector 9.4 p.92.

In order to support the multitude of APL primitive functions and operators, the translator draws upon an *array engine* to perform the operations. In essence, the array engine is a subroutine library with operations corresponding to the APL primitives. However, complication arises as variations of APL type, rank, and shape, each map to a different C# class.

At this time, the C# translator supports 15 different classes, which are every combination of Boolean, character, string, integer, and floating point, by rank 1, 2, and 3. Rank 0, of course, is scalar, and is supported by base C# types. It is important to note that every variation is a distinct class – integer vectors are distinct from integer arrays which are distinct from floating point arrays. Arrays of rank greater than 3 are not supported, on account of the exponential growth of function overloading possibilities.

This leads to the following distinct C# classes:

Scalar	Rank 1	Rank 2	Rank 3
Bool	bool []	bool [,]	bool [,,]
Char	char []	char [,]	char [,,]
String	string []	string [,]	string [,,]
Int	int []	int [,]	int [,,]
Double	double []	double [,]	double [,,]

It is precisely here where the beauty of the approach is apparent, where object orientation comes and does all the difficult work. Given the APL code fragment of

A + B

and given that both A and B are variables, the following possibilities exist:

plus	bool 0	bool 1	bool 2	bool 3	int 0	int 1	int 2	int 3	float 0	float 1	float 2	float 3
bool 0	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
bool 1	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
bool 2	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
bool 3	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
int 0	AE	AE	AE	AE	C#	AE	AE	AE	C#	AE	AE	AE
int 1	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
int 2	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
int 3	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE

flt 0	AE	AE	AE	AE	C#	AE	AE	AE	C#	AE	AE	AE
float 1	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
float 2	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE
float 3	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE	AE

(Score: Native C#, 4 – Array Engine, 140)

Needless to say, character data is outside the domain of addition and does not enter into the picture. If it did, we would have 16 squared possibilities, minus some of the scalar + scalar cases.

From the translator's perspective, it has to choose between "A+B" and "AE.Plus(A,B)"

This is one of the outstanding features of object orientation – the compiler keeps track of all the intermediate types and shapes (all implemented as different classes of objects) and knows exactly which one of the 140 versions of AE.Plus to deploy. C# has its own automatic type conversion rules which apply only to the scalar data types, otherwise, a version of an Array Engine function for every combination of the scalar and array types must be present. Needless to say, the developers of the Array Engine used a template-based approach to construct the library.

Control structures, introduced to the language in the last 15 years and adopted by at least two of the APL vendors, simplify code translation considerably. Control structures come out largely unchanged in the translation process, and their use contributes to the overall readability of the converted C# code. Standard APL branching is allowed, however some of the more complicated cases are not yet handled. Best to stick with simple branching.

In the compiling phase, the translator, where possible, translates the line of APL code into a line of C# verbatim. With scalar code, pure C# output is possible. With any APL arrays, regardless of rank, the array engine is invoked. Optimum readability is achieved with relatively short lines of APL code ("long and skinny" functions) and use of control structures. Long lines of APL code begin to resemble Lisp when translated to C#, owing to the large number of parenthesis required when using the array engine with a long or complex expression.

Also, with C#, the Boolean data type is not represented with the values of 1 and 0, rather "true" and "false", and do not automatically map into 1 and 0 as one would expect from other languages, notably C++ and Visual Basic.

Nested Arrays are partially handled in the form of intermediate results. One recent Dyalog extension, to be able to split positional nested array arguments directly in the function header to their constituent variables, has been implemented in the translator. This feature implements one important language construct as a consequence of nested arrays.

Lastly, the dreaded Execute function is not supported as it is not appropriate in a compiled environment. `⊖VI` and `⊖FI` work as expected, though Dyalog's `⊖VFI` function would need to be recoded to use `⊖VI` and `⊖FI` separately. Underscored variables (with the exception of delta-underbar) are not handled, and would need to be changed. User-defined traditional operators (but not in the form of `dfns`) are not supported.

Programming

One cannot reasonably expect to simply dump a bunch of functions into a sort of machine and have working C# code at the end. The APL code itself invariably will require some preparation and cleanup. C# is a strongly typed language, meaning that all variables in any program need to be declared to be of a given type. APL, of course, is anything but strongly typed, as it allows you to reuse variables, assigning them different types and/or shapes, not to mention functions and operators, and this cannot happen with C# programs. They will not compile. Better programming practice would suggest a certain discipline in the usage of variables - variables never deviate from the types with which they were originally declared.

Another important design goal of the APL to C# translator is to structure language changes in such a way that not only the code continues to work both the APL and C# language environments. All translator directives and code declarations are supplied in the form of APL comments. As mentioned, the Dyalog APL implementation of `⊖VFI`, a fusion of `⊖VI` and `⊖FI` which returns a nested result, is inappropriate. Alternately, `⊖VI` and `⊖FI` -like functions are supplied, which function correctly in APL and compile correctly to C#. Other changes are more subtle. The monadic *plus* function, defined in APL simply to return its argument, has been changed to promote its type. When executed in ordinary APL, the monadic plus has no effect on numeric arguments, and is thus harmless.

Variable declarations give the APL to C# translator a hint of what to do. Often, only a function's arguments and result need to be declared. By default, the translator treats arguments as integer scalar. With some data flow analysis, it is possible for the translator to deduce the type, shape, and rank of every variable at every point in a line of code. Thus for well-behaved programs, it may be sufficient

to simply declare the arguments. Additional declaration may be supplied simply to force the result of a computation to be of an expected type.

Places to look out for, where type conversion may be happening, would be uses of the APL *floor* and *ceiling* functions. As implemented in ordinary APL, these functions do the double duty of rounding a number to the next lower or higher integer *and* where possible, returning an integer result from a floating point argument. As implemented by the translator, floor and ceiling do work as in traditional APL. Conversely, there may be situations where a floating point argument is required, but only an integer argument is supplied. The implementation of the *monadic plus* is to promote the argument from integer to floating point, or from Boolean to integer. Again, C#'s implementation of the Boolean type is such that Boolean values are not interchangeable with 0 and 1. Scalar arguments are automatically promoted, at least from integer to floating point, by C#.

Another area where non-obvious but unexpected differences in APL and C# exist are in the implementation of *characters* vs. *strings*. Arrays of characters are no different than in APL. Strings, however, are immutable. That is, once created, they cannot be changed. C# favours strings for casual use of character data. As far as the translator is concerned, the use of strings vs. char is transparent to the user apart from one annoying limitation – you cannot use indexed assignment (or masked assignment) with a string.

Other translator directives, all in the form of comments, indicate to the compiler whether to produce “Public” (visible outside of the class) or “Private” (hidden inside the class) functions, allow the a comment (often the first comment of a function) to appear in documentation, to indicate special treatment of multiple arguments

Using the Microsoft *Visual Studio* program development facility allows, among other things, summary documentation to be viewed at the time code is being entered or edited. Potentially, this drastically improves the quality of the documentation available to the programmer at development time. Suppose several APL functions appeared as overloads, meaning different versions of the function which did the same thing but for different data types, Visual Studio would display the different possibilities.

At this time, I have used the APL to C# translator only on “little” things, meaning groups of 10 or fewer functions, and am truly impressed with the usability of the product. I've found little problems which were promptly fixed. The test and code in APL, and deliver in C#, for me, is ideal. In any event, using this product has had the effect of breaking some very bad habits, such as variable name reuse,

using variables which are too short, allowing functions to become too long, and overloading utilities. I have a number of “big” things on my plate and they all require a substantial amount of cleanup.

The cleanup effort can be daunting. My only suggestion is to start early, cleaning up as you can. Get rid of those executes, split up those large functions, get rid of branching and replace it with control structures.

Cleanup is quite a lot less work than Rewrite, which is quite a lot less work than Re-engineer. I had recycled a substantial amount of code before, but never had I thought that the code or ideas would make their way out of APL and into a mainstream computer language.

Conclusion

The computer software industry is very volatile – things come and go, today certain software may be either the hottest in software fashion and competitive advantage, only to be forgotten tomorrow. At least couple of things are here to stay, at least a while, object orientation is one of them, APL is another.

Object orientation has been around in one form or another since 1970, with it catching on in the mainstream market in the early 1990s. APL has been available since the mid 1960s, on hardware ranging from prehistoric mainframes to pocket PCs. It is also interesting to note that in the *mainstream* computer language world, only two truly new ideas in computer languages have appeared since the introduction of Fortran in 1956 – namely the notion of control structures, and object orientation. These two features also were key in making the APL translator and readable output code a possibility.

It is not obvious today whether .Net will have the staying power of some of the earlier technologies. One would reasonably expect that it will. However the APL to C# translation is a very portable concept which even today can be adapted to a variety of similar technologies. For example, Java would be a language which could be easily accommodated, doing this would substantially extend the reach of the product and more importantly, extend the reach outside of the Microsoft world. C++ is another potential target language with even greater far-reaching potential. .Net may fade into the software sunset, but whatever takes its place could have support for APL. Time marches on, so does computer science, and most certainly tomorrow's platforms can be accommodated.

Modest speedups should be expected, as with a combination of efficient algorithms, high-powered optimisations built into the .Net engine, and APL language activities such as type and rank checking now done at compile time, the

overall performance of an application should be better. However, to expect “miraculous” speedups of a factor of 100 or more is not realistic. Isolated examples of stunning increases in speed for selected code fragments do exist, but largely, code performs the same or slightly better.

In closing, the APL to C# translator supplies an excellent migration path to a new world. The productivity of using APL, the tool for thought, and the ability to test an idea or program in an interpreted environment all remain intact. The resulting code can be shipped over to the .Net world, where it can now interact with everything else, in a mainstream environment.

Using Delegates in an Interpreted Language

by Richard Smith (rsc36@cam.ac.uk)

Introduction

It is essential that an interpreter can handle events. For example, one should be able to react to a button being pressed – in Rowan:

```
button←#new Button[];  
button:Text←"Hello, world";  
button:Click←#delegate $EventHandler {sender message: Bsender:Text };
```

... the event handler must be called when the button is clicked – we expect to see `Hello World` in the session.

In .Net, events are represented by *delegates*, which are effectively typed function pointers. For example, to react to a button being clicked in C#:

```
void Start(){  
    button.Click = new EventHandler(myButtonClick);  
}  
  
void myButtonClick(object sender, EventArgs e){  
    // ... handle the event, do something cool  
}
```

The type *System.EventHandler* is a delegate, and to attach a handler to an event you must create an instance of the correct delegate type. Unfortunately for an interpreter, the *new* in the example above requires the address of a method which has the appropriate signature – which must exist at compile time! This article shows how to create a delegate at runtime, which is necessary for an interpreter to use them.

How it Works

It isn't possible to have a general event handler in the .Net world, whereas it is possible in ordinary Windows to have a general message-handling function which does broadly the same job. The method a delegate refers to must have the correct signature, and since there is no way to know beforehand what those signatures may be we need a way to create new methods at runtime.

Fortunately, the Framework does provide the *System.Reflection.Emit* namespace to emit IL (the high-level assembler C# is compiled to) directly in a temporary assembly in memory. So what we have to do is make a new type which defines a method of the correct signature and creates a delegate from it.

The Code

Here I walk through the C# code of the *CreateDelegate* method, which builds a new type that exposes a delegate of the requested type. Within the comments of this method are the lines of C# which the emitted IL represent.

```
internal static Delegate CreateDelegate(Engine engine, Type type, Symbol verb){
    if(!type.IsSubclassOf(typeof(Delegate))) throw new ArgumentException("Type
    "+type.FullName+" is not a delegate");
    SetUp();
```

Make sure what we have been asked for is a delegate, and set up the temporary assembly if we haven't already done so.

```
    if((verb.type & ObjectType.Token) != 0){
        verb.type = ObjectType.Token | ObjectType.StringRep;
    } else {
        verb.intValue = TokenType.Verb;
        verb.type = ObjectType.TokenTypeGiven | ObjectType.StringRep;
    }

    DelegateIdentifier di = new DelegateIdentifier(type, verb);
    object o = saved[di];
    if(o is Delegate) return (Delegate)o;
```

Check if we've already made a delegate from this verb, and if so return it. *saved* is a *Hashtable* which caches previous delegate requests.

```
    String name = "Type"+(index++);
    TypeBuilder ty = module.DefineType(name);
    FieldBuilder fb = ty.DefineField("e", typeof(IRowanEngine),
        FieldAttributes.Static | FieldAttributes.Private);
    FieldBuilder delegate_f = ty.DefineField("d", type,
        FieldAttributes.Static | FieldAttributes.Private);
    FieldBuilder verbsym_f = ty.DefineField("v", typeof(Symbol),
        FieldAttributes.Static | FieldAttributes.Private);
```

Set up the new type (with a unique name) and some fields it needs. *e* is the Rowan engine it will call back to when the event is fired, *d* is the stored delegate and *v* is the verb which is to be called. I use one-character names to save a few bytes of memory, as these names should never see the light of day.

```
// Get the Invoke method of the delegate and crib its parameter list
MethodInfo mi = type.GetMethod("Invoke");
ParameterInfo[] pia = mi.GetParameters();
Type[] paramtypes = new Type[pia.Length];
for(int i = 0; i < pia.Length; i++) paramtypes[i] = pia[i].ParameterType;
```

Take the parameter list of the appropriate Delegate type and copy it. The signature of a Delegate is the same as that of its Invoke method.

```
// private static {rettype} Handler({params}){
MethodBuilder Handler_m = ty.DefineMethod("H",
    MethodAttributes.Static | MethodAttributes.Private,
    mi.ReturnType, paramtypes);
ILGenerator ILGen = Handler_m.GetILGenerator();
```

Create a new method called *H* and prepare to generate the IL for it. *H* will be the method we attach to the delegate, so we set the return type and parameter types to what we just copied from *Invoke*.

```
// Make symbols out of the objects, and bundle up as 'right arg'
// ...
```

Emit the *H()* method with a series of calls to *ilGen.Emit()*. This method packages up all its parameters into an array and calls the Rowan function in *v* with it as the right argument. See the complete code at the end of the article for details of which IL codes are emitted; I haven't included it here because the details of what the handler does are Rowan-specific.

```
// public static void InitS(IEngine arg0, Symbol arg1)
MethodBuilder m = ty.DefineMethod("InitS", MethodAttributes.Static
    | MethodAttributes.Public, typeof(void),
    new Type[] {typeof(IEngine), typeof(Symbol)});
ILGen = m.GetILGenerator();
// Engine = arg0;
ILGen.Emit(OpCodes.Ldarg_0);
ILGen.Emit(OpCodes.Stsfld, fb);
// verbSym = arg1;
ILGen.Emit(OpCodes.Ldarg_1);
ILGen.Emit(OpCodes.Stsfld, verbSym_f);
// delegate = new {delegatetype} (Handler)
ILGen.Emit(OpCodes.Ldnull);
ILGen.Emit(OpCodes.Ldftn, Handler_m);
ILGen.Emit(OpCodes.Newobj, type.GetConstructor
    (new Type[] {typeof(object), typeof(IntPtr)}));
ILGen.Emit(OpCodes.Stsfld, delegate_f);
// return;
ILGen.Emit(OpCodes.Ret);
// }
```

Emit an initialisation method *InitS()* which sets the *e*, *d* and *v* fields. This, *d* and *H()* (above) are all declared as static, so we don't need to create an instance of our new type to use the delegate it exposes.

```
// public static Delegate GetDelegate(){
m = ty.DefineMethod("GetDelegate",
    MethodAttributes.Static | MethodAttributes.Public,
    typeof(Delegate), new Type[]{});
ILgen = m.GetILGenerator();

// return delegate;
ILgen.Emit(OpCodes.Ldnull);
ILgen.Emit(OpCodes.Ldfld, delegate_f);
ILgen.Emit(OpCodes.Ret);
// }
```

Emit a method *GetDelegate()*, which just returns the delegate in *d*.

```
Type finalType = ty.CreateType();
```

Create the type we've been building. It is now possible to call static methods defined within it, or create an instance of it, but it is no longer possible to add new methods to it.

```
finalType.InvokeMember("InitS", BindingFlags.Public | BindingFlags.Static
    | BindingFlags.InvokeMethod, null, null, new object[] {engine, verb});
Delegate res = (Delegate)finalType.InvokeMember("GetDelegate",
    BindingFlags.Public | BindingFlags.Static |
    BindingFlags.InvokeMethod, null, null, new object[] {});
saved[di] = res;
return res;
}
```

Call the *InitS()* method of the new type to initialise the fields, and then call *GetDelegate()* to retrieve the delegate and return it. We also save this delegate in our cache.

And that's it, basically! A call to *CreateDelegate()* can create a delegate of any type, even custom delegate types from third party controls. For another interpreted language, you'd need to change the type of *v* to your own language's function type, and modify the emitted method *H()* to call back into your engine instead of the Rowan one.

Appendix: Complete Code of the Internals Class

```

internal class Internals {
    private static AssemblyBuilder assembly = null;
    private static ModuleBuilder module = null;
    private static bool set_up = false;
    private static int index = 0;

    private static AssemblyBuilderAccess access = AssemblyBuilderAccess.Run |
        AssemblyBuilderAccess.Save;

    private static Hashtable saved;

    private static void SetUp(){
        if(set_up) return;
        AssemblyName name = new AssemblyName();
        name.Name = "RowanDynamicAssembly";
        assembly = AppDomain.CurrentDomain.DefineDynamicAssembly(name, access);
        module = assembly.DefineDynamicModule("RowanDynamicModule", "dynmodule.dll");
        set_up = true;
        saved = new Hashtable();
    }

    internal static Delegate CreateDelegate(IEngine engine, Type type, Symbol verb){
        if(!type.IsSubclassOf(typeof(Delegate))) throw new ArgumentException("Type
            "+type.FullName+" is not a delegate");
        SetUp();

        if((verb.type & ObjectType.Token) != 0){
            verb.type = ObjectType.Token | ObjectType.StringRep;
        } else {
            verb.intValue = TokenType.Verb;
            verb.type = ObjectType.TokenTypeGiven | ObjectType.StringRep;
        }

        DelegateIdentifier di = new DelegateIdentifier(type, verb);
        object o = saved[di];
        if(o is Delegate) return (Delegate)o;

        String name = "Type" + (index++);
        TypeBuilder ty = module.DefineType(name);
        FieldBuilder fb = ty.DefineField("e", typeof(IRowanEngine),
            FieldAttributes.Static | FieldAttributes.Private);
        FieldBuilder delegate_f = ty.DefineField("d", type,
            FieldAttributes.Static | FieldAttributes.Private);
        FieldBuilder verbSym_f = ty.DefineField("v", typeof(Symbol),
            FieldAttributes.Static | FieldAttributes.Private);

        // Get the Invoke method of the delegate and crib its parameter list
        MethodInfo mi = type.GetMethod("Invoke");
        ParameterInfo[] pia = mi.GetParameters();
        Type[] paramtypes = new Type[pia.Length];
        for(int i = 0; i < pia.Length; i++) paramtypes[i] = pia[i].ParameterType;
        // private {rettype} Handler({params}){
        MethodBuilder Handler_m = ty.DefineMethod("H", MethodAttributes.Static
            | MethodAttributes.Private, mi.ReturnType, paramtypes);
        ILGenerator ILgen = Handler_m.GetILGenerator();
        // Make symbols out of the objects, and bundle up as 'right arg'
        // Make array of tokens like "f ( p1 p2 p3 )"
        // ra = new RArray();
        LocalBuilder ra = ILgen.DeclareLocal(typeof(RArray));
        ILgen.Emit(OpCodes.Ldloca_S, ra);
        ILgen.Emit(OpCodes.Initobj, typeof(RArray));
        // ra.contents = new Symbol[no of params]
        ILgen.Emit(OpCodes.Ldloca_S, ra);
        ILgen.Emit(OpCodes.Ldc_I4_S, pia.Length);

```

```

ILgen.Emit(OpCodes.Newarr, typeof(Symbol));
ILgen.Emit(OpCodes.Stfld, typeof(RArray).GetField("contents"));
// {for each parameter}
for(int i = 0; i < pia.Length; i++){
    // ra.contents[i] = Symbol.CreateFromObject({params[i]})
    ILgen.Emit(OpCodes.Ldloca_S, ra);
    ILgen.Emit(OpCodes.Ldfld, typeof(RArray).GetField("contents"));
    ILgen.Emit(OpCodes.Ldc_I4_S, i);
    ILgen.Emit(OpCodes.Ldelema, typeof(Symbol));
    ILgen.Emit(OpCodes.Ldarg_S, i);
    ILgen.Emit(OpCodes.Call, typeof(Symbol).GetMethod("CreateFromObject"), null);
    ILgen.Emit(OpCodes.Stobj, typeof(Symbol));
}
// ArrayList al = new ArrayList();
LocalBuilder al = ILgen.DeclareLocal(typeof(ArrayList));
ILgen.Emit(OpCodes.Newobj, typeof(ArrayList).GetConstructor(new Type[]{}));
ILgen.Emit(OpCodes.Stloc_S, al);
// al.Add(verbSym);
ILgen.Emit(OpCodes.Ldloc_S, al);
ILgen.Emit(OpCodes.Ldsfld, verbSym_f);
ILgen.Emit(OpCodes.Box, typeof(Symbol));
ILgen.Emit(OpCodes.Call(OpCodes.Callvirt, typeof(ArrayList).GetMethod("Add"), null);
ILgen.Emit(OpCodes.Pop);
// al.Add(ra.getSymbol());
ILgen.Emit(OpCodes.Ldloc_S, al);
ILgen.Emit(OpCodes.Ldloca_S, ra);
ILgen.Emit(OpCodes.Call, typeof(RArray).GetMethod("getSymbol"), null);
ILgen.Emit(OpCodes.Box, typeof(Symbol));
ILgen.Emit(OpCodes.Call(OpCodes.Callvirt, typeof(ArrayList).GetMethod("Add"), null);
ILgen.Emit(OpCodes.Pop);
// Argument temp = new Argument(); engine.ArgumentStack.Push(temp)
ILgen.Emit(OpCodes.Ldsfld, fb);
ILgen.Emit(OpCodes.Callvirt,
    typeof(IRowanEngine).GetMethod("get_ArgumentStack"), null);
LocalBuilder temp = ILgen.DeclareLocal(typeof(Argument));
ILgen.Emit(OpCodes.Ldloca_S, temp);
ILgen.Emit(OpCodes.Initobj, typeof(Argument));
ILgen.Emit(OpCodes.Ldloc_S, temp);
ILgen.Emit(OpCodes.Call(OpCodes.Callvirt, typeof(ArgumentStack).GetMethod("Push"), null);
// engine.PushFunction("DelegateHandler", verbSym.ns);
ILgen.Emit(OpCodes.Ldsfld, fb);
ILgen.Emit(OpCodes.Ldstr, "DelegateHandler");
ILgen.Emit(OpCodes.Ldsfld, verbSym_f);
ILgen.Emit(OpCodes.Ldfld, typeof(Symbol).GetField("ns"));
ILgen.Emit(OpCodes.Callvirt, typeof(IEngine).GetMethod("PushFunction"), null);
// try {
Label try1 = ILgen.BeginExceptionBlock();
// engine.ExecuteBlock(al);
ILgen.Emit(OpCodes.Ldsfld, fb);
ILgen.Emit(OpCodes.Ldloc_S, al);
ILgen.Emit(OpCodes.Call(OpCodes.Callvirt, typeof(IEngine).GetMethod("ExecuteBlock"), null);
ILgen.Emit(OpCodes.Leave, try1);
// } catch (RException rx) {
ILgen.BeginCatchBlock(typeof(RException));
// throw rx.originalError;
ILgen.Emit(OpCodes.Ldfld, typeof(RException).GetField("originalError"));
ILgen.Emit(OpCodes.Throw);
// }
ILgen.EndExceptionBlock();
// engine.PopFunction();
ILgen.Emit(OpCodes.Ldsfld, fb);
ILgen.Emit(OpCodes.Call(OpCodes.Callvirt, typeof(IEngine).GetMethod("PopFunction"), null);
// engine.ArgumentStack.Pop() ... left on stack
ILgen.Emit(OpCodes.Ldsfld, fb);
ILgen.Emit(OpCodes.Call(OpCodes.Callvirt,
    typeof(IRowanEngine).GetMethod("get_ArgumentStack"), null);
ILgen.Emit(OpCodes.Callvirt, typeof(ArgumentStack).GetMethod("Pop"), null);

```



```

if(mi.ReturnType != typeof(void)){
    // (return engine.ArgumentStack.Pop() ... ) .result;
    ILGen.Emit(OpCodes.Stloc_S, temp);
    ILGen.Emit(OpCodes.Ldloc_A_S, temp);
    ILGen.Emit(OpCodes.Ldfld, typeof(Argument).GetField("result"));
    LocalBuilder res_v = ILGen.DeclareLocal(typeof(Symbol));
    //ILGen.Emit(OpCodes.Stloc_S, res_v);
    //ILGen.Emit(OpCodes.Ldloc_S, res_v);
} else ILGen.Emit(OpCodes.Pop);

ILGen.Emit(OpCodes.Ret);
// }

// public static void InitS(IEngine arg0, Symbol arg1)
MethodBuilder m = ty.DefineMethod("InitS", MethodAttributes.Static |
    MethodAttributes.Public, typeof(void),
    new Type[] {typeof(IEngine), typeof(Symbol)});
ILGen = m.GetILGenerator();
// Engine = arg0;
ILGen.Emit(OpCodes.Ldarg_0);
ILGen.Emit(OpCodes.Stsfld, fb);
// verbSym = arg1;
ILGen.Emit(OpCodes.Ldarg_1);
ILGen.Emit(OpCodes.Stsfld, verbSym_f);
// delegate = new {delegatetype} (Handler)
ILGen.Emit(OpCodes.Ldnull);
ILGen.Emit(OpCodes.Ldftn, Handler_m);
ILGen.Emit(OpCodes.Newobj,
    type.GetConstructor(new Type[] {typeof(object), typeof(IntPtr)}));
ILGen.Emit(OpCodes.Stsfld, delegate_f);
// return;
ILGen.Emit(OpCodes.Ret);
// }

// public static Delegate GetDelegate(){
m = ty.DefineMethod("GetDelegate", MethodAttributes.Static |
    MethodAttributes.Public, typeof(Delegate), new Type[] {});
ILGen = m.GetILGenerator();

// return delegate;
ILGen.Emit(OpCodes.Ldnull);
ILGen.Emit(OpCodes.Ldfld, delegate_f);
ILGen.Emit(OpCodes.Ret);
// }

Type finalType = ty.CreateType();
finalType.InvokeMember("InitS", BindingFlags.Public | BindingFlags.Static
    | BindingFlags.InvokeMethod, null, null, new object[] {engine, verb});
Delegate res = (Delegate)finalType.InvokeMember("GetDelegate",
    BindingFlags.Public | BindingFlags.Static |
    BindingFlags.InvokeMethod, null, null, new object[] {});
saved[di] = res;
return res;
}

internal static void SaveAssembly(String filename){
    assembly.Save(filename);
}

private struct DelegateIdentifier {
    Type type; VerbInfo Verb;

    internal DelegateIdentifier(Type type, Symbol verb){
        Type = type; Verb = (VerbInfo)verb.Value; }
}
}

```

LEARN

An Introduction to Object Oriented Programming For APL Programmers

by Morten Kromberg, Dyalog Ltd, December 2005

This has been written as a companion to version 11.0 of Dyalog APL. Our goal is not only to explain the details of new functionality in version 11.0 of Dyalog APL, but also be mildly provocative and entertaining, to convey a flavour of the thinking which is behind the object oriented extensions to APL, and something about how the development team imagines that you might make use of the new features – in the belief that this will make them easier to understand and use.

It is recommended that you have version 11.0 of Dyalog APL available for experiments as you work through the guide, and use it to verify your understanding of the new features as they are introduced.

1. Introduction

Version 11.0 of Dyalog APL introduces *Classes* to the APL language. A class is a blueprint from which one or more *Instances* of the class can be created (instances are sometimes also referred to as *Objects*). For example, the following defines a class which implements a simple timeseries analysis module:

```
:Class TimeSeries

:Field Public Obs      A Observed values
:Field Public Degree+3 A 3rd degree polynomials by default

▽ r←PolyFit x;coeffs
:Access Public
  A Fit polynomial to observed points and compute values at x
  coeffs←Obs⌈(1pObs)×.×0,1Degree A Find polynomial coeffs
  r←coeffs+.×x×.×0,1Degree      A Compute fitted f(x)
▽
```

```

▽ make args
  :Access Public
  :Implements Constructor
  Obs+args
▽

:EndClass A Class TimeSeries

```

The above description declares that instances of `TimeSeries` will have four *Members*: two *Public Fields* called `Obs` and `Degree` (the latter having a default value of 3), a *Public Method* called `PolyFit` (header plus 4 lines of code) and a *Constructor*, which is implemented by the function `make` (header plus 3 lines of code). Note that methods (or functions) begin and end with a `▽`. The term *Public* means that the methods are for "public consumption" by all users of the class.

The system function `NEW` is used to create new instances using the class definition. The first element of the right argument to `NEW` must be a class, the second element contains *instance parameters*, which are passed to the constructor:

```
ts1←NEW TimeSeries (1 2 2.5 3 6)
```

During *instantiation*, the constructor function `make` is called, and it initialises the instance by storing its argument in the public field `Obs`. We can now use the instance `ts1` in much the same way as if it were a namespace:

```

      ts1.Obs
1 2 2.5 3 6

      ts1.PolyFit 15
0.9714285714 2.114285714 2.328571429 3.114285714 5.971428571

      1*ts1.(Obs-PolyFit 1pObs)
0.0 0.1 0.2 0.1 0.0

      ts1.[n1 2 a NameClass 2 is roughly equivalent to "PropList"
Degree Obs
      ts1.[n1 3 a NameClass 3 is "MethodList"
make PolyFit

```

For negative arguments, `ONL` returns a vector of names rather than a matrix. The `Degree` field allows us to decide the degree of the polynomial function used when fitting the curve. The following example uses a straight line:

```

      ts1.Degree+1
      ts1.PolyFit 15
0.7 1.8 2.9 4 5.1

```

Arrays of instances are handled in much the same way as arrays of namespaces:

```
⎕rl+16807 A Value in CLEAR WS - so we get the same random numbers
tss←{⎕NEW TimeSeries ((10×15)+75ρ5)}"14
```

```
tss.Obs
```

```
11 24 33 43 52
11 24 34 45 52
13 25 31 41 53
14 21 32 41 53
```

```
tss.Degree←2 3 2 1
```

```
1⊣tss.PolyFit ⏟110
```

```
11.4 23.0 33.6 43.2 51.8 59.4 66.0 71.6 76.2 79.8
11.1 23.5 34.8 44.5 52.1 57.2 59.2 57.6 52.0 41.8
14.0 22.7 32.0 41.9 52.4 63.4 75.0 87.2 99.9 113.2
12.6 22.4 32.2 42.0 51.8 61.6 71.4 81.2 91.0 100.8
```

APL developers have often used naming standards, and in recent versions of Dyalog APL namespaces, to collect related functionality into modules. Users of Dyalog APL will recognise that an instance is very similar to a *namespace*. One of the big advantages of classes is that they make it possible to “clone” a namespace and create multiple data “contexts”, without copying the code. This saves space, but more importantly it means that you are less likely to lose track of where the source code is.

Imagine that we have prototyped our way through to a “classical” solution to fitting multiple polynomials. Looking back, we are able to copy the following expressions from our session:

```
data←(4ρ<10×15)+?4ρ<5ρ5      A Generate some test data
exp←0,"1"2 3 2 1             A Our polynomials vary by degree
coeffs←data⏟(⏟15)×."exp
1⊣tcoeffs+."⏟(⏟10)×."exp
14.2 23.0 32.8 43.6 55.4 ...etc...
```

After thinking a bit more about it, we might identify a couple of potential functions. If we use dynamic functions, we can refactor our solution as follows:

```
polyfit←{ω⏟(1ρω)×.0,1α}
polycalc←{α+."⏟ω×."1+1ρα}
1⊣(2 3 2 1 polyfit"data) polycalc"⏟110
14.2 23.0 32.8 43.6 55.4 ...etc...
```

In a traditional APL system, we could now create a workspace called POLY with these functions inside, write some documentation explaining how to call them, and then store that documentation in a variable in the workspace, or in a separate

document. Anyone wanting to use the functions would have to find the relevant documentation and make sure that he did not already have any functions with these names in his application (or variables with the same name as the documentation). A namespace could be used to isolate the names from our application code.

Classes make it possible for the developer to encapsulate functionality in a way which keeps related code and data together, avoids name conflicts and provides some degree of documentation which suggests and can limit how the solution is used. This makes the module easier to learn to use, while the control over how the module can be used makes it easier to maintain. At the same time, splitting an application into objects with well defined behaviour and interfaces is a valuable tool of thought when dealing with complex design issues.

On the other hand, it is also clear that the simple functions `polyfit` and `polycalc` are more *generally* useful than the `PolyFit` method of the class `TimeSeries`, which exposes a specific use of polynomial fitting. The encapsulation of data within instances can make it harder, slower, and sometimes virtually impossible to go "across the grain" and use the properties and methods in a way which is different from that which was intended by the class designer. OO fans may argue that object orientation will help you think more carefully about how things will be used and this is to your advantage. However, APL is often used in problem areas where requirements change *very* unexpectedly. Providing a flexible solution with OO design is as much of an art, and requires the same insight into where the solution might be heading, as any other technique.

A key design goal for version 11.0 has been to make it as easy as possible to blend the array and functional paradigms which already make APL so productive, with the object oriented view of data, a Tool of Thought in its own right.

This is one of the reasons why, if you have a namespace `POLY` which contains the two dynamic fns we developed above, you can add a line which says:

```
:Include #.POLY
```

... at the beginning of `:Class TimeSeries`, and subsequently write `PolyFit` as:

```
▽ r←PolyFit x;coeffs
  :Access Public
  A Use cubic fit of observed points to compute values at x

  coeffs←Degree polyfit Obs A Find polynomial coeffs
  r←coeffs polycalc x        A Compute fitted f(x)
▽
```

This *Introduction to Object Oriented Programming for APL Programmers* will attempt to illuminate the issues and put the reader in a better position to decide when and how to combine array, functional and object thinking. In order to achieve this, we will:

First, briefly explore the thinking which lead to the emergence of OO, to get an idea about the type of problems which OO is likely to help us solve.

Introduce the fundamentals of OO programming using a number of examples written in Dyalog APL version 11.0.

Illustrate how the new OO functionality in Dyalog APL makes it easier than ever before to implement components which can be “consumed” by other development tools.

Where possible, try to remember to discuss alternative solutions, and present some guidelines on how to choose between the various techniques which are available. Given that the temperament and environment of the developer, the department and the company will weigh heavily on any choice of technique, it is clear from the outset that there will be no universal answers.

2. Origins

Although OO feels like a recent invention to many of us, the first OO language saw the light of day around the same time as the first APL interpreter. SIMULA (SIMULATION LAnguage) was designed and implemented by Ole-Johan Dahl and Kristen Nygaard at the Norwegian Computing Centre between 1962 and 1967, based on ideas which Nygaard had developed during the 1950s¹. At the same time that Ken Iverson was working on new ways to conceptualise algebra and computation involving large groups of numbers, Nygaard was searching for ways to think about a different type of systems using symbolic notation².

The focus of the NCC work was on the simulation of complex systems. Nygaard explained the rationale behind SIMULA as follows:

SIMULA represents an effort to meet this need with regard to discrete-event networks, that is, where the flow may be thought of as being composed of discrete units demanding service at discrete service elements, and entering and leaving the elements

¹ For more on the fascinating story about SIMULA, see “Compiling Simula” by Jan Rune Holmevik, http://www.ifi.uio.no/~cim/sim_history.html

² Ken Iverson was born in Canada in 1920, but Norway also figures prominently in his heritage. Therefore, we should have reason to hope that the paradigms are not incompatible ☺.

at definite moments [sic] of time. Examples of such systems are ticket counter systems, production lines, production in development programs, neuron systems, and concurrent processing of programs on computers.

The desire to describe and model so-called discrete-event networks led to object-oriented notation, in which the description of the ways in which the “service elements” interacted with each other is separated from the details of how each service element (or *object*) manages its internal state. As with APL, the language subsequently evolved into a notation which could be executed by computers.

SIMULA turned out to be a powerful notation for simulating complex systems, and other OO languages followed. Initially, OO languages were used for planning and simulation applications (much the same areas as APL has been most successfully applied to), but with the arrival of graphical user interfaces, which are a form of discrete-event network, and concurrent or networked computing systems, OO languages proved that they had more to offer. As systems and teams used to implement them have grown in size and complexity, OO has grown from a humble start as a specialist modelling technique to become the most popular paradigm for describing computer systems.

With Dyalog APL version 11.0, Arrays, Functions and Objects are now happily married. It is possible to have arrays of instances, and instances can contain arrays (of more instances, if necessary). The challenge is to pick the best architecture for a given problem!

Queue SIMULATION

As an illustration, let us take a look at one of the classical examples which the inventors of SIMULA used the new language to model: The *queue*. Customers arrive at random intervals and enter the queue. An algorithm simulates the time required to process each customer. The goal is to run a number of simulations with different parameters and see how long the queue gets and how long customers have to wait. With luck, we will discover how the queue or system of queues can be optimized, and measure the effect of improving the system without having to perform expensive experiments. In the post-modern age, these systems are probably being used to see how much *longer* the queues will get if the Post Office spends *less* money. Assuming that the planning department has not already been “made redundant”.

The following is a simple Queue class, written in Dyalog APL version 11.0³.

```
:Class Queue

:Field Public Instance History←0
Customers←0

▽ r←Length
:Access Public
r←1+pCustomers
▽

▽ Join Customer
:Access Public
Customers←Customers,∘Customer,(3÷⌊AI),1+pCustomers
:If 1=pCustomers ∘ Serve&0 ∘ :EndIf
▽

▽ Serve dummy:t:elapsed
A Start serving queue in new thread when queue length grows to 1
A Stop when queue is empty.

:Repeat
  ⌊DL 9+?11 A Processing takes between 10 and 20 seconds
  elapsed←(3÷⌊AI)-1 2÷Customers A Since customer entered queue
  History,←∘(elapsed,1÷Customers)[2 1 4]
  A History records (Cust#, Total Time, Initial Queue Length)
  Customers←1+Customers A Customer has left the building
:Until 0=pCustomers
▽

:EndClass A Class Queue
```

The class has two *public methods* called *Join* and *Length*, which are used to add customers to the end of the queue and report the current length of the queue, respectively. There is a *public field* *History* which contains a record of customers who passed through the system. Note that *members* of a class are *private* unless they are declared to be public. Private members cannot be referenced from outside the class.

While the *TimeSeries* class in the previous chapter only had public members, *Queue* has a *private field* called *Customers*, and a *private method* called *Serve*, which is launched in a background thread when there are customers in the queue (thread-savvy readers are requested to ignore the potential race condition if *Server* drops the last element of *Customers* at the same time as *Join* adds one).

³ For a typical SIMULA solution, see Example 2 on page
<http://staff.um.edu.mt/jskl1/talk.html#Simulation>

If we have a workspace containing the above class, we can experiment with it from the session:

```
aQueue+[]NEW Queue
aQueue.Join"1 2 3    A 3 customers stormed in together
aQueue.Length      A Result will depend on how quickly you type

2
[]+[]DL 60
60.022
aQueue.Length
0
```

The variable `History` contains a log of the customers who passed through the queue, the time they spent in line and the length of the queue when they entered it. Since `History` was declared as a public field, we can refer to it from "outside":

```
taQueue.History      A Complete history
1 11032 1
2 26800 2
3 41833 3

[/aQueue.{"History") A Longest wait and max queue length
41833 3
```

Encapsulation

In our `Queue` class, we have decided that the `Customer` field, which contains the list of customers in the queue, is *private*. We do not want users of our class to reference it. We have provided a public method `Length` which makes it possible to determine how long it is.

Why have we not simply made the variable public and allowed the user to inspect `aQueue.Customers` using `p` or other primitive functions, rather than doing extra work to implement a method ourselves? We would typically do this if we want to reserve the right to change this part of the implementation in the future, or if we do not wish to take responsibility for the potential bugs resulting from the use of these members ("Warranty void if seal broken").

If we had exposed `Customers` directly, users would have the right to expect that we would continue to have a one-dimensional vector called `Customers` with similar characteristics in the future. It would be virtually impossible for us (as class designers) to estimate the impact of a change to this variable, without reading all the application code to see *exactly* how it was being used. And even then, we would probably get it wrong. The user might also feel tempted to modify the variable, which might cause bugs to be reported to us, even though there were no errors in our code.

If our class evolved into a more general tool where the contents of the queue were not necessarily customers, we could not rename it – so we would end up with a system where variable names were misleading⁴. We cannot turn it into a 2-dimensional matrix, store it on file, or make any other architectural changes which might be convenient as requirements evolve and we need to store more or different types of information about the queue.

We have thought ahead a little bit and decided that it will always be reasonable for users of the Queue class to ask what the current length of the queue is, and therefore we have exposed a public method for this purpose.

Information hiding is one of the cornerstones of OO. The ability to decide which members of a class are visible to the users on the one hand and the developer of the class on the other is seen as the key to reduced workload, improved reliability and maintainability of the code on *both* sides of the divide. Dyalog APL version 11.0 takes a strict view of encapsulation. It is *not* possible to reference private members (a VALUE ERROR will be signalled if you try).

```
aQueue.Customers
VALUE ERROR
aQueue.Customers
^
```

Clearly, if the designer of a class you are using has decided to hide information which you really need before 9am tomorrow, this can be frustrating. Before you get too concerned, the good news is that there are a variety of techniques for getting past the gatekeepers in an emergency, or in a “prototyping session”. We will discuss a couple of these in the following chapters. However, to enjoy the full benefits of OO, it is important to have the discipline to use such tricks only when required, and re-factor the class in question at the first available opportunity.

The variable *History* is obviously susceptible to the same problems as *Customers*, and seasoned OO designers would probably consider it to be bad form to expose it. This is perfectly true, exposing all the result data in this form in order to make it easy to analyze is a result of “traditional APL thinking”. We will investigate a number of alternatives which we could have used to expose data in subsequent chapters.

⁴ One-letter names tell no lies, of course ☹.

3. Working with Classes

In order to help you successfully experiment with APL as we explore more OO functionality, let us take a closer look at the practical details of functionality which has been introduced in the first two chapters, and get comfortable with actually working with classes and instances.

The easiest way to create a class is probably to use the editor. Start an)ED session, prefixing the name of your new class by a circle (ctrl+O on most keyboards). We're going to use a class for generating random numbers to illustrate some important issues:

```
)ed oRandom
```

This will present you with an empty class, which only contains the :Class and :EndClass statements. Insert a few lines to create the following simple class:

```
:Class Random

:Field Private InitialSeed←42
⌈←'Default Initial Seed is: ',⌈InitialSeed

▽ make args
:Implements Constructor
:If 0≠args ⌈ InitialSeed←args ⌈ :EndIf
Reset
▽

▽ Reset
:Access Public
⌈RL←InitialSeed
'Random Generator Initialised, Seed is ', ⌈RL
▽

▽ r←Flat x
:Access Public
A Return x random numbers in range [0,1>, flat distribution
r←(~1+?xp100)÷100
▽

:EndClass A Class Random
```

This class can be used to generate sequences of random numbers with a "flat" distribution between 0 and 1 (with only 2 digits, to allow us to easily recognize them in the examples). One advantage of encapsulating it in a class is that it can manage its own seed (⌈RL) completely separately from the rest of the system. We can generate repeatable or suitably randomized sequences according to the requirements of our application.

As you exit from the editor, APL evaluates the class definition from top to bottom. Most of the script consists of function definitions, but in our class there are two APL expressions, which are executed as the script is fixed in the workspace:

```
InitialSeed←42
⌵←'Default Initial Seed is: ',⌵InitialSeed
```

As a result, you should see one line written to the session as you leave the editor. If there are errors in any of the executable lines in your script, you will see one or more error messages in the status window, and the class will not be fixed in the workspace. If you are unable to correct (or comment out) all the errors, you can save your work and return to it later by changing the type of the name to a character vector.

Let's perform some experiments with our new class:

```
rand1←⌵NEW Random 0
Random Generator Initialised, Seed is 42
```

The constructor also has an output statement, which shows us which initial seed was selected for the instance.

```
rand1.Flat 6
0 0.52 0.73 0.26 0.37 0.19
```

```
rand2←⌵NEW Random 0
Random Generator Initialised, Seed is 42
```

```
rand2.Flat 3
0 0.52 0.73
```

```
?6 6 6
1 5 3
```

```
rand2.Flat 3
0.26 0.37 0.19
```

As can be seen above, each instance produces the same sequence of numbers, if the same initial seed is used. The sequence is unaffected by the use of ? in the root, or indeed anywhere else in the application.

```
rand3←⌵NEW Random 7913
Random Generator Initialised, Seed is 7913
```

```
rand3.Flat 6
0.06 0.85 0.1 0.29 0.78 0.62
```

```
rand3.Reset
Random Generator Initialised, Seed is 7913
```

```

rand3.Flat 6
0.06 0.85 0.1 0.29 0.78 0.62

```

We can create a generator with a non-default initial seed, and we can reset the sequence. If you are accustomed to using namespaces, the above behaviour will not come as a surprise, as you will be accustomed to each namespace having a separate set of system variables. However, the encapsulation provided by an instance is even stronger, as illustrated by the following example:

```

rand4+NEW Random 0
Random Generator Initialised, Seed is 42

rand4.Flat 3
0 0.52 0.73

rand4.(?6)
4

rand4.Flat 3
0.26 0.37 0.19

```

If `rand4` had been a namespace rather than an instance, the call to `?` inside `rand4` would have modified `RL` in the namespace, and the subsequent call to `Flat` would have continued from a different point in the sequence. However, APL expressions executed on an instance from outside are executed in an “APL execution space”, which is *separate* from the space in which the class members run.

In effect, when an instance of a class is created, APL encapsulates it within a namespace. This has always been the case for instances of COM or DotNet classes, and as a result, Dyalog APL also allows the use of APL expressions in parenthesis following the dot after the name of one of these instances. Such APL expressions have access to all the public members of the instance, but are (obviously, since Excel cannot run APL expressions) executed outside the instance itself, as in the following example:

```

'XL' DMC 'OLECLIENT' 'Excel.Application'
XL.(Version,'/',OperatingSystem)
11.0/Windows (32-bit) NT 5.01

```

In this example (which would work the same way in Dyalog APL versions 9 or 10), there is an APL expression which references the public properties `Version` and `OperatingSystem` and catenates them together. For consistency, the same approach is used for instances of APL-based classes, rather than simply running the expressions as if the instance was a namespace. Thus, the behaviour of an APL class will not change if it is exported to a DotNet Assembly or a COM DLL and subsequently used from APL.

When an instance method such as `Flat` is referenced in one of these expressions, *it* runs in the instance environment. For example, the example on the previous page could have been written:

```
rand4←NEW Random 0
Random Generator Initialised, Seed is 42
rand4.Flat 6
0 0.52 0.73 0.26 0.37 0.19

Rand4.Reset
Random Generator Initialised, Seed is 42
rand4.(Φ(Flat 3) (76) (Flat 3))
0 0.52 0.73 4 0.26 0.37 0.19
```

The reverse is required because the rightmost call to `Flat` happens first ☺. The important point is that the call to `(76)` in the middle of the expression executes in and uses the `URL` in the *APL space* and does not modify the value of `URL` in the *instance space*.

Note: There are a couple of small potential surprises which are worth mentioning:

First, the APL space inherits the values of `IO`, `ML` and other system variables when the object is instantiated – *not* where it is being used. Secondly, if you mistype the name of a property or field in an assignment, this will create a variable in the APL space. For example:

```
ts1←NEW TimeSeries (1 3 2 4 1)
ts1
#. [Instance of TimeSeries]
ts1.Obs
1 3 2 4 1

ts1.obs←1 3 2 4 2 A Lowercase "o", note no error message!
```

This is the same behaviour as you would get if you made a spelling error in APL, but might come as a bit of a surprise in an OO setting. However, we believe that it is desirable to allow a user to introduce own names for analytical purposes. For example, if `iPlan` is an instance of some object which exposes properties named `Actual` and `Budget`, it may be very useful to introduce a new property:

```
iPlan.(Variance←Actual-Budget)
```

It is possible that a future version of Dyalog APL will allow the class designer or the user to place restrictions on the introduction of new names into the APL space.

4. Some Useful Debugging Tricks

The *strict encapsulation* described in the previous chapter may be a bit disconcerting to APL developers, who are accustomed to having access to data on a “want to know” rather than a “need to know” basis ☺. What if we want to know what value `RL` or `InitialSeed` currently have in the instance, because the instance seems to be misbehaving?

The first thing which is important to realize is that if you set a stop in an method, or trace into a call, the internal environment where the method is running is available to you while the method is on the stack. To experience this first hand, create a new instance of `Random`, trace into a call to `Reset` or `Flat` and examine the value of `RL` while one of these functions is suspended.

It is also important to realize that classes and instances are dynamic in APL (as you would expect)! If you edit a class and fix it, all existing instances will be updated to include the new definition. You can inject temporary methods into a class for debugging purposes. Type `)ED Random` and add a public method to the class:

```

▽ r←RL x
  :Access Public
  r←RL
  :If x≠0 ◇ RL←x ◇ :EndIf
▽

```

Using our new method `RL`, we can now query and set `RL` in the instance as follows:

```

rand1.RL 77
42
rand1.RL 0
77

```

The system function `CLASS` returns the class of an instance. This can be useful in a debugging situation where you are faced with a misbehaving instance of unknown pedigree and need to know which class to edit. You could just display the instance, the default display will often tell you the class name, but as we will learn a bit later, it is possible to change this – so it is not a reliable way to determine the class.

```

    ⍵←rand1
#. [Instance of Random]

    CLASS rand4
#. Random

```

In one of the following chapters, we will show how it is possible to define a *derived* class. A derived class *extends* an existing class by *inheriting* its definition and adding to it. For an instance of a derived class, the result of `CLASS` will have more than one element, and document the entire class *hierarchy*. The first element always starts with a reference to the class which was used to create the instance.

The Mother of All Workarounds

The ultimate workaround or back door to break encapsulation is of course the introduction of a public method with a name like `Execute`, which allows you to execute any APL expression you like in the instance space. We can use the `:Include` keyword to embed a namespace containing suitable development tools in your classes. An example namespace called `OOTools` can be found in the workspace of the same name in the `OO4APL` folder. It includes a number of functions which may be useful during development. The functions with names beginning with `i` will execute in the instance space, those beginning with `s` will run in the shared space (more about the shared space later):

```

rand1←NEW Random 0
Random Generator Initialised, Seed is 42

rand1.xxx←'Bingo'      A New variable in the APL space
rand1.ΩNL 2           A Vars in the APL space
xxx
rand1.iNL 2           A Vars/Fields/Props in Instance Space
InitialSeed
rand1.iNC 'make'      A make is an instance Function
3

```

And:

```

rand1.ΩRL←77
rand1.ΩRL             A No surprise!
77

rand1.iExec 'ΩRL'     A ... the instance value is unchanged
42
rand1.iExec 'ΩRL←99'  A But now we are able to change it!
99

```

When you release the application for testing, you should empty the `OOTools` namespace to ensure that your application code does not use any of these "forbidden" methods.

Summary of Chapters 1–4

In the first four chapters, we have discussed the extensions to Dyalog APL which are summarized in the following table.

<code>⌋ED ⓄMyClass</code>	Edits the class <code>MyClass</code>
<code>Instance←ⓄNEW MyClass Args</code>	Create a new instance of <code>MyClass</code> , passing <code>Args</code> to the constructor.
<code>:Class MyClass</code> <code>:EndClass</code>	Statements which begin and end the class script for <code>MyClass</code> .
<code>:Field MyField</code> <code>:Field Private MyField</code>	A private field (can only be used by code defined in the class)
<code>:Field Public MyField</code>	A public field (visible from the outside)
<code>:Field ... MyField ←expression</code>	A field with a default value
<code>▽MyFn</code> <code>▽</code>	Beginning and end of a function or “method”
<code>:Access Public</code>	Declares the current function to be public
<code>:Access Private</code>	Function is private (the default)
<code>:Implements Constructor</code>	Identifies a method as the constructor, which is used to initialize the contents of a new instance
<code>:Include Namespace</code>	Makes all functions and variables in <code>Namespace</code> private members of the current class, unless this is overridden by <code>:Access</code> statements in the code.
<code>ⓄCLASS Instance</code>	Returns the class hierarchy for an instance.
<code>ⓄNL</code>	Negative arguments return a vector of names. Further extensions to <code>ⓄNC</code> and <code>ⓄNL</code> will be introduced shortly.

Coming Next

Properties, Constructors, Shared members, Inheritance and Interfaces.

Lucidity Through Infix Notation

or Destructuring APL

by Phil Last

The napkin, not the first to end up in Italy, I believe, was of superb quality, as evidenced by minimal spread of the ink from John's fibre tipped pen, as was the meal it was served with.

Introduction

What I wrote on it, or tried to write, my hand-eye coordination not being what it had been earlier, recalls a paper by David Eastwood in 1993 [E1] suggesting we could use defined operators to abolish the GOTO and get around the absence of control structures in APL. Something I'd been doing since APL2 arrived 10 years earlier.

My contention now is that in many situations, using control structures, which were not available in 1993, actually obscures the intent of our programs whereas infix notation using conjunctions is a more lucid way to describe them.

Meaningful sentences don't start with "If".

"Well, it ain't no use to sit and wonder why, Babe, if you don't know by now" [D1]

Most people have seen something like this ...

```

-----
else←{      | then←{
    α:αα ω  |   αα ω:ωω ω
    ωω ω    |   ω
}           | }
-----

```

We can do ...

`res←test consequence else alternative arg` (1)

where `res` is the data result, determined by boolean singleton `test`, of either `consequence` or `alternative` both of which are functions with `arg` in their domains, and ...

`res←proposition then consequence arg` (2)

where again both `proposition` & `consequence` have `arg` in their domain, `res` being either `arg` as is or the data result of `consequence` if `proposition`'s boolean result is true.

David Eastwood hoped to combine these two expressions to produce ...

```
res←proposition then consequence else alternative arg (3)
```

To quote David ...

The simplified construction:

```
... test THEN exp1 ELSE exp2 data
```

can be written with user-defined operators `THEN` and `ELSE` but presents a problem, The order of execution of the expression is:

```
... test THEN exp1
```

which will generate the left operand for

```
... ELSE exp2
```

It is difficult to come up with a safe, general return code from `THEN` which can instruct `ELSE` to evaluate its right operand ...

I suspect that as `(proposition then consequence)` is the left operand to "else" David was viewing it as atomic. The way operands are passed to operators `BEFORE` being executed means it is not the business of "then" to instruct "else" to do anything but rather for "else" to call its operand function which is derived from "then". It can do this or not in any way the coder chooses.

By constructing both operators such that "else" passes an otherwise meaningless left argument to the derived function which becomes "then" 's left argument, "then" can call either its "proposition" or its "consequence" rather than calling "consequence" conditionally on the result of "proposition".

And we can construct an "if" with commuted syntax (`consequence if proposition`) and if it is made to rely on "then" then it will also work under "else" giving us, besides a headache ...

```
res←consequence if proposition arg (4)
```

```
res←consequence if proposition else alternative arg (5)
```

Here is what else was written on the napkin ...

<pre> then+{ α+ 'M' α= 'Q' : ({ω} ∘ αα) ω α= 'Y' : ({ω} ∘ ωω) ω ({ω} ∘ αα) ω : ({ω} ∘ ωω) ω ω } </pre>	<pre> else+{ α+ 'M' α= 1 : ({ω} ∘ αα) ω α= 0 : ({ω} ∘ ωω) ω 'Q' αα ω : 'Y' αα ω ({ω} ∘ ωω) ω } </pre>	<pre> if+{ α+ {ω} α(ωω then αα) ω } </pre>
--	---	--

I use two techniques here that might not be universally recognised ...

$\alpha + \{\omega\}$

allows ambivalent expressions to be coded. It is executed conditionally by the D: interpreter only in a monadic use. In a dyadic use, array (α) to the left of a function indicates another dyad. In a monadic use, identity function (α) to the left of a function indicates another monad.

I use this here in "if", though I missed it from the napkin, so that it can pass on its current valence to "then", being called monadically in the standalone call (4) and dyadically under "else" in (5).

The other technique uses another type of ambivalence ...

$(\{\omega\} \circ \alpha\alpha) \omega$

is ambivalent in the sense that ($\alpha\alpha$) can be either an array or a function. If it is a function the $\{\omega\} \circ$ forces a monadic call even in the presence of a left argument. If it is an array (variable) it is the curried right argument to the function $\{\omega\}$ & is therefore returned unchanged.

This is used above wherever an explicitly monadic call is made to an operand and permits the following variations on the above sentences (1-5), using CAPITALS to indicate arrays ...

RES+TEST consequence else alternative ARG	(1)
RES+TEST(consequence else OTHER)ARG	(1.1)
RES+TEST(VALUE else alternative)ARG	(1.2)
RES+TEST(VALUE else OTHER)ARG	(1.3) *
RES+proposition then consequence ARG	(2)
RES+(proposition then VALUE)ARG	(2.1)
RES+TEST then consequence ARG	(2.2)
RES+(TEST then VALUE)ARG	(2.3)

RES+proposition then consequence else alternative ARG	(3)
RES+ (proposition then consequence else OTHER) ARG	(3.1)
RES+proposition then VALUE else alternative ARG	(3.2)
RES+ (proposition then VALUE else OTHER) ARG	(3.3)
RES+TEST then consequence else alternative ARG	(3.4)
RES+ (TEST then consequence else OTHER) ARG	(3.5)
RES+TEST then VALUE else alternative ARG	(3.6)
RES+ (TEST then VALUE else OTHER) ARG	(3.7) *
RES+consequence if proposition ARG	(4)
RES+ (consequence if TEST) ARG	(4.1)
RES+VALUE if proposition ARG	(4.2)
RES+ (VALUE if TEST) ARG	(4.3)
RES+consequence if proposition else alternative ARG	(5)
RES+ (consequence if proposition else OTHER) ARG	(5.1)
RES+consequence if TEST else alternative ARG	(5.2)
RES+ (consequence if TEST else OTHER) ARG	(5.3)
RES+VALUE if proposition else alternative ARG	(5.4)
RES+ (VALUE if proposition else OTHER) ARG	(5.5)
RES+VALUE if TEST else alternative ARG	(5.6)
RES+ (VALUE if TEST else OTHER) ARG	(5.7) *

In all the sentences lacking "else", the standalone expressions in "then" & "if", it can be correctly inferred that if (TEST) or (proposition ARG) is false then RES is ARG. In the sentences marked (*) ARG is not referenced and plays no part in the result. It is required only as a syntactic placeholder.

Normal rules of function derivation and grouping with parentheses permit us to add to the complexity without obscuring the sense to produce further constraints as with nested :Ifs or those qualified with :ElseIf and :AndIf but perhaps with greater lucidity. I hesitate to mention :OrIf which I have so far completely failed to circumscribe (i)

```
RES+proposition1 then (proposition2 then consequence           (6)
                      else alternative2) else alternative1 ARG
```

which works as expected.

The ability to write in-line D:fns and if necessary to break them inside of the braces {

thus

} makes it possible to control the visual representation of clauses which some may consider an advantage ...

```
RES+proposition1 then {                                     (6.1)
    proposition2 then consequence else alternative2 w
} else alternative1 ARG
```

Asides

Scarily, we can replace the character flags 'M', 'Q' & 'Y' in "then" & "else" above with arbitrary numbers 2, 1 & 0 and the operators become nearly indistinguishable though the semantics of the identical lines are quite different.

```

-----
then+{                                | else+{
  α+2                                | α+2
  α=1:({ω}◦αα)ω                      | α=1:({ω}◦αα)ω
  α=0:({ω}◦ωω)ω                      | α=0:({ω}◦ωω)ω
  ({ω}◦αα)ω:({ω}◦ωω)ω                | 1 αα ω:0 αα ω
  ω                                  | ({ω}◦ωω)ω
}                                    | }
-----

```

Oh yes, and in case anyone has noticed that none of this touches on the most ubiquitous use of "If", try ...

```

▽ res+{larg}f00 rarg;default
  default+42
  larg+(2=□NC'larg')then{larg}default
  ...
▽

```

Conclusions

I have shown that it is possible to write defined operators that, along with a functional approach to the code being controlled, offer the possibility of a more "natural language" style of programming in place of the commoner control structures. And that writing them on napkins is the quickest way to get them published.

Note

- (i) *circumcapitate* – vt – to get one's head around something.

References:

- [E1] David S. Eastwood. *Structuring functions with operators*. ACM SIGAPL APL Quote Quad archive Volume 24, Issue 1 (August 1993).
- [D1] Bob Dylan. *Don't think twice, it's alright*. Freewheeling Bob Dylan. Columbia. (1963).

Using DyalogAPL based .NET extensions in ASP.NET applications

by Pertti Kalliojärvi, Dinosoft Oy (at the Dyalog APL Users' Meeting)

Introduction

Building Web applications usually includes many evolutionary stages where different programming languages and tools have to be combined. With the new .NET-framework it is relatively easy to add dynamic DyalogAPL components as .NET classes into various parts of IIS/ASP.NET applications and even into the HTTP - pipeline. As a case study we will show how to add several functional extensions (dynamic image generation, dynamic components etc.) into a third party statistical web application called PX-Web. PX-Web belongs to a widely used family of PX-products, which are authored by a consortium of statistical offices in Europe (the main developer being Statistics Sweden).

This extended Web application is used to publish statistical data and automatically created vector graphics using techniques and languages like ASP, ASP.NET, C#, SVG, JavaScript and of course DyalogAPL and RainPro. We will also discuss the difficulties in learning and using .Net and also the pros and cons of involving DyalogAPL in such a complex environment.

PX-Web Publishing System

PX-Web is a system for storing statistical data and publishing it on the web. It belongs to the group of PX-products which are based on open architecture. The main platforms for running PX-Web are Windows 200X Server Families and IIS web servers.

What is PX-Web?

- Web publishing system developed by Statistics Sweden (SCB)
- PX-Web is used for distributing basic statistical data
- Designed for professionals to use statistical data for different purposes
- Used by Statistical offices and international organizations in almost 20 countries
- Used also by public institutions and municipalities
- Development is guided by the international "PX-Axis reference group"

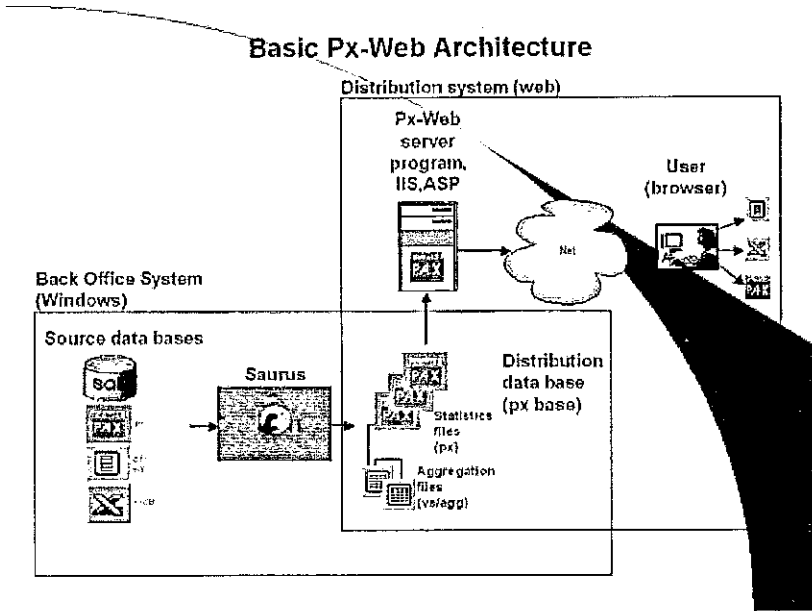


Figure 1. Px-Web Architecture

PX-Web features include

- Navigation in a hierarchical database structure
- Text search to find certain statistical tables directly
- Data selection by classification
- Aggregation of data
- Many different download formats including Excel
- Basic business raster graphics + maps
- Multilanguage application

Px-DataBase features include

- All Px-products use a special file standard: "PX-file", which defines the layout of a statistical table
- Files are stored in text format and can be edited with text editors
- PX-FileSystem is a standard hierarchical file system with little support for concurrent updates etc.
- An Xml version of the file standard is currently being developed

Extensions by Dinosoft

Px-Web is an ASP application, which is the older version of Microsoft Active Server Pages technology. Eventually there will be an ASP.NET release, and we had to design our software extensions to be able to function correctly even with the future releases. This was a very strong argument in favour of independent and pluggable technical solutions. Px-Web extensions can be grouped into four categories:

- a module for creating dynamic graphical images using SVG vector graphics
- a module for dynamic menus, which can use custom authentication and authorization
- a new picture database, which extends static images to automatically updated dynamic ones
- independent modules for logging user activities, error handling, custom authentication and authorization

In this paper we will not be covering the functionality of these extensions because other articles will be available for that purpose. Therefore the next sections will mainly concentrate on showing the techniques and basic ideas for adding modularity to your applications, and how DyalogAPL is used in this context.

The next figure shows the PX-Web functionality after adding Dinosoft extensions:

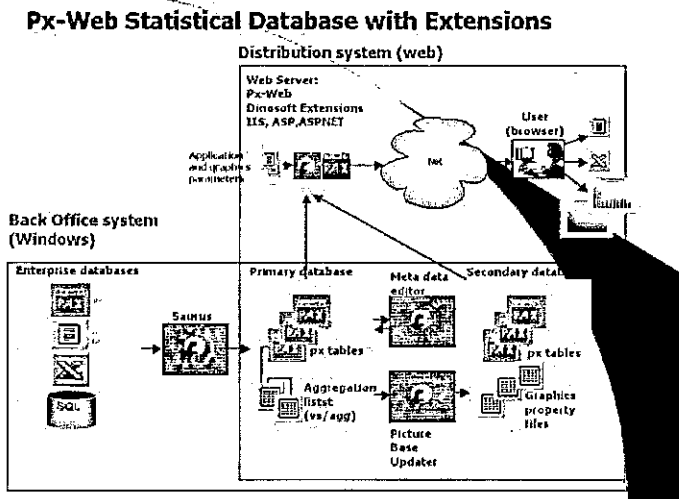


Figure 2. Extended Px-Web Architecture.

Programming in the ASP.NET / .Net environment

Programming and run-time environment management for ASP.NET applications has more “moving parts” than one might guess. The following list shows the different environments and versions into which we currently have to adjust our software.

- **IIS (Internet Information Server)** as the web server: versions 5.0 (W2000 Servers), 5.1 (XP Development), 6.0 (W2003 Servers), [7.0 beta]
- **ASP (Active Server Pages)** as server application runtime environment: ASP, ASP.NET versions 1.0, 1.1, [2.0 beta]
- **.Net Framework:** versions 1.0, 1.1, 1.1.x.x, [2.0 beta]
- **C#:** versions 1.0, [2.0 beta]
- **DyalogAPL:** versions 10.0, 10.1, [11.0 beta]
- **JavaScript/EcmaScript** as the client side scripting languages
- **Client Browsers:** Internet Explorer, NetScape, FireFox, Mozilla, ...
- **Client Browser Plugins:** Adobe's SVG Viewer, ...

So there is a lot of variety regarding possible problems with software updates, testing, troubleshooting etc. Sooner or later development and testing becomes a nightmare unless there is a way to utilize components and modularity. Especially bugs found in a single software module should not force any large recompilation and testing in other application modules. Here is a sample screenshot, where the user has selected some statistical table from her personal menu and gets a graphical output image of that data.

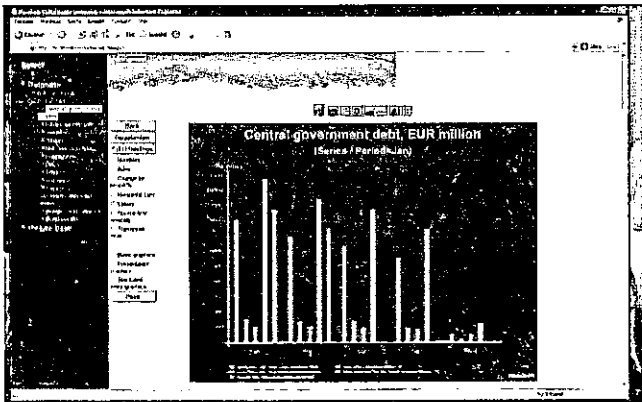


Figure 3. Basic Screen Layout for Menu and Graphics

In the figure on the previous page

- the left side menu is an aspx page (C#), which calls a DyalogAPL class for customizing menu data for each user. Client side activities use JavaScript to launch new functionality to the other window frame on the right.
- the main part of the right-hand window is an aspx page with dynamically created controls generated by C# code on the server.
- the Graphical image in the middle is rendered by Adobe's IE plugin-software and it gets its data from C# and DyalogAPL based server resources. The actual data for the image is generated using RainPro in DyalogAPL on the server.

One way to add modularity to your applications is to use the tools provided by the .Net framework and ASP.NET. They can easily be utilized in software development and help you to build pluggable and maintainable solutions, which customers can further extend according to their needs.

The ASP.NET Http Pipeline

Before going into the details of implementing .Net related components, let's first have a brief overview of the Request Pipeline and its functionality.

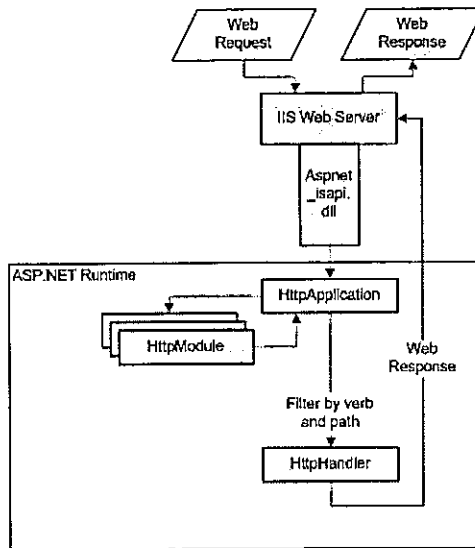


Figure 4. HttpRequest Pipeline (Copyright Microsoft).

When the client browser sends a request to IIS, it maps the extension of the received request according to stored configuration parameters and passes .aspx, .asmx, plus other extensions to aspnet_isapi.dll, which is an ISAPI filter. This in turn starts a new ASP.NET runtime (a chain of managed objects dubbed the HTTP Pipeline). An object of the `HttpApplication` class, which is the class behind your global.asax file, is created and started and it acts as the host for the ASP.NET Web application. This functionality differs between different versions of IIS and ASP.NET, but usually the following steps are the same

- `HttpApplication` reads the application (web.config) and machine configuration files (machine.config).
- It passes the request to a members of `HttpModules`, which return the request after providing some preprocessing on the request. Each module has its own functionality and each one responds to a certain predefined event.
- `HttpApplication` figures out what type of handler can best handle the request and finally passes the request to the selected and instantiated handler. The selection is based on the verb and path of the request.
- The handler processes the request and returns the corresponding response.

In ASP.NET you can extend, replace or reconfigure any of the module or handler classes or provide your own custom modules and handlers. Both of these can be configured at the application level or globally on machine level with *.config files.

HttpModules

`HttpModules` are in fact event-handlers: they respond to a certain event fired in the `HttpPipeline`. A module subscribes to an event by providing a method, which gets executed.

The request pipeline has a predefined set of events which will all be fired in a certain order (`BeginRequest`, `AuthenticateRequest`,...`EndRequest`) during the lifetime of the request. There are also other events which can be processed with `HttpModules` (for instance `ApplicationError`).

It is good to remember that these modules will be executed for all requests and therefore performance issues should be considered carefully before adding your own modules into the pipeline.

For defining a module you need to implement the module interface `IHttpModule` in your class definition and add the following section to your web.config file

```
<httpModules>
  <add name = "ModuleName" type = "ModuleType"/>
</httpModules>
```

where *name* stands for an arbitrary chosen module name and *type* is defined as "ModuleNameSpace.ModuleClass, ModuleAssembly".

You hook a module to your application by copying its dll to /bin - directory and adding the code above to your web.config file.

HttpModules can be used for URL-rewriting, authentication, authorization and for anything where the needed data is included in the request or response. Logging user activity is one example.

HttpHandlers

HttpHandlers are programs responsible for serving requests. Each time you define an aspx-page, you actually make a new **HttpHandler**. This is because a handler is defined by implementing an interface called **IHttpHandler** and that is what the **System.Web.UI.Page** class does. A handler for a certain resource serves the request by delivering a corresponding response. Defining a handler is very similar to defining a module: you implement the **IHttpHandler** interface and add some configuration information to your web.config file:

```
<httpHandlers>
  <add verb="*" path="*.extension" type="HandlerType" />
</httpHandler>
```

Here *verb* stands for the verb used in the request (GET,SET,POST) and *path* defines path and extension for this handler (the URL in the request). Type is once again defined as "HandlerNameSpace.HandlerClass, HandlerAssembly".

Adding a handler to your application works the same way as with modules: copy assembly to /bin and edit web.config.

One more thing has to be done, if you define a totally new extension: IIS must be configured to pass this request to the ASP.NET runtime. Otherwise IIS will try process it itself and the request will never reach your handler. With handlers you can easily serve requests for special resources and as a side effect implement web pages which do not actually exist.

With HttpModules and HttpHandlers it is possible to get

- re-usable components and modularity
- very easy installation and removal with web.config

- application-independent solutions
- well-defined programming interfaces for your components
- a way to avoid the burden of recompiling existing solutions

Using DyalogAPL with PX-Web?

Why DyalogAPL?

The background for integrating DyalogAPL components into PX-Web was that we were asked to explore the possibility of making a vector graphics extension in addition to the existing Px-Web graphics component, which is still raster based.

As we were familiar with RainPro's capabilities (and that time SharpPlot did not exist) we decided to try to make a component in DyalogAPL. The other alternative of using various control libraries seemed to be too difficult to handle after trying some of them.

Because statistical data is often multi-dimensional, one thing soon led to another and we were 'forced' to make more components in DyalogAPL.

We tried APL in various forms (APLScripts, aspx-pages, class libraries) but somehow we are gradually moving towards solutions where APL is only used as a pure .Net class without any UI components. There are several reasons for doing so, the most important one being the reusability of such classes.

Currently Px-Web Extensions include DyalogAPL in

- Graphics application (SVG graphics with Rainpro)
- Dynamic menu (based on each user's authorization).
- Dynamic image database (which is automatically updated when source data changes)

The technical ways of using DyalogAPL are

- pure independent .Net classes (used by other classes)
- functional classes used with interfaces (graphics generator, menu)
- pure .Net classes for database handling (custom databases i.e. picture base)
- utility libraries

Because the traditional use of APL is something we are all familiar with, the next chapters will cover topics perhaps not so widely known. The first example shows

how to design a `HttpModule` both in C# as well as in APL. The second case study shows a combination of both languages in implementing a `Handler`.

Building your own `HttpModules`

In Px-Web we have used custom `HttpModules` for

- Custom authentication and authorization
- Error logging
- Session management

While building these modules we needed to get access to a third party software which used a customized cookie-based authentication. The test phase was carried out to generate similar cookies in a `HttpModule`, which could be plugged in to any application.

Cookie `HttpModule`

Implementing a `HttpModule` is just as simple as using the corresponding interface class as the base class and writing your versions of every needed method and property.

The interface for `HttpModules` contains only two methods: `Init` and `Dispose`. `Init` is used for assigning your event-handler method to an event and `Dispose` is just a dummy method belonging to this interface.

So we have (in C#):

```
namespace Graph.Modules {
    using System;
    using System.Web;

    public class CookieWriter: IHttpModule {
        public void Init (HttpApplication application) {
            application.BeginRequest += (new
                EventHandler(this.WriteCookie));
        }
        public void Dispose() {
        }
    }
}
```

The `CookieWriter` class implements `IHttpModule` with two public methods. `Init` method just assigns our `WriteCookie` method to the `BeginRequest` event so that this method will be called on every request.

This is very dangerous, but this class is only used for test purposes.

Then we need the code for the WriteCookie method:

```
private void WriteCookie(Object source, EventArgs e) {
    HttpApplication app = (HttpApplication) source;
    HttpContext context = app.Context;
    string cdata = String.Empty;
    ... Get cookiedata from somewhere ...
    cdata = ...
    HttpCookie ck = new HttpCookie(cookieName,cdata);
    context.Request.Cookies.Set(ck);
}
```

It gets cookie data from somewhere and then makes a new HttpCookie instance. This instance is added to the Collection of the current request. (It would have been better to use a copy, but this is an example.)

After compiling our class to a dll (Cookie.dll) and moving it to /bin directory, we only have to change our web.config by adding the following module information into <httpModules> section.

```
<httpModules>
...
<add
    name ="cookie1" type = "Graph.Modules.CookieWriter,
    Cookie" />
```

Case: DyalogAPL version/cookie

The DyalogAPL version is quite similar, but there are minor changes. I have tested this solution and it works, but there might be more elegant solutions for passing the eventhandler information to application events.

```
)ns CookieTest
#.CookieTest
)cs CookieTest
#.CookieTest
    [using+'System' 'System.Web,System.Web.dll' 'Util, Util.dll'
    'CookieGenerator' ]wc 'NetType' ('Interfaces' 'IHttpModule')
)cs CookieGenerator
#.CookieTest.CookieGenerator
)ed Init
)ed Dispose
)ed AddCookie

[0] Init appl;eh
[1] A Init is the basic method for IHttpInterface where you
[2] A first create a new eventhandler and then assign it to
[3] A some event in the HttpPipeline
[4] A appl: HttpApplication
```



```

[5]
[6]  A Let's create a new eventhandler to be called
[7]  ehΔEventHandler.New(ΔOR 'AddCookie')
[8]
[9]  A Next we assign it to application BeginRequest event
[10] A Here we use a little utility class to do the
[11] A actual assigning
[12] Util.EvUtil.BeReqAdd appl

[0]  Dispose
[1]  A Dispose is part of the IHttpModule interface
[2]  A Do nothing in this

[0]  AddCookie x;appl;args;req;cookie
[1]  A This is a callback function for some web application event
[2]  A It takes standard arguments: source, args.
[3]
[4]  appl argsΔx
[5]
[6]  A Source refers to HttpApplication, which has HttpContext property
[7]  A which in turn has a HttpRequest property.
[8]
[9]  reqΔappl.Context.Request
[10]
[11] A Then we generate a new HttpCookie and assing our data to it.
[12] cookieΔHttpCookie.New 'mycookie'
    'this is data: a very important cookie test string'

[13]
[14] A And finally we add this new cookie to request's CookieCollection
[15] req.Cookies.Set cookie

```

The little C#-method in the Util namespace only assigns our event-handler to an application event (here BeginRequest):

```

namespace Util {
    using System;
    using System.Web;
    public class EvUtil {
        public static void BeReqAdd (HttpApplication
application,
                                EventHandler eh)
        {
            application.BeginRequest += eh;
        }
    }
}

```

I moved this simple assignment statement outside APL due to syntax difficulties, which might occur with some events. The utility can also be used from any .Net language and at the same time one does not have to worry about operational

difficulties with multicast delegates. So far this has only been used in test environments.

Next we have to assign the .Net related properties to our new functions:

- Init takes a `HttpApplication` argument and returns `Void`
- Dispose takes no arguments and returns `Void`
- AddCookie takes typical eventhandler arguments: `Object` and `EventArgs` and returns `Void`

After this we are ready to Export our workspace to a dll library and copy it to applications /bin directory.

The last thing we still need to do is edit our web.config by adding information to `<HttpModules>` section before our cookiegenerator is ready to be used.

When the next user-request arrives for this application, each request will be added as a custom cookie by our new `HttpModule`.

I used different class names so that both versions actually run at the same time. The reason for this is the way that events are assigned: each event has several eventhandler delegates and they all will be called when the event (`BeginRequest`) fires. After that we have two extra cookies in our request.

Of course you can always use the *traditional way* of handling application events by adding your code directly to the `global.asax` file, but then you have to recompile this (perhaps a code behind) file. In this case you will lose the dynamics of true plugins where code can be added and removed without touching any parts of the actual application.

In `global.asax` this would be :

```
...
protected void Application_BeginRequest(Object sender, EventArgs e)
{
    new CookieTest.CookieGenerator().AddCookie(sender, e);
}
...
```

Building your own HttpHandlers

The second case study uses C# code for building the `HttpHandler` and `DyalogAPL` classes to do the actual dirty work. Of course this time you can do it directly in APL without C#, but when you look at the code, you'll see a comment explaining why we use C#.

Steps we need to take:

- create a class that implements `IHttpHandler`
- create a `DyalogAPL` class that does the actual work (i.e. generates SVG)
- modify `web.config`
- let IIS know about the new resource

In fact we are implementing the rightmost parts of the following true scenario:

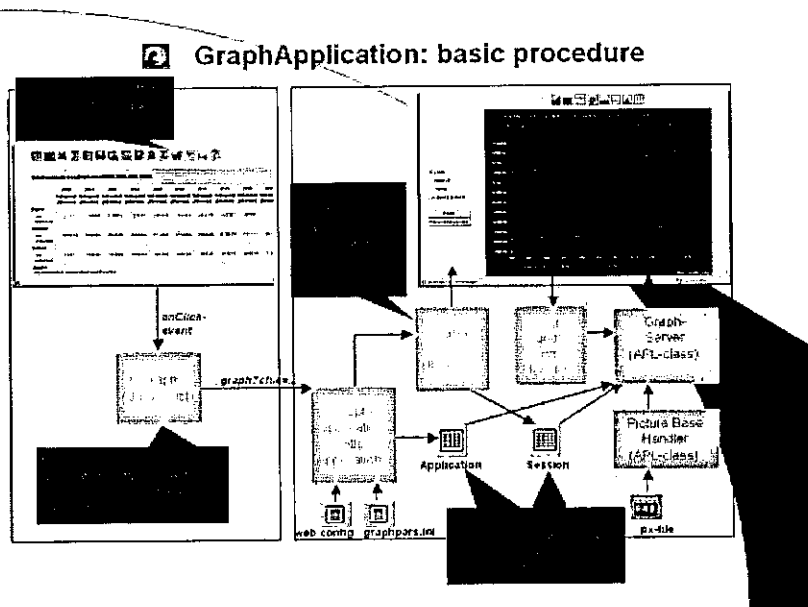


Figure 5. Basic steps in invoking dynamic graphics.

Here we have a `HttpHandler` which serves requests assigned to `'svg'` extensions and return `svg` image definitions (xml to be precise) for the client.

Case: Svg-Handler

Svg-handler generates images using url-parameters, global data from the workspace, and data from global initial parameters (called when the application starts) and it stores global data in the APL workspace.

The skeleton for the HttpHandler is very simple:

```
// DemoHandlers.cs
// By Dinosoft
namespace Graph.Handlers
{
    using System;
    using System.IO;
    ....
    /// <summary>
    /// Summary description for SvgHandler.
    /// Author: Dinosoft Oy - PK
    /// </summary>

    public class SvgHandler: IHttpHandler,IRequiresSessionState
    {
    public override void ProcessRequest(HttpContext context)
    {
    string output = string.Empty;

    // first we just check that this request is for a dynamic image
    if (context.Request.Path.IndexOf(GraphApplication.Graphserver)>-1)
    {
    // Get some parameters from the request and/or from user session
        NameValueCollection rs=context.Request.QueryString;
        string par1 = (rs["parname1"])...;
        string par2 = (rs["parname2"])...;
        string par3 ...
        ...
    // Get corresponding px-data from the picture base
    //(which uses DyalogAPL classes)
        GraphFile gf=new GraphFile(par1,par2,...);
    // Create a instance of our graphics generator
    //(which is a DyalogAPL class)
        GserverPage gp=new GserverPage(par3,par4,...);

    // Pass database information for the generator
    // to make the desired image
        output=gp.MakeSvg(gf);
```

```
}
else
{

    // This request was for a static resource,
    // so get it from somewhere else
    // We don't want to drag this stuff into APL
    // do we ?

}
// and finally write the response
context.Response.ContentType="image/svg+xml";
context.Response.Write(output);
}

public virtual bool IsReusable
{
    get { return false; }
}
}
```

What the code above does is:

- Create a handler class from `IHandler` base class and add `IRequiresSessionState` interface just to be able to use and store Session variables.
- `IHandler` has two methods: `ProcessRequest`, which does all the work and `IsReusable`, a boolean method which can be set to return false in this context.
- `ProcessRequest` gets a `HttpContext` class as input and uses one of its properties to find the actual Request instance.
- After that we dig some variables from the Request URL and use some of them to get data from the database
- `GraphFile` class is a `DyalogAPL` class, which manages the whole Picture Base
- We get a new instance to `GraphFile` and pass it to another `DyalogAPL` class called `GServerPage`, which in turn makes graphical images with the help of `RainPro`
- `MakeSvg` method of this class returns xml text to be flushed into the Response object.
- Finally the client browser receives this response and passes it to a client plugin (Adobe's SVG Viewer) which renders this xml code into an image.

We won't be showing any DyalogAPL code in this example, because space is limited. The following figure will give you an idea what is happening inside the APL workspace.

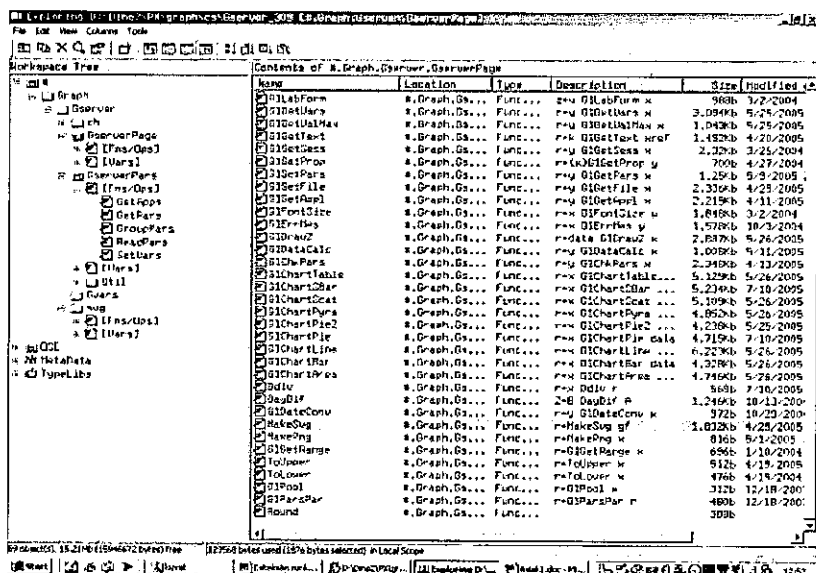


Figure 6. DyalogAPL classes for making graphical images.

In `web.config` we still have to add some configuration information:

```
<httpHandlers>
    <add verb="*" path="*.svg"
type="Graph.Handlers.SvgHandler,Graph.Handlers" />
```

This time we also have to configure IIS to pass '.svg' extensions to ASP.NET. That can be done easily using the IIS Manager.

Every request following this application path that has the extension `'svg'` will be forwarded to our new `HttpHandler`. It is supposed to manage all internal exceptions and return a response corresponding the client side expectations of the responses contents type.

How to Build and use General Plugins?

In fact things are not as simple as the two examples above may suggest. If we want to pass startup parameters to our new modules or handlers, we should do it in a similar way: independently of the current application. This can also be done with web.config by adding a section where we place our configuration information.

Luckily asp.net provides us most of the framework needed to read this information, but complex data has to be dealt with otherwise.

Another thing is to find a way to use customer/third party extensions which are designed to extend our own application extensions i.e. when someone has derived a class from our base class and we should be able to use it in our application.

One of our HttpModules (coded in C#) uses this kind of approach where the customer can implement her/his own derived class to be used instead of our base class. The .Net Framework also provides facilities for building such plugin solutions.

Here are some general principles for passing information and designing general plugins for both asp.net and windows applications.

- Create your own interface or a base class to be derived from
- Create a functional class that implements this interface (or have someone program it for you)
- Configure your application to find the information about this new plugin class (web.config in asp.net or App.Config in Windows.). Basically it is just a new section in the configuration file.
- Create a parser that reads this configuration information i.e. the type of the class and perhaps some parameters for it (for its constructor). Register this parser as a configurationhandler. if your interface is simple enough, you can use existing parsers as in the example below.
- Read the configuration information, instantiate the new class with a special class called **System.Activator** and cast it to the base class. [Here we need the inheritance schema and virtual methods from C#, because the derived class does not even exist when we deliver our base class application.]
- Invoke this new instance of your base class by calling its methods.

So far we haven't tried this in DyalogAPL, but there is actually no reason why it should not work after minor adjustments.

Below we have an example of a section in a configuration file, which defines a new group called <graph.logs>. This section just groups a set of subsections for us. The subsection <userLog> contains the relevant information: which

implementation of userLog should be used. The type field has the class name and assembly information to be passed to our application. This class is a derived class from our base class, and was implemented afterwards.

```
<configuration>
  <!-- Allows for a new section group to the Web.config -->
  <configSections>
    <sectionGroup name="graph.logs">
      <section
        name="userLog"

type="System.Configuration.SingleTagSectionHandler,
      System, Version=1.0.5000.0, Culture=neutral,
      PublicKeyToken=b77a5c561934e089" />
    </sectionGroup>
  </configSections>
  <graph.logs>
    <userLog
      type="Graph.Logs.XmlUserLog,
Graph.Modules.Logs"/>

    <!--
    <userLog
      type="Graph.Logs.CompFileUserLog,
      Graph.Modules.Logs" />
    -->
  </graph.logs>
...

```

Pros and Cons of using DyalogAPL

Here is a short list of pros and cons of using DyalogAPL to be used as a reference in further discussions.

Pros

- It is the best language for multidimensional data
- Lots of tested code to be reused in .Net
- Dyalog development tools (trace, thread tool, ...)
- Workspace structure allows special tricks (if documented)
- Code security: it is difficult to generate source code from a runtime assembly

Cons

- Installation is difficult for ASP.NET applications
- Debugging has to be done inside APL
- Workspace structure allows special tricks (if not documented)
- The interface to .Net is slow
- You cannot make a module instead of an assembly
- Sometimes it takes a long time to figure out the correct syntax to use .Net from APL
- Expensive to call APL from APL via .Net interfaces

Conclusion

We have touched the surface of the wonderful world of HttpModules and HttpHandlers and shown some examples of how Dyalog APL has been used in this context. Hopefully there will be more elegant examples and shared solutions to be seen in the near future.

References

Several articles in the MSDN library written by Scott Mitchell, Atif Aziz, Roy Osherove, Dino Esposito, Rick Strahl and others.

At Play with J: Belgian Numbers

by Eugene McDonnell

Belgian numbers were recently introduced by Eric Angelini. For N to be a Belgian number, it is necessary that it appear in the sum scan of lots of replications of N's digits. For example, try the digits of 16:

```
+/\8 $ 1 6
1 7 8 14 15 21 22 28
```

So 16 isn't Belgian. What about 17?

```
+/\8$ 1 7
1 8 9 16 17 24 25 32
```

So 17 *is* Belgian.

My first concern on hearing of them was to find out how one could tell whether a number was Belgian or not. Assume that N is 176; how many copies of 1 7 6 would be needed to reach 176 or thereabouts? This can be found by taking the ceiling of N divided by the sum of its digits S. Like this:

```
N =: 176
] D =: 10&#. ^: _1 N
1 7 6
] S =: + / D
14
] C =: >. N % S
13
```

Then multiplying this by its number of digits, in this case, 3, and sum scanning:

```
] R =: + / \ (C * 3) $ 1 7 6
1 8 14 15 22 28 29 36 42 43 50 56 57 64 70 71 78 84 85 92 98 99
106 112 113 120 126 127 134 140 141 148 154 155 162 168 169 176
182
```

and, sure enough, we find that 176 is a Belgian number. This was, for the moment, an easy way to determine which numbers were Belgian – if they were not too big. I thought it might be possible to use it for numbers up to about ten million – but thereafter it gets ridiculous. For example, suppose that one wants to see whether 1234567898765 is Belgian:

```

N =: 1234567898765x
] D =: 10 &. #. ^_1 N
#D
13
] N=:+/D
71
>. B % S
17388280265
13 * >. B % S
226047643445

```

Ouch. A fifth or so of a trillion (US style) digits. I concluded that this was impractical.

There had to be a better way. Eventually I thought I saw that in the case of 176 there was a threeness lurking. I decided to form them into a table of three-item rows, together with the first difference of the rows, and the 14-residues of the whole table:

```

qn =: 13 3 $ + / \ 39 $ 1 7 6
qn      ; (2--/\qn); 14|qn
+-----+-----+-----+
| 1   8  14|14 14 14|1 8 0|
| 15  22 28|14 14 14|1 8 0|
| 29  36 42|14 14 14|1 8 0|
| 43  50 56|14 14 14|1 8 0|
| 57  64 70|14 14 14|1 8 0|
| 71  78 84|14 14 14|1 8 0|
| 85  92 98|14 14 14|1 8 0|
| 99 106 112|14 14 14|1 8 0|
|113 120 126|14 14 14|1 8 0|
|127 134 140|14 14 14|1 8 0|
|141 148 154|14 14 14|1 8 0|
|155 162 168|14 14 14|1 8 0|
|169 176 182|      |1 8 0|
+-----+-----+-----+

```

The first row of the first table shows that the sum of the integers is 14; the second table shows that each row differs from the previous row by 14, the third table shows that the 14-residue of each row is 1 8 0.

The point made by the second table is that each row after the first is formed by adding 14 to the preceding row. This means that I could create qn exactly by adding multiples of 14 to the starting row:

$$(14 * i, 13) + / 1 \ 8 \ 14$$

1	8	14
15	22	28
29	36	42
43	50	56
57	64	70
71	78	84
85	92	98
99	106	112
113	120	126
127	134	140
141	148	154
155	162	168
169	176	182

The point made by the third table is that any concern about residues after the first row is irrelevant.

With this information at hand, we're almost there. The number 176 is 8 beyond 168, which is $14 * 12$. The $14 * 12$ is irrelevant; the 8 is significant. It means that 8, which is $14 \mid 176$, is of great importance. It is one of the numbers in the first row of the table, and after being added to by multiple 14s, must arrive at 176, and so, is Belgian! This is a perfectly general observation, and allows any integer to be tested for Belgianity. Here is an outline of the steps to take to see if N is Belgian:

Let D be the digits of N.

Let S be the sum-scan of the digits of N.

Let T be the sum of D (which is also the last item of S).

Let R be the T-residue of N.

Then N is Belgian if R is in S.

For the case of $N = 176$, we have:

D is 1 7 6

S is 1 8 14

T is 14

R is the T residue of N, or 8.

We ask: is R in S? The answer is Yes!

The general case is that N is Belgian if:

$$(M \mid N) \in S$$

is 1.

Now it's clear that 176 is Belgian if any of these residues is the same as $14 \mid 176$:

$$\begin{array}{r} 14 \mid 176 \\ 8 \end{array}$$

It is certain that if one adds the proper multiple of 14 to 8, the result must be 176. It isn't necessary to find what this multiple is. It is sufficient that there is an 8 in the sum scan of $1 \ 7 \ 6$.

Now we can write an expression to let us determine whether 176 is Belgian:

$$\begin{array}{r} (14 \mid 176) \in + / \setminus 1 \ 7 \ 6 \\ 1 \end{array}$$

Another way of doing this is: since 8 is the 14 residue of 176, it is sufficient to see whether there is a zero in the 14 residue of $176 - + / \setminus 1 \ 7 \ 6$

$$\begin{array}{r} 14 \mid 176 - 1 \ 8 \ 14 \\ 7 \ 0 \ 8 \end{array}$$

So we may write:

$$\begin{array}{r} 0 \in 14 \mid 176 - + / \setminus 1 \ 7 \ 6 \\ 1 \end{array}$$

to determine that 176 is Belgian.

We can now work with large numbers with some confidence. First, we'll need a way of getting the integer components of N. The function `dig` works very well:

```
dig =: 1j1 & # &. ": @ x:
```

We'll use this on 1234567898765:

```

    dig 1234567898765
1 2 3 4 5 6 7 8 9 8 7 6 5
+ / \ dig 1234567898765
1 3 6 10 15 21 28 36 45 53 60 66 71
+/1 2 3 4 5 6 7 8 9 8 7 6 5
71
71 | 1234567898765 - + / \ dig 1234567898765
20 18 15 11 6 0 64 56 47 39 32 26 21

```

There is a zero in this list, indicating that N is Belgian. We can verify this.

```

N =: 1234567898765x
S =: + / \ dig N
M =: {: s

S
1 3 6 10 15 21 28 36 45 53 60 66 71

M
71
Q =: <. N % M

Q
17388280264

P =: Q * M

P
1234567898744

R =: S + P

R
1234567898745 1234567898747 1234567898750 1234567898754
1234567898759 1234567898765 1234567898772 1234567898780
1234567898789 1234567898797 1234567898804 1234567898810
1234567898815

N e. R
1

```

So 1234567898765 is Belgian.

Here is a function that will determine whether or not a number is Belgian:

```
BN =: 3 : 0 " 0
N =. x: ^: (y. >: 2147483648) y.
'S M' =. (] ; {:) + / \ (1j1 & # &. ":) N
(M | N) e. S
)
```

The first line makes the argument an extended integer N if it is larger than the largest positive 32-bit integer, and otherwise leaves it alone. M and S are modulus and digits sum, as discussed above. See if BN agrees that N is Belgian:

```
BN 1234567898765
1
```

Why Belgian?

Why Belgian, indeed. Well, there are Roman numbers, Arabic numbers, even Catalan numbers. It's about time Belgium was recognized. I could pretend that Eric Angelini (the creator of these numbers) knew that my wife Jeanne was a Fulbright Scholar at the University of Brussels for the 1952-1953 year – and named them Belgian in her honour. But I have to admit that this is quite far-fetched. The truth undoubtedly is – because Angelini is Belgian.

J-ottings 45: A bit of OUTReach (or do I mean outrEACH?)

by Norman Thomson

In APL2 there was just one primitive symbol (double dot) for "each" – the "pepper" effect is well imprinted on some of us with long memories, although, as Stephen Taylor hinted in his last editorial, APL just doesn't look like that any more! I find it interesting that in spite of J being considerably richer in primitives than APL, it does not include "each" as a primitive. Nevertheless it is customary for many users to define an adverb "each", even although it requires more characters to type than the J which defines it! Unfortunately, depending on the writer, the definition is sometimes `&>`, sometimes `&.>`, and occasionally `"0 . I` do not recommend this last practice, since a verb-noun combination is effectively one major case of a primitive operation, and in my view is best recognised as such. I therefore confine discussion of adverbial "each" to the other two possibilities, defined as

```
each=.&>      NB. uses bond
EACH=.&.>     NB. uses compose
```

The definitions of the conjunctions `u&v` and `u&.>v` in mathematical terms as (uv) and (v^1uv) show that the rule

```
(f each) -: (>f EACH)
```

is completely general, and so only one of `each` and `EACH` is logically necessary. `EACH` has the general merit of greater compactness through generating what APL2 users would recognise as ragged arrays. On the other hand `each` has the merit of avoiding, or at least reducing, boxing on display, but often at the expense of requiring greater numbers of fill characters to achieve global homogeneity. In writing J it can be very desirable to have a 'rule of thumb' appreciation of how `each` and `EACH` work without having to consider the semantic details of "box" and "open" at every point of use.

Thinking in terms of lists is helpful in using `each` and `EACH` effectively, as is sensitivity to the ways in which argument rank works. For example, given

```
h=.i.2 3      NB. two 3-lists, rank = 2
```

compare the following


```

      h,each 6 7      NB. result=one 2-list, items=three 2-lists, r
rank=3
0 6
1 6
2 6

3 7
4 7
5 7

```

```

      h,EACH 6 7      NB. result=two 3-lists, items=2-lists, rank=2
+---+---+---+
|0 6|1 6|2 6|
+---+---+---+
|3 7|4 7|5 7|
+---+---+---+

```

Applying the general rule above (`h,each 6 7`) is identical to `>h,EACH 6 7`, or to put it in another way, the two displays above show the same six 2-lists, in one case individually boxed, in the other integrated into a structure of higher rank.

Using either `EACH` or `each` with a dyadic verb often requires the user to box one of the arguments to achieve a desired result through bringing about correct argument matching. Often this consists of a simple boxing of one of the arguments as in

```

      x=.6 7;8;9 10 11 12
      (<h),EACH x      NB. left rank 0, right rank 1, result rank 1
+---+---+---+---+
|0 1 2|0 1 2|0 1 2 0|
|3 4 5|3 4 5|3 4 5 0|
|6 7 0|8 8 8|9 10 11 12|
+---+---+---+---+

```

The result is a 3-list of scalars (and so of rank 1) each made from the same scalar (`<h`) joined to each item of a 3-list, with everything opened and boxed following the (`v-1uv`) rule which defines `&.` (under). There is a local fill character in the first box, a local scalar expansion in the second, and the third containing two fill characters in the final column has a different shape (\$) from the other two. Compare this with

```

      (<h),each x      NB. left rank 0, right rank 1, result rank 3
0 1 2 0
3 4 5 0
6 7 0 0

```

```
0 1 2 0
3 4 5 0
8 8 8 0
```

```
0 1 2 0
3 4 5 0
9 10 11 12
```

where there is no v^{-1} boxing. Again the result is a 3-list, but now not of scalars but of three filled lists, each of which is a 4-list of scalars after filling. APL2 talked about "ragged" and "heterogeneous" arrays – a more appropriate differentiation in J is between locally- and globally-filled lists, corresponding to *EACH* and *each* and respectively. Using "0 as an alternative gives a result rank between those of *EACH* and *each* :

```
(<h), "0 x      NB. left rank 0, right rank 1, result rank 2
+-----+
|0 1 2|6 7      |
|3 4 5|          |
+-----+
|0 1 2|8         |
|3 4 5|          |
+-----+
|0 1 2|9 10 11 12|
|3 4 5|          |
+-----+
```

Here the scalar left argument gives rise to scalar replication, and the result is a 3-list of non-homogeneous 2-lists without fill characters, that is, it is only the *display* which is rectangular not the result object itself, as is made explicit by

```
$EACH (<h), "0 x
+-----+
|2 3|2|
+-----+
|2 3| |
+-----+
|2 3|4|
+-----+
```

In order to avoid monotonous repetition, subsequent examples will use just one of *EACH* or *each*, the other case being covered by the general rule $(f \text{ each}) - :$ $(>f \text{ EACH})$. Consideration of rank is almost always a preliminary to well-considered usage of *EACH*. There are no overall rank rules because these depend on the semantics of each individual f . In the next set of examples three figures in square brackets in a comment give the left, right and result ranks of the expression

being commented. Consider first the basic case of the verb "shift" with a rank 1 argument on the left and a rank 2 argument on the right :

```

      0 1|.h      NB. 0-shift on rows ; 1-shift on cols [1 2 2]
    1 2 0
    4 5 3

```

Applying EACH with "box" on either right or left changes the ranks :

```

      0 1|.EACH<h  NB. 0-shift on h ; 1-shift on h      [1 0 1]
+-----+-----+
|0 1 2|3 4 5|
|3 4 5|0 1 2|
+-----+-----+

```

```

      (<0 1)|.EACH h  NB. (<0 1)-shift applied to items of h [0 2 2]
+---+---+
|0|1|2|
+---+---+
|3|4|5|
+---+---+

```

Between these extremes is the possibly more useful case of "left rank 1, right rank 1":

```

      0 1|.EACH<"1 h  NB. 0-shift to 1st row ; 1-shift to 2nd [1 1 1]
+-----+-----+
|0 1 2|4 5 3|
+-----+-----+

```

Next; given the laminated rank 3 object :

```

      ]hlam=.h,:h+10  NB. hlam is h laminated with h+10
    0 1 2
    3 4 5
   10 11 12
   13 14 15

```

here are the effects of four rather similar expressions, along with annotated descriptions which attempt to explain the differences between them. What is the best medium for such descriptions?

Why J, of course, hence the comments which follow each of the four executable lines supply matching equivalent statements

```
(<0 1)|.EACH <"2 hlam      NB. (0 1|.h);(0 1|.h+10)    [0 1 1]
+-----+
|1 2 0|11 12 10|
|4 5 3|14 15 13|
+-----+
```

```
0 1|.EACH <"2 hlam      NB. (0|.h);(1|.h+10)          [1 1 1]
+-----+
|0 1 2|13 14 15|
|3 4 5|10 11 12|
+-----+
```

```
0 1|.EACH <"1 hlam      NB. (0|.EACH<"1 h),:(1|.EACH<"1 h+10)
+-----+
|0 1 2   |3 4 5   |      NB.                      [1 2 2]
+-----+
|11 12 10|14 15 13|
+-----+
```

This example is also equivalent to `2 2$0 0 1 1|.EACH ,<"1 hlam`, since `<"1 hlam` is a 4-list whose items are 3-lists such as `0 1 2`. The next example illustrates how "0 means apply a shift *at the scalar level* which is necessarily a "do nothing" operation.

```
(0 1)|."0<"2 hlam      NB. equivalent to <"2 hlam    [1 1 1]
+-----+
|0 1 2|10 11 12|
|3 4 5|13 14 15|
+-----+
```

A further set of examples illustrates what happens with characters. `g` is a pair of 2-lists and thus has the same shape as `h`, only its items are characters lists rather numbers.

```
]g=.2 3$'ant';'bee';'cat';'dog';'elk';'frog'
+-----+
|ant|bee|cat |
+-----+
|dog|elk|frog|
+-----+
```

Where items are characters lists, there are necessarily possible ambiguities between genuine space characters and space fill characters, something which is clarified in

1 .EACH g	NB. 1-shift at item level	[0 2 2]
+-----+		
nta eeb atc		
+-----+		
ogd ke rogf		
+-----+		

The next example shows simultaneous opening of both arguments followed by v^{-1} boxing :

(<0 1) .EACH <g	NB. <0 1 .g	[0 0 0]
+-----+		
+-----+		
bee cat ant		
+-----+		
elk frog dog		
+-----+		

The next example is one in which the result rank is less than an argument rank :

0 1 .EACH <g	NB. (<0 .g),(<1 .g)	[1 0 0]
+-----+		
+-----+		
ant bee cat dog elk frog		
+-----+		
dog elk frog ant bee cat		
+-----+		

In the final example in this set the shapes of the items within the two outer boxes are not identical, thus resolving an ambiguity between fill and genuine space characters which each would not have shown :

0 1 .EACH<"1 g	NB. (<0 .0{g),(<1 .1{g)	[1 1 1]
+-----+		
+-----+		
ant bee cat elk frog dog		
+-----+		

However, given that each by definition reduces one level of boxing in every case, it often gives less fussy looking results than EACH where character lists are involved.

The examples so far have been chosen to illustrate the principles of each and EACH. An example of a practical problem involving each is that of evaluating linear expressions such as $x + 2y - z$ over separate ranges of values of their variables. The results in this simple example are easy to check.

```

t3=.1 2;3 4 5;6NB. x={1,2} y={3 4 5} z={6}
{t3} NB. catalog generates all combinations
+-----+-----+-----+
|1 3 6|1 4 6|1 5 6|
+-----+-----+-----+
|2 3 6|2 4 6|2 5 6|
+-----+-----+-----+

ip=.,+/. * each NB. inner product each
({t3})ip <1 2 _1 NB. 1 2 _1 are a set of linear
                    coefficients

1 3 5
2 4 6

```

The advent of "each" and ragged arrays in APL2 breathed new life into the language. J deals with such matters more subtly by forcing choices between two possibilities! Broadly speaking EACH makes objects more compact by allowing raggedness and reducing homogeneity to a local level, each reduces boxing at the cost of imposing greater homogeneity through globally applied fill characters.

PROFIT

ARQUE - Assessment Reports on the Quality of University Examinations

by Alan Sykes (given at the Dyalog APL Users' Conference 2005)

Introduction

ARQUE, an acronym for 'Assessment Reports on the Quality of University Examinations' is a software product, written in Dyalog APL, using many of the Causeway products (CPro, Newleaf, and Rainpro). It was developed in the last five years of my employment in the Business School at the University of Wales Swansea. It is now in use at universities in Swansea and Bangor, with other customers in the pipeline for later this year.

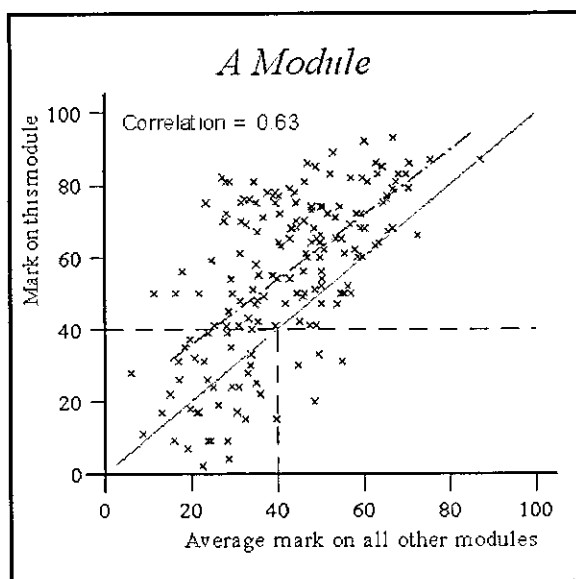
University examinations are produced by university lecturers themselves, subject usually to the scrutiny of external examiners who are there to check consistency of standards across academic institutions. Universities themselves are subject to 'quality inspections', which would seek, amongst other things to be re-assured that each university has a quality control system that guarantees high standards in its assessment of student performance. My work developed from the 'bottom up' where I was concerned with individual departments within a university and how each department could obtain useful feedback from the examination processes it was responsible for. If a department could establish a clear appraisal of its examinations which demonstrably indicated where quality was assured and also flagged problems if they occurred, then the university concerned could point to such appraisals, and take remedial action where appropriate. If such a system were used amongst all departments, then clear criteria can be developed at a managerial level to assure the quality of all its university examinations.

Modern universities, some more so than others, consist of a plethora of different degree schemes, each specified by a list of modules that students must take in each year of that scheme. Some modules may be compulsory, some optional, and typically, any module may be used by more than one degree scheme. Hence, from the perspective of a specific university lecturer, faced with a collection of marks

for the module he or she is responsible for, it is a non-trivial task to explain, for example, why the marks are so low/high as compared to last year, or as compared to other relevant modules. It is only too easy for a lecturer to be accused of poor marks, simply because, for example, the students opting to do the module concerned were, in general, weak students. Thus, it is clear that any appraisal of the results of university examinations must, necessarily go well beyond a mere reflex reaction of quoting a few means and standard deviations! Such a scenario is indicative of only one of many questions that can be asked about the assessment process but gives us the key to the most important aspect of *ARQUE*'s methodology

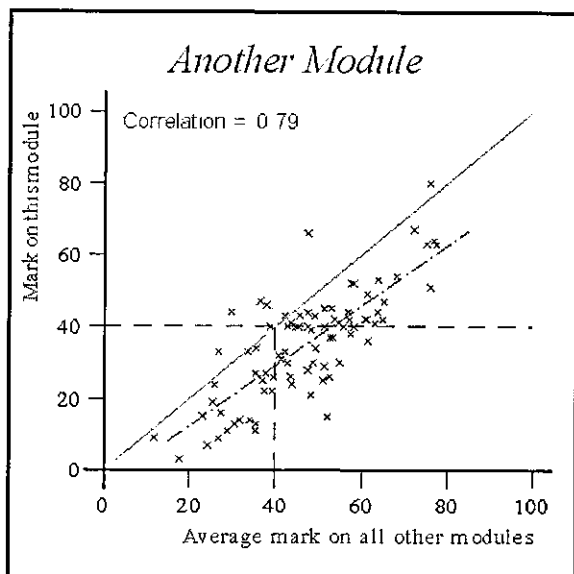
Comparing Marks on a Module with Average Marks on other Modules

By doing this, *ARQUE* can attempt to tease out some of the confounding effects on a set of module marks. Here is an example:



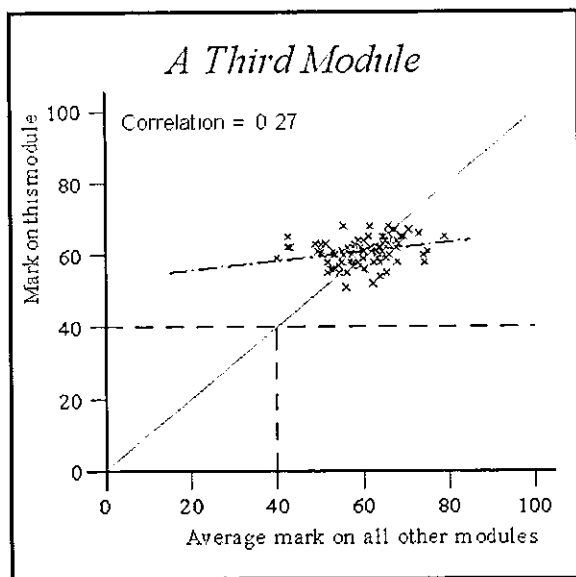
The diagonal line represents parity between mark on this module and average marks on other modules. The regression line here is parallel to this line, but above it, by 14, so that one can say, on average, on this module, students are scoring approximately 14 marks more than one might expect.

Of course, the regression line could lie parallel, but underneath the parity line:



so for this module, students are scoring on average, approximately 12 marks below what one might expect.

Other scenarios are possible, as the regression line need not be parallel to the parity line, indicating an assessment strategy that might, for example penalise the weaker students, or fail to discriminate between the weakest and strongest students in the class:

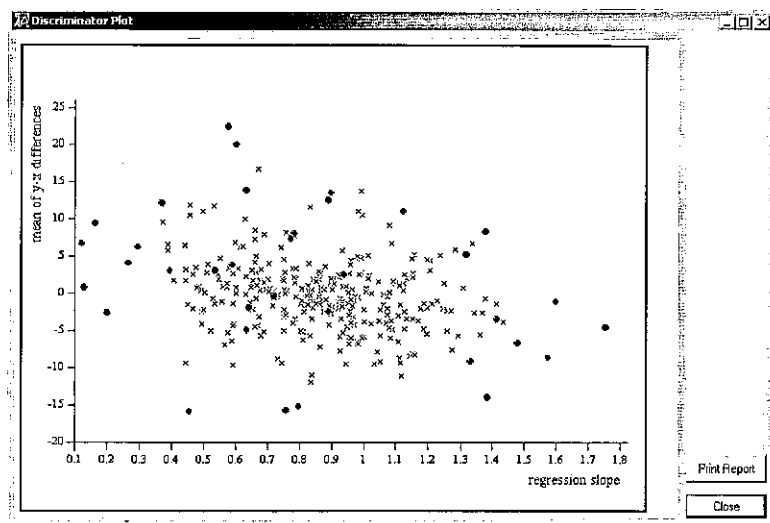


In an age when academics (at least in the UK) are under pressure to be more innovative in trying different methods of assessment, the radically different assessment profiles presented above (all genuine examples) make a strong case for substantial feedback from the examination results to the examiner, the department concerned and the university.

ARQUE's Top-Down Facilities

Whilst the top-down facilities are the latest additions to the facilities, it seems sensible to follow the motivation above, by examples of how one might view graphically, what is happening for the institution as a whole. Data for an institution are compiled from raw csv files and stored in APL component files to facilitate the production of reports as required. (For example, the examination results would be compiled by ARQUE from a csv file each of whose lines present information from one student-examination – i.e. *student identity*, *module taken* and *module mark* obtained.)

A data-mining facility is provided that enables you to look at different characteristics of the examination process across the institution. For example, the following plots the mean difference between average mark and average mark on other modules (basically measuring how far above or below the parity line, the regression line is) against the slope of the regression line:-



Here, every point represents one examination – the points in red indicate modules with an assessment profile that is extreme on one or more (or a combination of) characteristics. Specifically, points which are high on the vertical axis indicate a profile where students gain a large number of marks over and above their average on other modules (as for example 1 above), whereas a point with a very low value on the horizontal axis indicates an assessment profile similar to example 3 above.

Such data-mining reports give an insight to the amount of variation in assessment profiles across the institution and facilitate the setting of critical values which can then be used in the provision of an institutional report, consisting of lists of modules that are extreme for one reason or another.

Individual Module Reports

The original motivation for writing *ARQUE* came from my department trying to understand as much as possible as to what was happening in a busy department and to give individual lecturers as much feedback as possible on how their assessment strategy had worked. An individual module report may be produced for each examination in an institution. Such a report comments on the salient features of the assessment profile, providing graphical and numerical information, together with a substantial written report (which may be in English or Welsh). These reports are, to my mind, the most important reports, as they provide the means whereby a lecturer can relate the content of the report with other information about the examination process (e.g. verbal comments from students

and other staff) and make, where necessary, important changes in future assessment strategies.

Other Reports

It is often useful to be able to compare performances on comparable modules, may be, for example, all first year modules in a department. Comparative reports of this nature are also available.

Using ARQUE

Currently, *ARQUE* is used in universities by central administrators who make appropriate reports available to departments after the examination process is complete. For a university of some 12000 students with over 2000 modules, this is feasible within a day, even if paper copies are supplied. Alternatively documents produced by *ARQUE* may be compiled in pdf form and transmitted electronically.

Some Technical Comments

As indicated in the introduction, I would not have been able to produce *ARQUE* without (a) APL, and (b) the Causeway products. The graphical user interface is a series of CPro forms, hence changes and enhancements are easily made. Reports are produced using Newleaf, and all charts are constructed using Rainpro. Customers are impressed with the quality of graphical information, and the speed with which it all works. The written information is compiled with the aid of a dictionary (two-column matrix) with English statements in the first column, and Welsh statements in the second.

The Future

Obviously, I hope to sell more copies as I have the time and energy to persuade universities that they need the facility. From an APL perspective, I envisage that in the future, the facilities of *ARQUE* will be made available via a University Intranet – how this should be done is yet to be determined.

Alan Sykes
Director of Acadvent Ltd
Author of *ARQUE*

Retired Senior Lecturer in Statistics in the Business School of the University of Wales, Swansea.

Statistical Methods Discovered by the Power of APL

by Peter-Michael Hager (email: hager@dortmund.net)

Through its array functionality APL can treat masses of data, piled up in huge spreadsheets or databases, much more intelligently to generate meaningful perceptions, rather than just doing primitive operations such as column reductions.

Odd Man Out

Spotting the 'odd man out', the exceptional figure in a mass of numbers, is a recurring task of management. A simple APL technique finds and highlights anomalous figures in arrays, where human eyes are likely to overlook them. By using these results to add colour, the tables immediately gain visual appeal.

In magazines there are sometimes intelligence tests with questions like this:

Which number does not belong?

1001 996 1000 1005 2004 998 997 999 1004 1003

Table 1: Magazine's level 1 IQ question

Of course, it is easy to see that one number is twice as big as the others. Anyway, for us this indeed is a question APL can solve:

```

      IQuestion
1001 996 1000 1005 2004 998 997 999 1004 1003

      0*Normalize IQuestion
1 1 1 1 2 1 1 1 1 1

```

All we need for that is a little function which will average the values to 1. The rounding to whole numbers will then do the rest:

```
Normalize←{(ω×ρω)÷+÷|ω}
```

You do not think it should have been necessary to ask APL for a solution? Then let's take on the next level in the contest.

Which number does not belong?

507	364	1196	689	611	1235	78	1001	1014	1079
78	56	184	106	94	190	12	154	156	166
2691	1932	6348	3657	3243	6555	414	5313	5382	5727
3393	2436	8004	4611	4089	8265	522	6699	6786	7221
2457	1764	5796	3339	5922	5985	378	4851	4914	5229
2886	2072	6808	3922	3478	7030	444	5698	5772	6142
2847	2044	6716	3869	3431	6935	438	5621	5694	6059
3900	2800	9200	5300	4700	9500	600	7700	7800	8300
3471	2492	8188	4717	4183	8455	534	6853	6942	7387
936	672	2208	1272	1128	2280	144	1848	1872	1992

Table 2: Magazine's level 2 IQ question

I hope you could find the non-matching number! Again APL can do it for you:

```

Sales←(10?100)∘.×(10?100)
Sales[5;5]×+2
Sales
507 364 1196 689 611 1235 78 1001 1014 1079
78 56 184 106 94 190 12 154 156 166
2691 1932 6348 3657 3243 6555 414 5313 5382 5727
3393 2436 8004 4611 4089 8265 522 6699 6786 7221
2457 1764 5796 3339 5922 5985 378 4851 4914 5229
2886 2072 6808 3922 3478 7030 444 5698 5772 6142
2847 2044 6716 3869 3431 6935 438 5621 5694 6059
3900 2800 9200 5300 4700 9500 600 7700 7800 8300
3471 2492 8188 4717 4183 8455 534 6853 6942 7387
936 672 2208 1272 1128 2280 144 1848 1872 1992
0*Normalize Sales
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 2 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1

```

Imagine that the numbers in *Sales* are the revenues of item groups in several regions. Then this function will show, by colouring a table, that there were far more jackets sold in Germany, when compared to other regions. It would have been difficult to get that information just by studying the numbers.

Region\Item	Pants	Jeans	Shirts	Skirts	Jackets	Suits	Swimsuits	Slippers	Sneaker	Socks
Austria	507	364	1196	689	611	1235	78	1001	1014	1079
Belgium	78	56	184	106	94	190	12	154	156	166
Denmark	2691	1932	6348	3657	3243	6555	414	5313	5382	5727
France	3393	2436	8004	4611	4089	8265	522	6699	6786	7221
Germany	2457	1764	5746	3339	5922	5985	375	4851	4914	5229
Great Britain	2886	2072	6808	3922	3478	7030	444	5698	5772	6142
Holland	2847	2044	6716	3869	3431	6935	438	5621	5694	6059
Italy	3900	2800	9200	5300	4700	9500	600	7700	7800	8300
Spain	3471	2492	8188	4717	4103	8455	534	6853	6942	7387
Sweden	936	672	2208	1272	1128	2280	144	1848	1872	1992

Table 3: Sales chart

To handle similar variables of higher rank, it is necessary to extend the *Normalize* function. Now it will average one dimension after another. As averaging a further dimension changes the result of previously averaged dimensions, this needs to be iterated several times until the result becomes stable. Fortunately this algorithm converges with any number of dimensions.

```

Normalize={ A Normalize an array of arbitrary rank - Dyalog V11 version
  AxialNormalize+{(w*a>pw)÷[(1ppw)~a]÷/[a]|w}
  LeftChained+{>aα/(φa),cω}
  Preprocess+{w+1E~180×0=ω}
  Postprocess+{w×1E~160<|w}
  Postprocess((1ppw)AxialNormalize LeftChained*≡Preprocess ω
}

Normalize={ A Normalize an array of arbitrary rank - Compatible version
  AxialNormalize+{(w*a>pw)÷[(1ppw)~a]÷/[a]|w}
  LeftChained+{>aα/(φa),cω}
  Until-{w ωw x+αα w:x ∅ ∇ x}
  Preprocess+{w+1E~180×0=ω}
  Postprocess+{w×1E~160<|w}
  Postprocess{(1ppw)AxialNormalize LeftChained ω}Until≡Preprocess ω
}

```

(The Preprocess and Postprocess functions are to avoid division by zero.)

This population chart illustrates the power of the method very clearly:

Age \ Year	1999	2000	2001	2002	2003
0	771223	766554	735755	719250	706449
1	788905	772940	772147	739547	722845
2	814425	789766	775918	773772	741235
3	799778	814674	792198	777811	775035
4	773303	799910	816966	794141	778756
5	782784	773740	802195	819000	794971
6	814607	782802	775663	803429	819102
7	833793	814593	784716	777074	803783
8	866345	834766	817082	786980	778263
9	954074	867444	837502	819359	788064
10	943186	955399	870230	840052	820427
11	970098	945001	958622	873253	841560
12	949410	972105	948491	961356	874970
13	933225	951793	975676	951535	963201
14	901858	935755	955683	979021	953449
15	900050	905014	940218	959420	981339
16	908955	904300	911412	945119	962594
17	936004	913630	910945	916538	948796
18	937447	942036	922226	917900	921679
19	950649	947452	955826	934459	927797
20	911786	963406	963199	969933	945735
21	908151	922857	978755	976442	980454
22	910085	920340	939322	992803	987005
23	905701	921590	936609	952985	1003813
24	890669	916064	936232	948860	962464
25	912320	898814	928552	946009	956147
26	931254	918245	909454	936263	951628
27	1029212	935974	927431	916169	940907
28	1144718	1032886	943227	933124	919707

29	1202052	1147340	1038983	947840	935548
30	1296324	1203587	1151707	1042534	949498
31	1362258	1296851	1207656	1154077	1043319
32	1398811	1362199	1299552	1209961	1154414
33	1439408	1398313	1364225	1300709	1209537
34	1450859	1438889	1399998	1364686	1300032
35	1477274	1450005	1440128	1400187	1363600
36	1469050	1476581	1450670	1439945	1398377
37	1424872	1468164	1476899	1450555	1438447
38	1404723	1424090	1468690	1476463	1448475
39	1368060	1403988	1424674	1468226	1474533
40	1326138	1367322	1404263	1424198	1466359
41	1261066	1325466	1367630	1403660	1422470
42	1238416	1260096	1325031	1366909	1401548
43	1210553	1237357	1259536	1324022	1364641
44	1177514	1208956	1236607	1258214	1321898
45	1161536	1176030	1207862	1234722	1256051
46	1128557	1159795	1174673	1205707	1232037
47	1133521	1126482	1157765	1172503	1202886
48	1113454	1131138	1124315	1155464	1169433
49	1123726	1110862	1128577	1121436	1152162
50	1087755	1120698	1108022	1125457	1117790
51	1003191	1084091	1117385	1104667	1121792
52	947557	999443	1080585	1113405	1100655
53	834565	943614	995557	1076523	1108775
54	742176	830302	939581	991293	1071635
55	987985	737949	826037	934937	986273
56	1006682	981853	733724	821345	929925
57	981990	999971	975354	729066	816200
58	1189781	975077	992976	968455	
59	1258458	1181002	968067	985523	960921
60	1235197	1247770	1171255	959970	976976
61	1153916	1223925	1236424	1161033	951271

62	1078082	1142454	1212233	1224590	1150029
63	1049741	1066784	1130839	1199711	1211778
64	1012960	1037232	1054855	1118264	1186400
65	940888	999448	1024140	1042034	1104720
66	757258	926714	985463	1010252	1028050
67	752064	744532	912376	970422	995364
68	762724	738522	731607	896637	954207
69	799984	747726	724474	717779	880013
70	770179	782641	732099	709693	703388
71	758324	752048	764629	715508	693668
72	695572	738595	732999	745642	697782
73	673646	676352	718511	712768	725229
74	655313	653544	656444	696700	691416
75	595573	633548	632558	634908	673450
76	568893	574342	610897	610122	613167
77	573642	545926	551956	587216	586192
78	574718	547767	521892	528378	561897
79	537637	545651	520874	496753	503089
80	384281	506703	515478	492383	469764
81	238615	360199	475438	483677	462122
82	144722	203166	335240	442751	450653
83	214385	193387	201446	309447	408189
84	255958	195477	177109	184069	282744
85+	1635134	1612696	1540092	1451680	1374891

Table 4: German population chart

Differences between Products

The second sample will show you how to quantify the reasons for deviations in calculations. Though a standard solution already exists, by doing APL examinations we quickly find out that there is something wrong with it, and we can come to more meaningful and realistic results.

A pre-calculation in its simplest form consists of the addition of item expenses, each being a product of price $\times$ quantity. These can then be compared by a post-calculation against the actual prices and quantities.

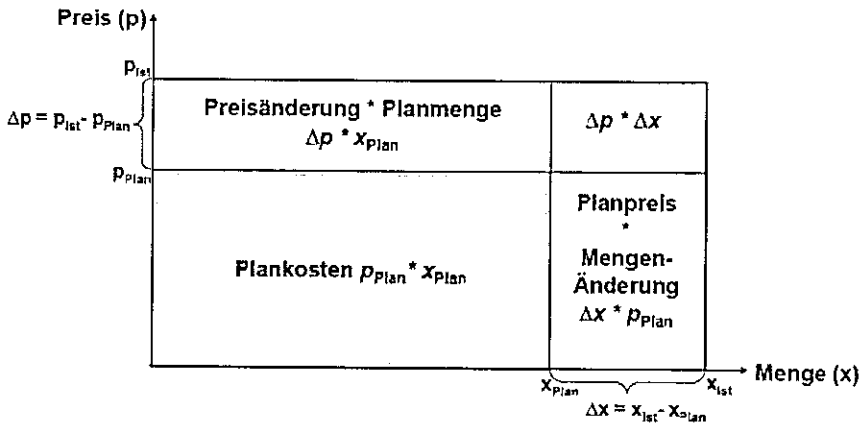
The differences between pre- and post-calculation should desirably be expressible as components of the costs, as only those can get summarized.

Item	Pre-Calculation			Post-Calculation			Diff.
	Price	Qty.	Costs	Price	Qty.	Costs	
Bricks	0.75 €	165 pc	123.75 €	0.80 €	135 pc	108.00 €	-15.75 €
Cement	2.50 €	5.0 lb	12.50 €	2.70 €	6.0 lb	16.20 €	3.70 €
Sand	0.30 €	2.0 lb	0.60 €	0.40 €	30.0 lb	12.00 €	11.40 €
Total			136.85 €			136.20 €	-0.65 €

Table 5: Bricklayer's calculation

To determine the reasons for the differences, we need to do a more detailed analysis on a per item basis. In the German literature about management-economics I found the chart below to explain the concept. (Sorry about not having found it in my English literature [1].)

The pre-calculated costs of an item are represented by the grey field. The difference to the post-calculated cost consists of three fields: one resulting from the difference in the price; one from the difference in quantity; and a third one is called mixed deviation.



$$\begin{aligned} \Delta K &= K_{Ist} - K_{Plan} = p_{Ist} * x_{Ist} - p_{Plan} * x_{Plan} = \\ &= (\Delta p + p_{Plan}) * (\Delta x + x_{Plan}) - p_{Plan} * x_{Plan} = \\ &= \Delta p * x_{Plan} + \Delta x * p_{Plan} + \Delta p * \Delta x \end{aligned}$$

└─ 1. Grad (Preisabweichung) └─ 1. Grad (Mengenabweichung) └─ 2. Grad (Gemischte Abweichung)

[Ref: Zitzmann, Betriebliches Rechnungswesen, Plankosten (chapter 9, page 31), Berner Fachhochschule]

Differences in the prices are caused by inflation or other changes in the market. For differences in quantities the calculating person is responsible. Fortunately the third component, the mixed deviation, is not very much in this sample, so economists tend to neglect it.

Applied to the bricklayer sample, these differences may be separated into the three different causes:

Item	Pre-Calculation			Post-Calculation			Differences by		
	Price	Qty.	Costs	Price	Qty.	Costs	Price	Qty.	Mixed
Bricks	0.75 €	165 pc	123.75 €	0.80 €	135 pc	108.00 €	8.25 €	-22.50 €	-1.50 €
Cement	2.50 €	5.0 lb	12.50 €	2.70 €	6.0 lb	16.20 €	1.00 €	2.50 €	0.20 €
Sand	0.30 €	2.0 lb	0.60 €	0.40 €	30.0 lb	12.00 €	0.20 €	8.40 €	2.80 €
Total			136.85 €			136.20 €	9.45 €	-11.60 €	1.50 €

Table 6: Bricklayer’s calculation with 1st grade and 2nd grade deviations

Though the total differences in the first table were as low as 65¢, they are now exposed as being 9.45€ and 11.60€, and furthermore having a mixed difference of 1.50€. When exchanging the pre- and post-calculation values, strangely not only the deviations' signs change, but also their values:

Item	Post-Calculation			Pre-Calculation			Differences by		
	Price	Qty.	Costs	Price	Qty.	Costs	Price	Qty.	Mixed
Bricks	0.80 €	135 pc	108.00 €	0.75 €	165 pc	123.75 €	-6.75 €	24.00 €	-1.50 €
Cement	2.70 €	6.0 lb	16.20 €	2.50 €	5.0 lb	12.50 €	-1.20 €	-2.70 €	0.20 €
Sand	0.40 €	30.0 lb	12.00 €	0.30 €	2.0 lb	0.60 €	-3.00 €	-11.20 €	2.80 €
Total			136.20 €			136.85 €	-10.95 €	10.10 €	1.50 €

Table 7: Same as Table 6 with exchanged values for pre- and post-calculation

At this point, one began to have doubts about having found an appropriate solution for the problem. The situation worsens drastically when item costs are the result of three factors:

$$\text{Costs} = \text{Price} \times \text{Quantity} \times \text{Rebate Factor} [2]$$

In such cases you will get:

- One deviation based on price difference,
- One deviation based on quantity difference,
- One deviation based on rebate difference and
- Four mixed deviation components.

As it is not unusual that the sum of these four mixed components will be bigger than any of the three meaningful components, the usability of this method becomes rather questionable. So we must search for a better strategy.

To help you follow the quite terse functions, I will explain the resulting strategy for two factors first.

These are the conditions:

$$a \times b + x + y = d \times e$$

$$a \times b + y = a \times e$$

$$a \times b + x = d \times b$$

What we further need, are these two factors...

$$p = (b \times d)^E$$

$$q = (a \times e)^E$$

...to offer this solution:

$$x = (d - a) \times (b + (e - b) \times p / (p + q))$$

$$y = (e - b) \times (a + (d - a) \times q / (p + q))$$

With the exponent $E=0$, the mixed deviation is cut into halves:

$$x = (d - a) \times (b + (e - b)/2)$$

$$y = (e - b) \times (a + (d - a)/2)$$

With $E=1$, the bigger causer gets the bigger part of a deviation:

$$x = (d - a) \times (b + (e - b) \times (b \times d) / ((b \times d) + (a \times e)))$$

$$y = (e - b) \times (a + (d - a) \times (a \times e) / ((b \times d) + (a \times e)))$$

Here is the same as a dynamic function:

```
ProDiff←{E←1
  b←φa+α+1E-100
  d←ω+1E-100
  h←φg+d-a
  q←φp←(d×b)*E
  g×b+(h×p)÷p+q}
```

The additions in lines 2 and 3 are required to avoid division by zero.

These are the calculation results where the deviations are separated according to their origins.

Item	Pre-Calculation			Post-Calculation			Differences by	
	Price	Qty.	Costs	Price	Qty.	Costs	Price	Qty.
Bricks	0.75 €	165 pc	123.75 €	0.80 €	135 pc	108.00 €	7.50 €	-23.25 €
Cement	2.50 €	5.0 lb	12.50 €	2.70 €	6.0 lb	16.20 €	1.10 €	2.60 €
Sand	0.30 €	2.0 lb	0.60 €	0.40 €	30.0 lb	12.00 €	1.60 €	9.80 €
Total			136.85 €			136.20 €	10.20 €	-10.85 €

Table 8: Bricklayer's calculation of two factors with $E=0$
(mixed component shared equally)

Item	Pre-Calculation			Post-Calculation			Differences by	
	Price	Qty.	Costs	Price	Qty.	Costs	Price	Qty.
Bricks	0.75 €	165 pc	123.75 €	0.80 €	135 pc	108.00 €	7.40 €	-23.15 €
Cement	2.50 €	5.0 lb	12.50 €	2.70 €	6.0 lb	16.20 €	1.09 €	2.61 €
Sand	0.30 €	2.0 lb	0.60 €	0.40 €	30.0 lb	12.00 €	0.43 €	10.97 €
Total			136.85 €			136.20 €	8.92 €	-9.57 €

Table 9: Bricklayer's calculation of two factors with E=1
(appropriately shared to its causers)

With the exponent set to E=1 we get the best matching results, as it shares the mixed deviation component into parts appropriate to its cause.

As previously mentioned, these are calculation samples consisting of three factors. The four mixed deviations could get removed, and the calculation concept is symmetric. [3]

Item	Pre-Calculation				Post-Calculation				Differences by		
	Price	Qty.	Disc.	Costs	Price	Qty.	Disc	Costs	Price	Qty.	Disc.
Bricks	0.75	165	-12 %	108.90	0.80	135	-16 %	101.52	6.82	-21.17	6.97
Cement	2.50	5.0	-5 %	11.88	2.70	6.0	-16 %	15.23	1.04	2.46	-0.14
Sand	0.30	2.0	-20 %	0.48	0.40	30.0	0 %	12.00	1.49	8.87	1.17
Total				121.26				128.75	9.34	-9.84	-7.99

Table 10: Bricklayer's calculation of three factors with E=0
(mixed component shared equally)

Item	Pre-Calculation				Post-Calculation				Differences by		
	Price	Qty.	Disc.	Costs	Price	Qty.	Disc.	Costs	Price	Qty.	Disc.
Bricks	0.75	165	-12 %	108.90	0.80	135	-16 %	101.52	6.73	-20.98	6.88
Cement	2.50	5.0	-5 %	11.88	2.70	6.0	-16 %	15.23	1.03	2.46	-0.14
Sand	0.30	2.0	-20 %	0.48	0.40	30.0	0 %	12.00	0.41	10.81	0.31
Total				121.26				128.75	8.17	-7.72	7.04

Table 11: Bricklayer's calculation of three factors with E=1
(appropriately shared to its causer)

In detail, 8.17€ of the difference are inflation based, -7.72€ are based upon the calculator's fault, and 7.04€ are the result of a purchase manager who failed to ask for the discounts.

```

ProDiff+{ A Distribute differences to their cause
E+1
2=θρφρω:α{
  b+φa+α+1E-100
  d+ω+1E-100
  h+φg+d-a
  q+φp+(d×b)*E
  g×b+(h×p)÷p+q}ω
3=θρφρω:α{
  c+1φb+1φa+α+1E-100
  d+ω+1E-100
  i+1φh+1φg+d-a
  r+1φq+1φp+a×E
  s+d×E
  pt+1φrs+r×s
  pu+2φqs+q×s
  puq+1φrtp+1φqsr+qs×r
  g×(b×c)+((h×c×qs)÷qs+pt)
  +((i×b×rs)÷rs+pu)
  +(h×i×qsr)÷qsr+rtp+puq}ω
ω-α
}

```



```

      8 2*PreCalc
0.75 165.00    0.88
2.50   5.00    0.95
0.30   2.00    0.80
      8 2*PostCalc
0.80 135.00    0.94
2.70   6.00    0.94
0.40  30.00    1.00
      8 2*PreCalc ProDiff PostCalc
6.73 -20.98    6.88
1.03   2.46   -0.14
0.41  10.81    0.31

```

Footnotes

- [1] Some translations: Preis = price; Menge = quantity; Preisänderung = price change; Planmenge = planned quantity; Planpreis = planned price; Plankosten = planned costs; Preisabweichung = deviation by price; Grad = grade; Mengenabweichung = deviation by quantity; Gemischte Abweichung = mixed deviation.
- [2] Though a discount is usually declared as a percentage, it is easy to transform into a multipliable component. e.g. 3.5% corresponds to a rebate factor of 0.965.
- [3] This means, when exchanging the left and the right argument, the result will change its sign, not its values. When having three factors, the APL solution on the next page does not fulfil this requirement completely, because the full solution would be an iterative formula, which has been terminated at an acceptable precision.

October 2005

Peter-Michael Hager
 HAGER-ELECTRONICS GmbH
 Hamburger Str. 97, 44135 Dortmund, Germany
 email: hager@dortmund.net

Pair Programming With The Users

by Stephen Taylor (slt@5jt.com)

Written from the notes for my talk at the Dyalog Users' Meeting.

We have two kinds of delegate at this meeting. Some of us have used APL to write software which we use to offer goods and services. A while back Stefano Lanzavecchia was quoted as saying while he didn't think APL had anything still to teach computer science, it was still the best language to use if you wanted to get rich.

How many people do you know who have made a million dollars or more with software they have written themselves – and how many of them wrote in APL? The prospects for entrepreneurs like these remain as bright as ever. But not for the other kind of delegate here.

The other kind of delegate here is someone like me, who likes to be paid to write software in APL. For us, the decline in the popularity of the APLs jeopardises how we like to make a living. And that is what I want to talk about today.

Two years ago at the meeting in East Horsley I argued that our best prospects are closely aligned with the movement for agile systems development, sometimes called Extreme Programming or just XP. Today I want to report how that future is emerging in the work I'm doing with Optima Systems at one of the UK's largest life and pensions offices, and I'll review what has and hasn't worked in our systems development there.

I'm also going to discuss a new paradigm for developing software, an alternative to the analogy with civil engineering popularised by the Software Engineering movement; because how we talk and think about what we're doing matters far more than we usually realise.

Finally, I'll advance a new metric, *semantic density*, relating to the clarity of source code, and I'll discuss its importance to the agility of our software development.

Coming Back

Two years ago, our rôle at the pension company had an unpleasant familiarity. From several mergers the company had inherited APL systems developed over the previous 15 years. Mergers are always predicated on economies of scale to be obtained from rationalising administrative work and the computer systems that

support it. Such strategies require great faith that yet unidentified problems will be solved within the chosen technical strategy. In this world of faith, a technology that is not part of the solution is part of the problem.

Contemporary conversations about IT strategy provide no place for development in APL. It followed inexorably that our systems were integration problems and scheduled for replacement by 'strategic technology'. Here we see the cost to us of ceding our claim to an important rôle for APL in the development of key systems.

Another familiar aspect of our position was that our systems proved particularly resistant to replacement. Manager after manager compared the cost of maintaining our systems with the cost of replacing them and concluded he had more pressing business.

As other legacy systems were wound up, the APL team languished in the exit lounge. We wanted to break out and win recognition as a useful resource for new systems development. But, as many other APL teams have found, a record of unmatched productivity is not enough to overcome the unblinking faith required by a corporate integration strategy.

We promoted ourselves as writers of tactical systems, interim solutions, stopgaps to be used until corporate strategy bore its long-awaited fruit. Eventually we found a senior manager with new responsibilities who needed results badly enough to give us a break.

The opportunity he gave us was to help him process pension claims. This is work of a complexity that Dickens would have admired. Pension funds have suffered decades of changes in products, legislation, regulation and tax law. When the time comes for a policyholder to retire and convert savings into a pension, someone has to unwind all this, ensuring that restrictions placed on rights vested in certain parts of the funds are respected. Two years ago this process took about an hour a policy. It took six weeks to train a new clerk to process the simpler claims, and as many months before she could process all the possible variations. Managers were largely unable to switch staff to tackle backlogs, because cross-training took three weeks. The senior clerks had improvised all kinds of tools to support their work: spreadsheets, Word document templates, checklists – at least the quill pens were long gone.

We approached the work in a way that will be familiar to APL programmers, writing code alongside the end users. Two years later we have halved the average processing time, can get very simple claims processed in under ten minutes, and we have become the primary tool for 'business re-engineering' in the pensions division. We are now winning jobs from the IT division's Too Hard basket, and

have doubled the size of our team – large by APL standards, tiny by IT's – to handle the new work our customer is asking us to take on.

And for certain portions of its business, we are now the designated strategic solution.

What Worked, What Didn't

Something that failed from the beginning was analysis. You all know analysis. You have a conversation with the user, take notes, write a description of what the computer is supposed to do, have the user confirm it. Then you go write the program. None of this worked.

Our expert users are the senior clerks who train new clerks. They are the source for what is needed from the system; no one knows the work better than they do. They train new clerks to do the work, so you would suppose they can explain the requirements. Not so. They train new staff in procedures, not by explaining theoretical concepts, but with "do this, then do that" instructions. Staff do eventually work out for themselves the implications of what they're doing. It's just as well humans can learn this way, because the staff's linguistic skills don't allow them to talk and reason about the work with any precision or clarity. Over and over again our conversation floundered into self-contradictions and misunderstandings, unable to navigate through the many variables in play. I saw that even if I were able to write requirements I had confidence in, I had no hope of getting a useful review of them.

A radical change of approach was needed. We were working on the rules for determining whether a particular penalty was to be applied to a policy before benefits are taken. The checklist the clerks used to decide this covered five A4 pages. "Show me your first example," I said. We worked through it, with me copying numbers from the mainframe screens and defining variables and rules. Eventually I had a function that used the same facts she did, and produced the same answer. I saved the cluster of facts as my first test case and started modifying the function as we worked through her second example. When I had a second right answer I retested the first case and reworked the function until it produced both answers correctly. By the end of a fortnight we had working code and a test suite with forty cases, and I was starting to see the principles involved.

Over time I was able to refactor these rules for clarity, using the test suite to confirm I was still getting the same results. The rules written in APL now cover about half an A4 sheet; that's one order of magnitude smaller than the checklist in English. As far as I know it is the only formal specification of the penalty rules the

organisation possesses. And – this is the kicker – it's maintained jointly by the APL programmers and the clerks. It's been some time now since we found a case for which we needed to modify the rules; but when we do so, we jointly trace execution, examining values, to identify where the machine's logic varies from the clerk's. We correct the rule – the programmer modifies it and the clerk confirms the new code appears to match her meaning – then execute it to confirm it now yields the right answer.

Extreme Programmers work in pairs to produce immediate feedback on code quality, and get user feedback on a 2–3 week release cycle.

We work in pairs with the users to get *immediate* feedback on the requirements. When the specification's confirmed, the programming is finished, for the specification is executable. Collaborating with system users on an executable description of the system's behaviour is a key breakthrough in our work. We call it 'Pair programming with the user'.

Another hard-learned lesson concerned collaborating with the internal IT division. No one in the business has been able to grasp the speed and flexibility our work requires except the people participating with us. Where conventional IT delivers in weeks and months, we deliver in hours and days. Usually we can write an executable description of system behaviour that solves the business problem (ie a program) faster than an analyst can write a system specification. That time includes generous rewriting as we get user feedback of detail, quantity and quality largely unavailable to the analyst. As a result, our programs are a better fit to the business needs than any formal specifications we've seen the organisation write. Ever.

In this work it makes little economic sense to plan ahead much. This makes our progress hard to predict. We rarely know how complex or simple a job is until we've finished it. This is madness, of course. The only justification for it is that we progress so much faster. Our customer's managers naturally hate this unpredictability, but consistently prefer our rapid progress to predictable delay.

Making Movies

There are in this naughty world programs that have to run exactly right the first time – ask NASA. Arguably, the programs we are writing for the pension company fall far away from those on a scale defined in terms of cost of errors, complexity of function, stability of requirements and cost of delay.

You might suppose that the NASA-style project best represents general systems development, and that the conventional 'engineering' approach of 'plan it, then

build it right' is the best way to work. You might allow that what we are doing makes sense, but only in our unusual situation, and that few general lessons may be drawn from it.

For a moment consider instead the possibility that some of the more unusual features of our project merely shed strong light upon common aspects of software development that the engineering approach does not avoid but only masks.

That would be interesting if it were true, because we have other reasons to suspect the foundations of Software Engineering. SE was adopted in the 1980s in response to a widely-shared view that most software projects were out of control. Writers like Boehm¹ and deMarco² drew unfavourable comparisons between programmers and civil engineers and proposed we would do better if we could adopt engineers' methods. Thus Software Engineering and the rise of the formal methodologies.

SE has had some success at making software projects more predictable; but the modesty of these successes has to be contrasted with the effort expended upon them. When intelligent people work long and hard at something for little result, we are entitled – perhaps obliged – to ask if the problem has not been misconceived. More recently, some writers have argued the analogy with civil engineering has been misapplied from the beginning.

It is a standard strategy of industrialisation to conserve expensive intellectual skills by defining simple and repetitive jobs that can be done with minimum skill. So, in building a bridge, expensive engineers develop plans, then cheaper workers pour concrete and weld steel. In the SE model, analysts and designers develop specifications and database designs, which are implemented by programmers. In SE, programming is the analogue of construction; and programmers the navvies of IT.

Ward Cunningham³ points out important differences in the material properties of bridges and software. Erecting steel and concrete is slow, and expensive to modify. Compiling and copying software is not.

A computer program is an executable description of the behaviour of an imaginary machine. 'Executable' means that a Turing Machine can read the program and emulate the behaviour of the imagined machine. Given plenty of PCs and a program, construction of the imagined machine is nearly immediate and free.

Cunningham argues that compilation is the analogue of construction, not programming. The rest of the work – developing the software – is all design work.

It can be done in pretty much any order (like drawing the bridge and unlike erecting it) and can be changed or redone relatively cheaply.

This view of software development casts doubt on the industrialisation project itself. No architect would try to analyse his architectural practice into simple and repetitive tasks. Perhaps industrialising the writing of software is similarly inappropriate?

What other processes could we use as a model? Another example of a collaborative writing projects, is making a movie. A producer friend of mine in Hollywood claims my software projects sound more like her movie projects than engineering work. Movie-making turns out to have important differences from writing software too, but also enough similarities to make comparison instructive.

A movie project starts with an initial idea and finishes with a film master, from which copies can be made without limit for distribution. The serious money is spent during 'principal photography'. Until then the work consists of developing increasingly specific descriptions of the movie.

The first description is usually very short ("Godzilla Meets Bambi!"), and does not need writing down, much as a director might tell his board, "We need a new order-entry system!" More elaborate descriptions follow in succession: treatment, screenplay and shooting script. Sponsors work on these descriptions to satisfy themselves the imagined movie will meet their own demands. By the time the shooting script is ready it incorporates considerable feedback from participants concerned about finance and marketing.

In a shooting script all the scenes and dialogue have been defined. The director has what he needs, but may still make extensive changes during photography. (My friend: "A script? That's what you use to get funding.") Scenes the director has imagined in detail sometimes fail to work despite many takes, and get rewritten. Within the constraints of the shooting schedule and the actors' availability, new problems get identified and solved, scenes get reshot to reflect changes made elsewhere. Then all the film goes to the cutting room for further revisions.

Movies are like our software project in incorporating lots of feedback during their making. Design continues into the cutting room and after first screenings, the 'sneak previews'. They are unlike our software project in that design stops after release, while our and other agile projects release as little as possible as early as possible, and keep modifying it as long as people think the effort worth paying for.

Both movies and software projects start off by writing increasingly elaborate descriptions of the project. In movie projects this process ends with principal photography. The project continues to evolve in the director's hands, watched anxiously by his producer, but no more descriptions are written.

In building a bridge the same moment arrives when the engineers stop drawing and the workers start pouring and welding. In the engineering view of software development, that moment arrives when coding starts. It's the time to stop describing and start making. And it is the perennial complaint of the programmers that they are never left alone to write the described program. If you imagine yourself as a construction worker who finds himself repeatedly tearing down what he has just built to erect something like it nearby, you have a sense of the programmer's frustration.

Why *aren't* the programmers left alone to do their job? The comparison with making movies is instructive. In both fields the projects develop a succession of increasingly elaborate descriptions of the work, using them to imagine the end product in increasing detail and to communicate about it. But the movie people can understand their descriptions, right down to the shooting script, and that difference is everything.

Of course movie people can read a shooting script. They're movie people! They can read a screenplay, envisage the result and write a reliable estimate of production costs on the back of an envelope. All the descriptions of the movie allow the sponsors to communicate about the end product.

Nothing like this is true in software. When the director tells his board, "We need a new order-entry system!" he knows what he is talking about. By the time a fat binder of 'requirements' has been through the ritual of 'sign-off', communication has stopped, for the project description has escaped the language of the managers. The requirements now consist of formal specifications they cannot understand, plus reassurances such as "user responses will be sub-second or better". Unlike the movie producers, the system sponsors cannot read the description and reliably envisage the end result.

"Whereof I cannot speak, thereof I must be silent."

Ludwig Wittgenstein

There are Two Great Lies of Software Development, which sponsors and developers tell to each other. The first lie is "We can tell you what we need." The second lie is "We can tell you what it will take to produce."

Both lies enclose a kernel of truth. The sponsors *do* know a lot about what they need. The developers *do* have a lot of useful experience of writing. Sponsors and developers tell the Two Great Lies to each other because they think it shameful not to know these things completely.

The reality is that they don't. Canny project managers allow for this and treat system requirements as a shooting script – something you use to get funding. Sponsors cheerfully accept breathtaking variation from the specification provided the needs of the business are met. The unspoken agreement that underlies the contract is that the developer will meet the needs of the business in something like the time estimated, and at something like the projected cost; and it is this unspoken agreement that provides the context for the project manager's work.

Movie sponsors have feedback loops in their development of the movie that are missing from the initial development of a software project. Where does this feedback show up instead? It shows up when the software is first delivered. This precipitates the usual argument. The customer objects it doesn't meet his business need, and the developer replies that it exactly fits the description the customer signed. This conversation proceeds to a discussion of what change can be made to save the day, and who will pay for it, either as an extension (customer) or correction (developer).

The hard fact here is that it is very difficult to envisage the behaviour of an imaginary computer system; it is also very hard to describe. Of course, it is also hard to imagine and describe a movie, but these are skills movie people cultivate. Business people do not cultivate skills in imagining and describing computer systems.

Suppose the near-impossible: you have a customer who can envisage in unlimited detail the behaviour of a computer system he needs – he can give consistent answers to all questions about it. How would he describe it to you? Not using formal methods, for he is a businessman, not a developer. He must use informal, ambiguous, flexible, elusive English. He might well have all the detail (*pace* Wittgenstein) 'in his head', but he will never communicate all of it to you. Unlike the movie people, you don't share a common notation.

In this light we can begin to distinguish some interesting elements of our recent work on pension claims. In true XP style, we are always modifying a running system. We do not rely on imagination to carry us far beyond that. We generally discuss only small changes to observable behaviour.

My personal practice in this regard has become somewhat extreme. The clerks who guide our work no longer write even emails to me about changes they want.

To get a change, they come and sit beside me. They used to begin by describing the change they want, until recently I noticed what a waste of time this usually is, and put a stop to it. Now they begin by 'breaking' the system, either by showing me some way they've found to crash it or by getting it to behave in a way they want to change. Then we talk a bit. I'll use my knowledge of the system to locate where that behaviour is described and interrupt execution there. We trace execution and modify the code together as I wrote above.

Like the movie sponsors, we share a common notation for describing the end product. Our end product is not a movie, but the system behaviour. Our common notation is the source code. We are able to use the source code as a specification because of some special properties of the writing (which I'll come to shortly) and because the system animates it for us.

'Animates' is a better word here than the usual 'execute'. Recall that source code is nothing but a description of the behaviour of an imaginary machine. It differs from (e.g.) "an order-entry system" in being formal (so the machine can animate it) and more elaborate. (It also has a different semantics, but more of that later.) The same is true of the C code into which our source is translated, and arguably of the machine language into which that is compiled. This imagined machine is never built, only described; then that description is animated by the computer. It is the great virtue of computers that this is all we need: a description of a machine and a computer to animate it.

The computer animates our description, giving me and my user immediate feedback on what we have written, much as a movie director pores over 'rushes' at the end of a day's shooting. When we are satisfied, I save the modified system where she can test it further.

From this you can see we have collapsed the stages of analysis, specification, design, coding and testing into a short, uninterrupted dialogue with our user. Communication is high-bandwidth, face to face; feedback is rich and immediate.

You might suppose that in these conditions, and having worked together for over a year, communication is very good. Not at all. Misunderstandings are rife. Mercifully, the feedback quickly exposes them.

When we contemplate doing the same work through descriptions written in English, we shudder.

Just such description writing lies at the heart of the 'waterfall' development methodologies. (Perhaps they are really named for the cascade of non-executable system descriptions they produce.) From our perspective, conventional waterfall

projects look mostly like collaborative literary projects to produce documents that neither men nor machines can read effectively. Consider: a program is a machine-behaviour description that a computer can animate. How many descriptions do you want to produce that *can't* be animated?

An engineer who knows a component is expensive and unreliable uses it as little as she may. Communication (in English or other natural languages, without machine animation) is unreliable and expensive. Yet conventional methodologies employ it as if it were reliable and free. Worse, it is repeatedly advocated as the remedy for software projects in difficulty. Meetings abound. "We just need to ensure everything's documented and signed off," as if everything important could be pulled out of the changing world and fixed on the page. Well, some things can; but there are important limits. Perhaps software developers should study poetry more.

Semantic Density

"I gotta use words when I talk to you."

TS Eliot

What can we use to replace expensive and unreliable communication in English?

Our answer is to shorten the cascade of descriptions that starts with "an order-entry system" and finishes with the source code. Most software development methodologies call for maintaining and synchronising the entire sequence, so that changes at one level are reflected in the others. Many waterfall projects that subscribe to this in theory avoid it in practice. This is widely known as 'writing the documentation after the code'; and sometimes it is known as 'writing the specification after the code'.

Keeping the non-executable descriptions correct and in step is a lot of maintenance work, especially for descriptions that cannot be run on the computer, and which are very hard to validate. We should avoid as much of this work as we can. We admire the XP practice of destroying design artefacts after use. This prevents both misuse (when no longer accurate) and maintenance. But destroying design artefacts is practical only to the extent that the source code is readable.

Put it the other way around: improvements in the readability of source code permit order-of-magnitude reductions in the development work. In this last section I shall discuss a source of coding clarity. This extends ideas I first advanced in "Three Principles of Coding Clarity" (Vector 18.4)⁴.

Like many programmers seeing APL for the first time I was excited by seeing the expression

$$\text{SALES} \leftarrow +/ \text{PRICE} \times \text{QUANTITY}$$

where PRICE and QUANTITY were understood as table columns. No loops, no counters! I understood that their absence was important, but was unable to say why. People who felt the same way spoke of being able to "concentrate on the problem", and Ken Iverson wrote of the importance of "Notation As A Tool Of Thought"⁵.

We intuit that talk about sales, prices and quantities is distinct from talk about loops, counters and conditions. In "Three Principles..." I used the phrase *semantic consistency* to sharpen this distinction. The terms *sales*, *price* and *quantity* are drawn from the conversation of our customers. These terms refer to things our customers discuss: their referents are in the business world. The terms *loop*, *counter* and *condition* come from the conversation of programmers. These terms refer to things programmers discuss: their referents are confined to the world of program logic. We could say that if the tokens of a program refer to both the business world and the programming world, then it has two *reference schemes*, or *semantic domains*. And the exciting aspect of $\text{SALES} \leftarrow +/ \text{PRICE} \times \text{QUANTITY}$ is that it has only one.

One can quibble about this. The $\times$ function belongs to both domains. So does the $+$ function, but $+/$ is known only to APL programmers; while *sum* would belong to both. Syntactic elements such as assignment, if/then/else and select/case turn out in practice to be easily assimilated into the language of non-programmers. Linguistic philosophy warns us that individuation of semantic domains is likely to bear only limited weight; it is a 'loose and popular' usage.

But in practical terms the distinction of semantic domains is clear and useful. We can now measure *semantic density*: the proportion of writer-defined tokens in the semantic domain of the system user.

Our experience is that at high levels of semantic density (SD) we are able to share source code and 'pair program' with our business users in the way I have described. SD does not have to reach 100% to get this effect, but misunderstandings decrease sharply as SD approaches it.

The effect does not seem to be much diluted by the use of primitive APL functions. The functions $+$ and $\times$ are read as usual, and $+/$ is often read as *sum*. The assignment arrow is understood without explanation. Otherwise, most primitive functions and unnamed dynamic functions (Dyalog APL supports something like the lambda calculus) are simply ignored.

This is consistent with a distinction between close reading and skimming. A close reading requires attention to every detail; our users need to see only that (e.g.) the tax-free cash allowance is derived from the tables of assets and previous transfers and from nothing else.

A moderate use of control structures is helpful. Our users read if/then/else more easily than expressions which return a value as a function of the condition. But nesting control structures appears to degrade readability, as to a lesser extent does verbosity. Using IF/BUT is often a useful compromise.

[See Phil Last's more elaborate treatment of inline conditional structures in this issue. – Ed.]

```
MinAge ← 65 BUT 60 IF Sex='F'
```

The APL glyphs for the primitive logical functions have not become familiar enough to be read easily; so for shared code we define aliases: and, or, not. Unused to formal logic, business users require the most careful phrasing to keep logical structures clear.

Balancing these considerations is a matter of writing style, calling for nice aesthetic judgement. Programmers traditionally regard writing style as of small consequence. Our development practice puts writing style in the front line; illiteracy costs too much.

Using a local, task-specific vocabulary in the user's semantic domain...

```
asIDN←{dft 3p#.IDNToDate α α #.DateToIDN dtt ω}
daysAfter←+asIDN
daysBefore←-asIDN
oneOf←{(cα)∈ω}

all←^/ ♦ not←~ ♦ before←< ♦ after←>
not←~ ♦ (v/) ♦ and←^ ♦ or←v ♦ any←v/
```

we define some further terms in the business domain

```
isDeferred←OrigNRD≠Nrd
isBefore911←OrigNRD before 20040911
termOver5←OrigNRD≥5 yrsAfter StartDate
termUnder5←not termOver5
deferredBy5←Nrd≥5 yrsAfter OrigNRD
```

with which we can write business rules...

```
:If not InUWP
  answer+doesNotApply
:Else
  :If termUnder5 and RetirementAge≥75
    answer+refer

    :ElseIf RetirementAge≥75
      answer+doesNotApply

    :ElseIf ExitDate after oneMonthBefore 75 yrsAfter Dob
      answer+doesNotApply

    :ElseIf ExitDate before 183 daysBefore OrigNRD
      answer+any MVRs>0

    :ElseIf IsVista
      answer+ExitDate before ThreePolAnniversariesBefore
OrigNRD
```

NB. this is not actual production code, but merely resembles it.

Three-Wish Limit

I have argued against distinguishing software design from software construction.

What would software development look like if construction were immediate and free? Imagine that as soon as you had described what you want, you had it. That would be the world of fairy tales. Genies grant wishes for immediate delivery.

This is our world. When we and our users have worked out our description of what is needed, it is there.

In the stories there's a catch to the wishes, and it is always the same catch. You get what you ask for but it is never what you want. Often it is the very opposite. And there's a Three-Wish Limit. You get only three chances to describe what you want.

It is almost as if the old stories set out to warn us about the unreliability of communication. Did we stop listening when we grew up?

Unlike genies, we get paid, so there is no Three-Wish Limit. That matters, because working out what is needed from the system is too complex to get right in three tries.

You can think of conventional software development imposing an effective Three-Wish Limit through slow feedback. It takes so long to feed mistakes in the specification back to the designers: much work gets redone once; very little a third time. It takes a brave user or developer to propose a fourth try. So Three-Wish Limit becomes an expensive game of poker played between client and developer.

Our development work looks very unusual. At the end of most days, my work is complete. I have no outstanding work remaining, nothing on which to report status. I usually have a good idea of what we will work on the next day, but the users are free to move something else to the top of the list, and from time to time surprise me. They control the course of development.

The business users, knowing the complexities of what they want automated, are better than we are at predicting the time required. They often consult us in case we can see complications they don't; but we do not generally estimate task effort. Not having status to report, we don't often attend project meetings: our users plan and prioritise the work ahead without us.

Here's how that can play in practice. Fifteen specialist clerks in another part of the division had been processing pension claims in another city. Managers had speculated for some time that the work could be moved to our city and the APL system, but no resources could be spared from the company's business re-engineering project for analysing what it would take and specifying the system changes.

Finally two of our users visited the other city one Monday, and discovered that, as the managers had suspected, the work could be done using the APL system provided a few program changes were made. During their visit they announced they would be ready the following Monday to process the new claims. Since such migrations normally take months and sometimes fail entirely, it is hard to suppose they were entirely believed.

Back at work the next day, our users worked with us to change the APL system to handle the new work. By the end of Tuesday they had finished testing the changes, and they went into production. The following Monday, four clerks in our city smoothly took over the new business. All done.

What I love most about this story is that, while visiting the other city, our users never even phoned us.

Our customers never hear that what they want is "out of scope". We can't produce everything instantaneously, but they know we can produce what they

want about as fast as they can work out with us what that is. As in XP, we are always working on their top priority; and they can change priorities at any time.

In this work we depend on our ability, supported by APL, to write terse code we can share with our users.

References

1. *Software Engineering Economics*, B. Boehm, Prentice-Hall, 1981
2. *Controlling Software Projects: Management, Measurement and Estimation* T. deMarco, Prentice-Hall, 1982
3. *Keynote address*, W. Cunningham, Fourth International Conference on Extreme Programming and Agile Processes in Software Engineering, May 2003 (unpublished)
4. *"Three Principles of Coding Clarity"*, S. Taylor, Vector 18.4
5. *"Notation As A Tool Of Thought"*, K. Iverson, 1979 ACM Turing Award Lecture, Communications of the ACM, Vol.23, No.8, August 1980

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

Index to Advertisers

Bendyl Solutions LLC	3
Vector Back Numbers	142

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles to the Vector Working Group via the Editor:

The Editor, Vector
81 South Hill Park
London
NW3 2SS

Tel: +44 (0) 1439-788385
Email: editor@vector.org.uk

Please supply as much material as possible in machine-readable form. Our preferred format is HTML, using simple markup – we apply Vector's style sheet. Source code can be tagged with `<pre>` or `<code>` tags as usual. You can further distinguish language within these tags (which would be important if, for example, you were comparing J and APL expressions) with the class attribute. For example: `<code class="J">MEAN=:+/ %#</code>` would be typical. Authors wishing to use MS Word should install a copy of the APL385 Unicode TrueType font and use Unicode APL symbols if their interpreter permits 'copy as Unicode' from the APL session. Otherwise we can convert from Dyalog APL, APL+Win, APLX, and APL2 mappings if necessary.

Camera-ready artwork (e.g. advertisements) and 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ or emailed to gill@apl385.com.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2005/2006 Committee

Chairman	Paul Grosvenor 01293-562700 paul@optima-systems.co.uk	Optima House, Mill Court Spindle Way, CRAWLEY West Sussex, RH10 1TT
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stephen Taylor sjt@vector.org.uk	81 South Hill Park LONDON NW3 2SS
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Ian Clark earthspot2000@hotmail.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stephen Taylor	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852) Bob Hoekstra (01483-771028), Marc Griffiths, Ian Clark	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Carslaw Computer Systems Ltd
Brook House, Gilling East
York YO24 4YJ
Tel 01938 793443
Web: www.carslaw.com

Diagol Ltd
Greys Close, Wilmslow, Cheshire
Cheshire, M26 1JN
Tel 01256 333333
Email: sales@diagol.com
Web: www.diagol.com

Insight Systems Aps
Notre Dame, 1105
DK-4500, Vejle, Denmark
Tel: +45 76 26 19 26
Email: info@insight.dk
Web: www.insight.dk

MicroApl Ltd
The Royal Villa, Williams
Wrexham, Cheshire, Wrexham, Wrexham, Wrexham
Tel: 01925 763050
Email: MicroApl@microap.com
Web: www.microap.com

Compass Ltd
Gymnasia House
60 Polesay Road
Gillingford, Dorset, GU24 0JY
Tel: 01983 414500

APLnet Inc
One Research Court
Suite 111
Rockville MD 20850 USA
Tel: +1 800 208 7961
Email: sales@apl2000.com
Web: www.apl2000.com

APW Computing
Farnborough
Hants, GU14 7JF
Tel: 01489 490 499
Email: info@apw.com

Solitaire
47 Vantage Street, Suite 200
Farmingdale, NY 11737
Tel: 516 364 3855
Email: sales@solitaire.com
Web: www.solitaire.com

First Derivatives
First Derivatives, 100
Kilmorey Business Park, Kilmorey Street
Newry, Co Down, N. Ireland BT24 2JH
Tel: 028 3028 2242
Email: enquiries@firstderivatives.com

Optima Systems Ltd
Optima House, Mill Court, Spindle Way, Grayly,
West Sussex, BN11 1TH
Tel: 01293 662700
Email: paul@optima-systems.co.uk

APL Ireland
APL Ireland Ltd, 44, Wilton, City
Dublin 4, Ireland
Tel: 01 470 066 2080
Web: www.apl-ireland.com